

LEARNING TO THEORIZE IN MENTAL TERMS

MSc Thesis (*Afstudeerscriptie*)

written by

Gidon Kaminer

under the supervision of **Dr Fausto Carcassi**, and submitted to the
Examinations Board in partial fulfilment of the requirements for the degree of

MSc in Logic

at the *Universiteit van Amsterdam*.

Date of the public defence: Members of the Thesis Committee:

August 28th, 2026

Dr Fausto Carcassi (supervisor)

Dr Malvin Gatteringer (chair)

Dr Martha Lewis

Dr Giorgio Sbardolini



INSTITUTE FOR LOGIC, LANGUAGE AND COMPUTATION

Abstract

Contents

1	Theory of Mind and its Acquisition	5
1.1	The nature and scope of Theory of Mind	5
1.2	Experimental Evidence: Developmental Milestones	6
1.3	What a good account of ToM acquisition ought to explain	6
1.4	Theoretical perspectives on ToM acquisition	8
1.4.1	Nativism and core knowledge	8
1.4.2	Empiricism, association, and cultural learning	8
1.4.3	Rational constructivism and the theory theory	9
2	Learning as Bayesian Program Synthesis	12
2.1	The computational-level problem	12
2.2	Bayesian inference: the logic of learning	12
2.3	Programs as the hypothesis space	13
2.4	Program induction and the search problem	15
2.5	Learning the language itself	15
2.6	Our claim	17
2.6.1	What is not claimed	18
3	Bayesian Theory of Mind	19
3.1	Agent as planner	19
3.2	Inverse planning	19
3.3	Belief as an input	20
3.4	What is stipulated by the BToM modeler	21
4.	The Model	23
4.1.	The learning problem	23
4.2.	The task corpus	24
4.3.	Program space: type-directed enumeration	28
4.4.	The primitive library	29
4.4.1.	Composing belief attribution	31
4.4.2.	Where the decomposition stops	33
4.5.	Inference	34
4.5.1.	Search	34
4.5.2.	Library revision	35
4.5.3.	The two endowments	36
4.6.	The model does not encode a Bayesian theory of mind	37
4.6.1.	The space of admissible divergences	38
4.6.2.	The space of goals	38
4.6.3.	Rationality, and the one thing we do grant	39
4.6.4.	Two confounds that are ours rather than BToM's	39
4.6.5.	Two structural differences worth naming	40
4.7.	What would count as success or failure	40

5	Results	42
5.1	What the loop finds	42
5.1.1	The agency signature	45
5.2	The discovered term against BToM	46
5.3	The objective selected it	48
5.3.1	The corpus description length	48
5.3.2	What the library buys each family	49
5.3.3	No non-mental rival reproduces the scenes	49
5.4	Reachability, and the order in which things arrive	51
5.5	Generality and selectivity	52
5.5.1	The constructor spans the corpus	52
5.5.2	The symmetric complements go elsewhere	53
5.5.3	Attribution is selective	55
5.6	Scaffolding	56
5.7	The behavioural probe	57
5.8	The verdict	59
5.9	Objections	60
5.9.1	“fork and sync_to_world are just believe cut in half”	60
5.9.2	“The corpus was built to produce this result”	61
5.10	What the results do not show	61
6	Further work	64
6.1	Polymorphism	64
6.2	Channel arity as a free parameter	64
6.3	persistent models and interaction	65
6.4	breaking down optimize	65
6.5	probabilistic primitives	65
6.6	no sequencing	66
6.7	vs. real world	66
6.8	richer belief content	66
6.9	recursive mindreading	66
6.10	POMDP	66
7.	Appendix	67
7.1.	Full Corpus	68
7.2.	The BToM correspondence	70
7.3.	The libraries, verbatim	70
7.4.	The per-family description-length census	73
7.5.	The interpreter	74
7.6.	The primitive catalogue	75
7.7.	Program representation	79
7.7.1.	The library object	79
7.8.	Task generators and necessity filters	80

7.9. Cost-bounded enumeration	80
7.10. Compression with Stitch	81
7.10.1. Normalization before compression	82
7.10.2. No recognition model	83
Bibliography	84

Chapter 1

Theory of Mind and its Acquisition

1.1 The nature and scope of Theory of Mind

The ice cream peddler’s scratchy jingle got louder still as he brought his cart to a stop nearby. A child, hitherto engrossed in sandbox amusements, abandoned her playmates abruptly. She darted across the playground, hurtling at full speed towards a gaggle of parents. Absorbing the impact of a 20 kg cruise missile, the target continued his conversation unfazed by the girl now tugging at his jeans. Only when he deigned to look down at the child did he shriek and go pale. But the girl barely had time to react, because just then she heard her name shouted from the other side of the playground. She turned to its source and ran towards a similar-looking man whose outstretched arms held two ice cream cones.

You chuckle at the scene, observing from a nearby park bench. The girl, wanting ice cream, pestered a man she thought was her father. The man, undoubtedly accustomed to his own nagging kids, ignored the girl thinking it was his own child. Descriptions like these, in terms of desires, beliefs, and intentions, seem trivial and obvious. But there’s something miraculous about them. How are you able to peer into other peoples’ minds? Mental content is unobservable; all you see is a sequence of physical actions. And yet humans, even infants, are experts at inferring mental content rapidly and accurately.

Humans have the cognitive ability to impute mental states to others: to attribute a belief, a desire, or an intention to another person. This is the capacity for “theory of mind” (ToM), also known as “commonsense psychology”, “naive psychology”, “folk psychology”, “mindreading”, “mentalizing” (Margolis et al., 2012). We’ll primarily use the term Theory of Mind (ToM), to emphasize that this capacity is a *theory* proper in that it enables behavioral prediction by positing states that are not directly observable (Gopnik & Wellman, 1992).

The capacity for ToM is integral to our humanity: to cooperate with someone you have to understand what they think and what they want, to decode someone’s words you have to begin from what they are trying to do with them. Humans make these inferences so naturally that the capacity can look like no achievement at all. We eagerly take the intentional stance towards entities in our environment (Dennett, 2002) and attribute mental states to them.

The mental states we attribute include perceptual states, bodily feelings, emotional states, and propositional attitudes such as beliefs, desires, hopes and intentions (Margolis et al., 2012). This thesis is concerned with the attribution of propositional attitudes, particularly belief. A propositional attitude relates an agent to some content (“she believes that *m*”). So to attribute a belief to an agent

is to attribute to her a model m of the world w (where m may differ from w), and then to predict that she will behave according to her own private model m rather than according to the actual world w (Crane, n.d.; Perner, 1991). In this thesis we study how the *structure* of this attribution function might be acquired. How do humans, even at a young age, come to have this ability?

1.2 Experimental Evidence: Developmental Milestones

ToM doesn't arrive all at once, but in pieces over roughly four years and in a roughly fixed order (Beaudoin et al., 2020; Coşkun et al., 2020; Rakoczy, 2022; Wellman, 2018; Wellman & Liu, 2004). Long before children can say anything about minds, they parse motion as goal-directed. Woodward habituated 5- to 9-month-olds to a hand reaching for one of two objects and then swapped the objects' positions. Infants looked longer when the hand reached to the old location for a new object than when it reached to a new location for the old object (Woodward, 1998). So the infant encodes the goal object, not the trajectory.

Gergely et al. showed that, more than object-directed, the interpretation is even efficiency-sensitive: 12-month-olds expect an agent to take the most efficient means its situation affords, and are surprised when it does not (Gergely et al., 1995). Gergely and Csibra call this schema the *teleological stance* and distinguish it from Dennett's intentional stance (Csibra & Gergely, 2007; Gergely & Csibra, 2003). So before the first birthday, an infant can read bodies as taking efficient means to ends. This is *not* ToM. There's no attribution of mental states, it's just an extensional story.

In the second year, infants become sensitive to what an agent can (and can't) see, and begin to condition their expectations about an agent's goal based on the agent's perceptual access to the objects involved (Luo & Baillargeon, 2010). This is a perspective-taking ability, but restricted mostly to the epistemic case of perceptual access. ToM similarly requires perspective-taking, but pertaining to more complex mental states.

The jump to ToM involves a transition from reality-centered to belief-centered prediction. In the original *false-belief* task, Maxi puts his chocolate in one cupboard and leaves; his mother moves it to another; Maxi returns, and the child is asked where he will look (Wimmer & Perner, n.d.) (the task is also commonly known as the Sally–Anne task (Baron-Cohen et al., 1985), with similar setup). Though the observing child knows the chocolate is in the other cupboard, the child must predict Maxi's behavior according to *his* (false) belief, not according to the *child's* (true) belief. Children under about 3.5 years consistently fail, and (tellingly) their scores don't improve when the scenario is made more explicit. The age threshold is robust across task variations, countries and manipulations, aggregated over 178 studies (Wellman et al., 2001).¹

1.3 What a good account of ToM acquisition ought to explain

We seek an explanation for how ToM is acquired. To evaluate competing theories, we'll set out some of the features of ToM that must be explained by an adequate account of its acquisition.

¹4 years as the threshold has been challenged by studies seemingly showing *implicit* false-belief ability around 1.5 years (Buttelmann et al., 2009; Onishi & Baillargeon, 2005; Southgate et al., 2007). But the findings have replicated badly (Barone et al., 2019; Dörrenberg et al., 2018; Kulke et al., 2018) and even granting them, they may be accounted for by early automatic tracking abilities (Apperly & Butterfill, 2009; Butterfill & Apperly, 2013). For the purposes of this thesis, we take the ToM scale (Wellman et al., 2001; Wellman & Liu, 2004) as our explanandum.

(1) *Mental content isn't in the data.* Take for example someone trying to learn an intuitive physics by watching a ball's motion. The learner can simply extract regularities from what she sees, because a ball's position and velocity are recoverable from a sequence of images. A learner of ToM (intuitive psychology) doesn't have that luxury. What is available to the observer is a trajectory of bodies through space, but what has to be acquired is a vocabulary of attitudes that appear nowhere in the trajectory (Churchland, 1981; Sellars, 1956). So an acquisition procedure which works by finding structure in the observable variables can't yield ToM.

(2) *Attributions are underdetermined by behavior.* An agent walking across the room to the cupboard can be explained as wanting the chocolate and believing it is there, or as wanting to check whether it has been moved and believing it is there, or as wanting to be seen approaching the cupboard, and so on without end. Any behaviour is compatible with indefinitely many belief-desire pairs and no amount of further observation narrows the field to one, similarly to Quine on translation (Quine, 2013).² Yet humans select a fitting belief-desire explanation quickly, confidently, and on very little data. An acquisition theory should explain what does the selecting.

(3) *Novel concepts must be admitted.* The underdetermination problem presupposes a learner who already has the concepts and must choose among ascriptions built from them. But a learner who lacks the concept *believes* cannot make even a wrong attribution. So the acquisition problem isn't just curve-fitting or parameter estimation within a given theory. It requires constructing the theory's terms, positing unobservable entities to explain observable behavior. Fodor argued that no learning process could deliver a concept the learner did not in some sense already have (Fodor, 1992; 1998), and Carey's work on conceptual change tries to answer Fodor's challenge (Carey, 2011). An account of acquisition must explain how novel conceptual terms can come to exist.

(4) *Mental content must compose and embed* Because "believes" takes any proposition as an argument, the set of available attributions is unbounded: one can entertain attributions never encountered in the data, including attributions of beliefs one takes to be false and beliefs about things that do not exist. The operator also embeds in itself, which enables second-order mentalizing (Perner & Wimmer, 1985). And the attributions come in systematic families: anyone who can represent that Sally believes the marble is in the basket can also represent that Anne believes it is in the box (Fodor & Pylyshyn, n.d.). The terms get their content from their role in a network of inferences rather than from links to observations one at a time. So a good acquisition theory must offer a representational medium in which mental content composes, embeds, and licenses inference.

(5) *False-belief performance is delayed* Finally, we saw in Section 1.2 that goal attribution is in place before the first birthday while explicit false-belief prediction is not reliable until around four. A good account of ToM acquisition should explain this long delay. Also, the later competences look like assemblies of the earlier ones: belief attribution appears to be goal attribution plus subjective perceptual access. So in a good account, this ordering would be explained by the acquisition mechanism itself.

²This is a special case of the fact that data does not select a hypothesis without a prior (Goodman, 1983).

1.4 Theoretical perspectives on ToM acquisition

We now survey the standing accounts of how theory of mind is acquired and evaluate them according to the ToM features (Section 1.3). Nativism, empiricism, and constructivism are general positions in the debate about cognitive development, but we’ll consider them specifically in the ToM context.³

1.4.1 Nativism and core knowledge

The nativist holds that ToM is a built-in system that matures over time, so it’s not acquired from data in any interesting sense. Nativist positions are mainly motivated by the poverty of stimulus argument (PoS): the data available to the child is too sparse and too degenerate to fix the target by general learning, so the constraints that do fix it must be innate and domain-specific (Berwick et al., 2011; Chomsky, 1975; 2014).

Leslie argues for a theory of mind module (ToMM): a natively endowed, domain-specific system for interpreting the behaviour of intentional agents, which introduces attitude concepts such as *believes* and deploys a “proprietary representational system” for ascribing them to agents (Leslie, n.d.; 1994). The dissociation found in autism is held as evidence that ToM is a dedicated system, since it can be selectively impaired (Baron-Cohen et al., 1985).

Fodor’s nativism argues that the child’s theory of mind is not learned, because there is no coherent story on which a concept like belief could be learned (Fodor, 1992). So concepts are not constructed, and the productivity and systematicity of ToM are properties of a representational format that was there from the start. There’s also the “core knowledge” nativist position, which posits a small stock of encapsulated systems that give the child a skeletal domain-specific ontology from which they elaborate to more complex concepts (Spelke, 2022; Spelke & Kinzler, 2007).

The almost three year developmental delay from goal attribution to false-belief prediction is a challenge for nativists, since they must explain why an innate capacity isn’t manifested until later in development. A ToMM account can appeal to maturation of the module, or argue that task performance demands mask underlying native competence (Leslie, 2005; Scholl & Leslie, 2001). But these concessions accommodate the developmental data by auxiliary hypotheses. What would be better is a theory that itself predicts staggered developmental results.

There is also a question about how much the early findings can show, independent of the delay. That abstract domain knowledge is in place early (Wellman & Gelman, 1992) does not by itself favour nativism, since a learner that constructs such knowledge also ends up possessing it, and observing that it is possessed at a given age says nothing about which of the two happened. Earliness is not evidence of innateness, and that inference is doing a great deal of work in the nativist argument.

Nativism answers the hard requirement — how a learner comes to have terms for unobservables — by declining it.

1.4.2 Empiricism, association, and cultural learning

The empiricist holds that theory of mind is learned like anything else: through experience, from statistical regularities, by bottom-up association. In a connectionist implementation knowledge is

³There’s also simulationism, which denies that mindreading proceeds by a theory at all. On this account the attributor runs her own decision-making machinery offline on pretend inputs rather than applying general principles connecting states to behaviour (Goldman, 2006; Gordon, 1986).

distributed over the weights of a network, concepts are patterns of activation rather than symbols, and structure in behaviour is emergent rather than represented (McClelland, 2010; Rumelhart, 1986). Connectionist networks have even been shown to account for false belief without using symbolic representation (Berthiaume et al., 2013).

There's also Heyes' non-connectionist position which takes that ToM is a cognitive gadget, i.e. a culturally inherited skill assembled out of general-purpose learning mechanisms and transmitted through social interaction and talk about the mind (rather than a genetically inherited instinct) (Heyes, 2018; Heyes & Frith, 2014). Ruffman argues in a similar spirit that the developmental data can be accounted for without crediting children with belief concepts at all, by low-level and statistical means (Ruffman, 2014). In the linguistic domain, one empiricist approach derives the capacity to represent a false complement from acquiring the syntax of sentential complementation (Milligan et al., 2007; Villiers & Colle, n.d.).

Empiricist accounts are well-suited to explaining how ToM emerges even though mental content isn't in the data. But these explanations often deny that the structure of ToM is acquired at all, instead they just show how the learner acquires useful associations from data: it learns a mapping. And then the empiricist can't account for systematicity, since there's no representational medium with constituent structure (Fodor & Pylyshyn, n.d.).

Against a purely associative empiricism, the observed developmental order (Section 1.2) is not obviously the order of frequency in experience. Nothing about the input makes goal-directed pursuit a hundredfold more frequent than the conjunction of pursuit with restricted perceptual access. And no empiricist implementation shows how a data frequency difference leads to a three year false belief delay.

Furthermore, the later competences appear to presuppose the earlier ones rather than to be acquired independently alongside them as a separate regularity learned from its own statistics. The cultural-learning empiricist could attribute the competency ordering to the ordering of the input (Heyes & Frith, 2014). But that's a hypothesis about the input rather than a mechanism, and also it predicts that the developmental sequence should vary between children more than it appears to (Wellman & Liu, 2004).

Empiricism takes seriously the fact that the content is not in the data, and then concludes that there is no content to acquire.

1.4.3 Rational constructivism and the theory theory

WORK ON THIS SECTION

The constructivist holds that the child is analogous to a scientist (Downes, 1999; Gopnik, 2012). A child constructs intuitive theories of mind and world which she revises in light of evidence, by something like the process that drives scientific progress (Gopnik, 1997; Gopnik & Wellman, 1992; Wellman et al., 1990). So the child uses domain-general statistical machinery to construct domain-specific structured knowledge. Infants have been shown to be sensitive to sampling and to statistical evidence in ways that indicate they do inference to hidden structure, including psychological structure (Wellman et al., 2016; Xu, 2019; Xu & Kushnir, 2013). And in general children's exploratory

behaviour is well described as evidence-gathering aimed at resolving uncertainty about which theory is right (Schulz, 2012).

Rational constructivists specify that Bayesian inference is the formal mechanism of theory change (Gopnik & Wellman, 2012; Newcombe, 2011; Xu et al., 2012). Structured, abstract knowledge, embedded in a probabilistic framework, can be acquired from sparse and noisy data (Gopnik & Tenenbaum, 2007; Griffiths et al., 2010; 2011; Perfors et al., 2010; Ullman & Tenenbaum, n.d.; Xu, 2019). Theory formation generally has been modeled as probabilistic inference over systems of related concepts (Goodman et al., 2011; Kemp et al., 2010), and theory change has been modelled as stochastic search through a space of theories generated by a grammar (Ullman et al., n.d.).

The position is a middle way between the previous two. Nativism posits domain-specific logical symbolic mechanisms, whereas empiricism posits domain-general probabilistic statistical mechanisms. Rational constructivism instead asks “how can domain-general mechanisms of learning and representation build domain-specific systems of knowledge? How can structured symbolic knowledge be acquired through statistical learning?” (Griffiths et al., 2024; Tenenbaum et al., 2011). It grants the nativist’s premise that our minds “get so much from so little” (Griffiths et al., 2024) while denying that the remedy has to be innate content, since “structured symbolic representations need not be rigid, static, hard-wired, or brittle. Embedded in a probabilistic framework, they can grow dynamically and robustly in response to the sparse, noisy data of experience” (Tenenbaum et al., 2011).

On this account the transition from failure to success in the false-belief task is analyzed as an evidence-driven shift between two theories. The infant starts with a simple model, but after encountering evidence inexplicable by that model switches to a more complex one which offers greater explanatory resources (Goodman et al., n.d.).

So the theory theory comes closest. It has the right shape for every one of the requirements in Section 1.3: theories have terms, terms are constructed, construction is answerable to evidence, the medium is structured, and construction is cumulative so the order of acquisition is a consequence of what has already been built. But having the right shape is not the same as being a mechanism. The account says that children construct theories; it does not say what process does the constructing, nor by what standard one candidate theory is preferred to another, nor — and this is the sharp version — how a learner searches a space of possible theories that is infinite. The metaphor of the child as scientist is powerful, and it describes what happens rather than naming something that would make it happen.

That gap is what keeps the nativist in business. Absent a mechanism, the nativist can say that no constructive process exists, that what looks like construction is really the maturation of something already in place, and that the appearance of theory change is a redescription of a developmental timetable. The strength of that reply is parasitic on the absence of a mechanism. It is not an argument that construction is impossible; it is an argument that nobody has exhibited one.

So the business of the next chapter is to supply one. If the theory theory needs a process that searches a space of structured hypotheses, prefers among those that fit the data on principled grounds, introduces new terms whose content is fixed by their role rather than by observation, and

makes what is learnable at each stage depend on what has already been learned, then that is a specification, and it is a specification we can go looking for a formal system to meet. Chapter 2 argues that Bayesian program induction is such a system, and that DreamCoder is a concrete algorithm that implements it.

Chapter 2

Learning as Bayesian Program Synthesis

2.1 The computational-level problem

The learner takes as input a trajectory of bodies through space, and outputs the process that generated the trajectory. The output process is formulated in terms of mental states (beliefs, desires, intentions) that don't appear in the data. The trajectory underdetermines the process, since any behaviour is compatible with indefinitely many belief-desire pairs. So no amount of further observation eliminates all but one.

So the problem is of the form: given data, entertain hypotheses about the process that generated it, assign each a prior plausibility, and revise those plausibilities as new data comes in. There's already a known answer to the normative question of how such revision ought to proceed, which is Bayes' rule. So our computational-level claim (Marr, 2010) is that acquiring a theory of mind is the problem of finding the best explanation of observed behaviour, where the best explanation takes into account both its prior plausibility and how well it fits the data.

2.2 Bayesian inference: the logic of learning

Say a learner observes data d , a trajectory of bodies through space. A learner's task is to choose among possible explanations $\mathcal{H}$ a hypothesis h that best explains the trajectory. The task is to choose the most plausible hypothesis given the data, i.e. to find the hypothesis whose posterior probability $P(h|d)$ is highest. In calculating this quantity, the learner should weigh two things. First, the prior probability $P(h)$: how plausible was hypothesis h before any data d was observed at all? Second, the likelihood $P(d|h)$: how well does hypothesis h account for the data d , now that data is available? Bayes' rule specifies how to combine these considerations:

$$P(h|d) = \frac{P(d|h)P(h)}{\sum_{h' \in \mathcal{H}} P(d|h')P(h')}$$

Since the denominator is a normalising constant that doesn't depend on h , it can be ignored for the purpose of ranking hypotheses against each other. So we rewrite the equation as the proportion

$$P(h|d) \propto P(d|h) \cdot P(h)$$

i.e. the posterior probability $P(h|d)$ of a hypothesis h given data d is proportional to the likelihood $P(d|h)$ of the data d given hypothesis h times the prior probability $P(h)$ of hypothesis h before data d was observed. Bayes' rule is a theorem of the probability calculus: given the basic commitment to representing degrees of belief as probabilities, it's the unique coherent way to revise them in light of

evidence (Jaynes & Bretthorst, 2003; MacKay, n.d.). Having stated the computational problem the mind faces, Bayes’ rule specifies the ideal solution.⁴

The prior $P(h)$ expresses the a priori plausibility of a hypothesis, i.e. how likely the learner takes h to be, independent of the current observations. So the prior encodes the learner’s background knowledge. For example a hypothesis that is outright impossible has prior $P(h) = 0$ so it’s excluded from $\mathcal{H}$ and no data observation can increase its posterior probability. The prior determines which explanations are more natural, so we can start to generalize from among the indefinitely many hypotheses consistent with the data.

Anything can be a prior, since it’s just what encodes the learner’s background knowledge, assumptions, and preferences. In this thesis the prior we’ll focus on is the preference for simplicity. Hypotheses that are short, that reuse existing structure rather than stipulating new structure, will have higher priors than hypotheses that are long and idiosyncratic. This assumption is supported by accounts of human rule-based concept learning (Chater & Vitányi, 2003; Goodman et al., 2008), and by the finding that the difficulty of learning a Boolean concept tracks the length of its shortest logical expression (Feldman, 2000). In Section 2.3 we’ll say how the simplicity of a hypothesis is calculated, after we’ve said what hypotheses are made of.

The likelihood $P(d|h)$ expresses how probable the observed data would be if the hypothesis were true. In our setting a candidate hypothesis either explains the observed behaviour (i.e. it reproduces the trajectory exactly) or it does not. So the likelihood is all-or-nothing: the data have probability zero $P(d|h) = 0$ if they disagree with the hypothesis, and constant probability $P(d|h) = c$ otherwise (Goodman et al., 2008). The binary likelihood can only ever eliminate hypotheses (likelihood $P(d|h) = 0$ yields posterior $P(h|d) = 0$). So once a set of hypotheses all fit the data perfectly (only those with likelihood $P(d|h) = c$ remain), the likelihood can’t distinguish between the remaining hypotheses and their ranking is decided entirely by the prior.

2.3 Programs as the hypothesis space

Bayes’ rule doesn’t specify what we should define our hypothesis space over, so we can choose the best medium for representing our hypotheses. Since we choose simplicity as our prior, hypotheses should have parts that can be counted so their relative size and complexity can be compared. And a hypothesis must be executable, since the indicator likelihood asks whether running a hypothesis reproduces the observed trajectory (Section 2.2). A finite learner must have unbounded and systematic capacity (Section 1.3), so the hypotheses must be generated by rules of combination (Fodor & Pylyshyn, n.d.). And also the hypothesis space must admit new terms, since we want to model how a learner acquires the terms that denote mental content (attitudes) even though they don’t appear in the data.

These requirements rule out connectionist representations (Section 1.4.2) where a hypothesis is a weight vector or a point in a high-dimensional parameter space (Tenenbaum et al., 2011). Though

⁴To be clear this claim is at the computational level, not at the algorithmic level. Nobody is saying that a child computes posterior probabilities, or represents the hypothesis space $\mathcal{H}$ explicitly, etc. The claim is that the child’s behavior approximates the solution to an inference problem whose ideal form is described by Bayes’ rule. It’s not an algorithmic-level claim about the procedure by which the child does this, though we’ll later introduce an algorithm for approximating this inference.

they’re efficient to update, they lack a natural notion of countable constituent parts so we can’t compare their relative sizes. What we need is a structured symbolic representation like a grammar, predicate logic, relational schemas, or functional programs (Griffiths et al., 2024; Tenenbaum et al., 2011). What we need is a language of thought (Fodor, 2010), where mental representations are symbolic objects combined by syntactic rules and evaluated as a function of their parts (Bruner et al., 1986; Carey, 2011; Siskind, n.d.).

Programs are the natural medium for our hypothesis representation. They are compositional, since the value of a composite expression is a function of the values of its parts. And a program admits any expression of the right type in any argument position of that type. They are executable, which is what the indicator likelihood requires. And they encode procedural and declarative knowledge in the same format (Rule, n.d.).

Problem solving has been treated as symbol manipulation since the advent of the computational theory of mind which views the mind as a compact program that mirrors the structure of the world (Baum, 2004; Goodman et al., 2014; Newell et al., 1958). A symbolic medium also leaves the learner free to adopt new vocabulary beyond what the modeler supplied (Gopnik & Wellman, 2012). And furthermore, if knowledge is programs then learning is program induction (Flener & Schmid, 2008; Gulwani et al., 2017; Schmid et al., n.d.) which we’ll soon make use of.

We adopt the λ -calculus as our program representation formalism, since it’s the standard choice for models of learning across domains (Piantadosi et al., n.d.; 2012; Schmidt, n.d.; Zettlemoyer & Collins, n.d.). Most importantly it gives us λ -abstraction as the operation of taking a recurring piece of structure and giving it a name. So this way a learner can come to possess a term that names something that isn’t observable in the data, since the new term’s content is fixed entirely by what it does in combination with the other terms.⁵

Since our programs are composed of terms of a grammar, a distribution over a grammar’s productions induces a distribution over expressions. Since the probability of an expression is the product of the probabilities of the choices made in generating it, longer expressions (involving more choices) get less probability mass. The cost of writing a message in a code optimal for that distribution is the negative log of its probability (Shannon, n.d.), so h ’s description length under the given grammar is $DL(h) = -\log P(h)$. So the prior is $P(h) \propto e^{-DL(h)}$. And we have an indicator likelihood for whether executing h reproduces the observed behavior. Substituting into Bayes’ rule, the posterior is computed as

$$P(h|d) \propto e^{-DL(h)} \cdot \mathbb{1}[h \text{ solves } d]$$

So maximizing the posterior is just minimizing the description length of correct programs. The learner should adopt the program which has the shortest description length of all programs that solve the data (i.e. that reproduce the trajectory). This is basically the theory theory (Section 1.4.3): the learner should prefer the simplest theory that accounts for the evidence.

⁵Again, we don’t claim the mind implements a λ -calculus interpreter, only that its ToM competence is well described as inference over hypotheses that compose, execute, embed, and can be measured for size (which are all properties the λ -calculus representation supplies).

2.4 Program induction and the search problem

The trouble is that the optimization problem of searching through program space for one that fits the data is about as badly behaved as an optimization can be⁶. The space is infinite, since a grammar whose productions can nest inside one another generates unboundedly many expressions, and the number of expressions of a given size grows exponentially in that size. There's no gradient to follow, because programs are discrete: there's no meaningful sense in which one is slightly perturbed into another. And the binary likelihood doesn't give partial credit, so a program that differs from a correct one by a single token receives exactly the same likelihood as a program picked at random (namely zero). So the search is blind: there's no signal anywhere in the space to hill-climb on, and no way to tell that one is getting warm.

To make the search less inefficient at least we can enumerate hypotheses in decreasing order of prior probability, checking each against the data until one fits, so given enough time we find a solution program. Since $P(h) \propto e^{-DL(h)}$, enumerating every program of description length at most ℓ takes time on the order of e^ℓ . The time to find a solution is exponential in the description length of that solution under the current library. Note that in Section 2.3 we introduced description length as a way of ranking hypotheses, but this way it also determines which hypotheses are even reachable at all. The prior is simultaneously a measure of plausibility and a budget. A 40 nat long program is certainly expressible, but it won't be evaluated within a reasonable time. A concept's being expressible in the language is no guarantee whatever that it's learnable in the language within a reasonable budget.

Description length is relative to the primitive vocabulary available, so the same program may be long under one grammar and short under another that happens to contain the right terms. So facing the search problem, we should just change the language so the target gets shorter. If we add primitives to the library that are useful for the domain, programs that were previously out of reach will fall within the budget since they can now be expressed more succinctly. This doesn't make the problem tractable in general, but it might make particular programs tractable when they weren't before.

2.5 Learning the language itself

WORK ON THIS SECTION

If we're permitted to adjust the vocabulary whenever the search is going badly, then the modeler can make anything learnable by just adding a primitive that does the work. What we need is an account on which changing the language is not a device the modeler applies from outside but something the learner does as part of the same inference it was already doing.

In the setup (Section 2.2) the learner is given a hypothesis space $\mathcal{H}$ and a prior $P(h)$ which is the type-uniform distribution over primitives in the grammar. So revising the prior is just changing which symbols the learner has. When a structure recurs across the solutions found for a domain, that recurrence is evidence that the structure is useful. It can then be named, added to the inventory, and thereafter costs a single symbol wherever it appears. Programs that previously were prohibitively

⁶Here we start to move from the computational level toward the algorithmic one, i.e. the procedure in which the computational problem is solved (Marr, 2010; Tenenbaum et al., 2011)

long can now be expressed more succinctly, since the complexity of the often-used structure is abstracted into a new named primitive.

Adding a primitive, changing the language, follows a similar inferential principle as in Section 2.3 but applied a step further out. Given a corpus of observations $D = \{d_1, \dots, d_n\}$, the goal is to find the best language of primitives $\mathcal{L}$ for that corpus. The posterior $P(\mathcal{L}|D)$ is given by:

$$P(\mathcal{L}|D) \propto P(\mathcal{L}) \cdot \prod_{i=1}^n \sum_h P(d_i|h) \cdot P(h|\mathcal{L})$$

A library’s prior $P(\mathcal{L})$ is determined by its simplicity: a library with fewer primitives (and less complicated primitives) has higher prior probability. Without this complexity cost the best language is always the one carrying a dedicated symbol for every task in the corpus, so it overfits to the specific corpus and won’t generalize to similar corpora of the same domain.

The product runs over the corpus of n tasks $D = \{d_1, \dots, d_n\}$. For a given task d_i and a given hypothesis h , the likelihood $P(d_i|h)$ indicates whether executing program h correctly produces d_i (zero otherwise). And $P(h|\mathcal{L})$ is the description length prior (Section 2.3) relative to a language, so $e^{-\text{DL}(h|\mathcal{L})}$ is the cost of writing h in the vocabulary $\mathcal{L}$. Then the program h is marginalized away, since a language just needs to make an adequate hypothesis available.

Because the likelihood is an indicator, every h that fails to reproduce d_i contributes nothing to the sum. So the inner sum collapses to the total prior mass the language places on programs that solve the task, $\sum_{h \text{ solves } d_i} P(h|\mathcal{L})$. So the equation says that a good language is one under which solutions are cheap, and cheap for every task in the corpus at once. This is the posterior over libraries inferred in (Ellis et al., 2020).

Learning the language rather than a hypothesis within it has been done for intuitive theories generated by a probabilistic grammar (Ullman et al., n.d.), and for program fragments transferred from solved problems to unsolved ones (Liang et al., n.d.). DreamCoder adds that what gets learned is a library of λ -terms, so that revising the language means acquiring new concepts rather than redistributing probability among terms the learner already had (Ellis et al., 2020).

But settling a language is an even worse optimization problem than program space search (Section 2.4), since the space of candidate languages is larger than the space of hypotheses under any one of them. DreamCoder approximates it by maximizing a lower bound on the posterior (Ellis et al., 2020). In rounds, it alternates solving a batch of tasks under a fixed language, then revising the language in light of what the solutions turn out to have in common (Dechter et al., n.d.). An abstraction found in a later round may be defined in terms of abstractions found in earlier ones, so the vocabulary accumulates in layers.

In the implementation the learner doesn’t have available a distribution over languages. So the algorithm instead maximizes, carrying a single library forward from each round as a MAP point estimate. Also, the sum over h ranges over every program the language can express. So instead the algorithm replaces it with a finite set of the best solutions actually found for each task.

The library is built iteratively as more primitives are added, so nothing recovers a library discarded from an earlier round. So the procedure is liable to settle on a local optimum that full

inference would have avoided. So the implementation makes inference computable, but at the cost that the library might not be posterior-optimal.

2.6 Our claim

WORK ON THIS SECTION

The nativist usually makes the poverty of stimulus argument about data, claiming there isn't enough data to fix the target concepts to be learned (Section 1.4.1). We can now also give the search version of the argument: even granting the learner all the data it could want, and granting it the right hypothesis space, the target might still be exponentially far away. So something must be built in to put it within reach. In this thesis we respond that, though the search indeed isn't easy, the distance isn't fixed since the language isn't fixed.

We claim for learning, “strong but domain-general inference and representational resources are aided by weaker, domain-specific perceptual input analyzers” (Goodman et al., 2011). Of course something must be built-in, so our disagreement with the nativist is about what that must be. On the modular view what's innate includes the attitude concepts themselves, a proprietary representational format for ascribing them, and the agent-directed inferential machinery that operates on it (Leslie, 1994). On our view what's innate is a capacity to represent structured hypotheses, a preference for short ones, a procedure for searching, and enough perceptual apparatus to individuate objects and their motions. So anything specific to the mental is constructed.

The construction of a conceptual vocabulary is cumulative (Carey, 2011; Piantadosi et al., 2012). An abstraction acquired at one stage becomes a term in the language in which the next stage's hypotheses are written, so at each stage more complex hypotheses can more easily be posited. This predicts a particular relationship between the milestones of Section 1.2: theory of mind is not one concept acquired at a stroke but a layered vocabulary, and its components should become available in an order fixed by what each one is built out of. Goal-directedness is available early because it's an abstraction over a great many observed pursuits. Belief is available late because it's an abstraction over goal-directed pursuit together with perceptual access, and it cannot be assembled before its parts exist. So the developmental delay (Section 1.3) is a consequence of the compositional structure of what's being learned. Whether it actually does so is an empirical question about a particular corpus and a particular library, and one this thesis puts to the test rather than assumes: Section 5.4 reports where in the run each capacity becomes available and in what order.

What the loop buys, then, is the thing Section 1.3 asked for and Section 1.4 found missing: a mechanism by which a learner constructs new conceptual terms out of earlier ones, adopts them on grounds of compression alone, and thereby brings within reach concepts that were unreachable before — which is what the theory theory was accused of lacking in Section 1.4.3. And because reachability is fixed by description length under the current library, while the library changes as a function of what has already been learned, the *order* in which concepts become available is a consequence of the mechanism rather than a stipulation added to it. The developmental ordering of Section 1.2 becomes something a model of this kind can be asked to predict rather than to accommodate (Palmarini et al., 2024) — which is what the rest of the thesis puts to the test.

2.6.1 What is not claimed

Fixing the level fixes the scope of the claim, and it is worth being explicit about what falls outside it. We are not studying how a child in fact comes to learn theory of mind. That is the province of developmental and cognitive psychology, and nothing here should be read as a proposal about neural implementation or about the moment-to-moment course of a particular child's development.

Nor is the claim that children run the algorithm just named. The library-learning system described and implemented in Chapter 4 is offered as an algorithmic-level *realisation* of the computational-level claim: a demonstration that the problem, so stated, is solvable by a mechanism that begins with no mental vocabulary at all. Its role in the argument is one of sufficiency rather than fidelity. If a learner equipped only with domain-general primitives and a preference for short programs comes to posit belief-like structure, then positing such structure is something a learner *can* do rather than something it must be given — and the nativist's inference from “no mechanism has been named” to “no mechanism exists” is blocked. Whether human learners use this mechanism, or some other one that solves the same problem, is a further question that this thesis does not settle.

Chapter 3

Bayesian Theory of Mind

3.1 Agent as planner

In this chapter we first claim that Bayesian Theory of Mind (BToM) is the best available computational account of the mature ToM capacity described in Chapter 1. BToM formalises the intentional stance as inverse planning, and its posteriors quantitatively track human judgements. We then claim that in BToM the structure of belief attribution is stipulated by the modeler rather than inferred from the data.

To adopt the intentional stance towards a system is to treat it as a rational agent. First attribute to it the beliefs and desires it ought to have (given its situation and history), then predict its behavior by working out what a rational agent would do (Dennett, 2002). People adopt the intentional stance even towards two triangles and a circle moving about a rectangle, seeing them as chasing, hiding, bullying, and escaping each other (Heider & Simmel, 1944). The intentional stance is a strategy that gets used because it regularly yields predictive success (Dennett, 2002).

Bayesian Theory of Mind (BToM) gives a computational account of this strategy, modeling the agent as a planner. The agent occupies one of a set of states, has available in each state a set of actions, and the actions move it between states according to a transition function. The planner takes as input a utility function and a model of the environment, both defined over the agent’s state space, and returns the actions the agent should perform to maximize utility (Baker, 2012; Baker et al., 2009; Lake et al., 2016). Even twelve-month-old infants encode actions as directed at a goal state, and expect an agent to use the most efficient means available to Section 1.2 (Csibra & Gergely, 2007; Gergely et al., 1995; Gergely & Csibra, 2003).

3.2 Inverse planning

A planning agent computes in the forward direction: given the model of the world m and a goal g , find the optimal behavior (the trajectory t) for achieving that goal. So the agent computes $t = \text{optimize}(g, m)$, where “optimize” is a planning algorithm that finds the best path to g which is available on m , since the agent is rational (Section 3.1).

An observer watching an agent needs to compute in the backward direction: given the world model m and the agent’s trajectory t , find the goal that, assuming a rational agent, would yield such a behavior. The observer can simulate what the agent would have done if its goal were g' , calculating the resulting trajectory as $t' = \text{optimize}(g', m)$ since the observer assumes the agent is rational. Then if the simulated trajectory t' matches the actual trajectory t , then the hypothesis that g' is the agent’s goal (that $g = g'$) is correct. The process is analogous to modeling intuitive physics by running a physics engine forward under different settings and scoring the outcomes (Battaglia et al., 2013; Lake et al., 2016).

Using Bayes’ rule (Section 2.2), the learner finds the hypothesized goal that best explains the agent’s behavior. Writing g for the agent’s goal, m for its model of the environment, and t for the observed agent’s trajectory, the posterior is calculated as

$$P(g, m \mid t) \propto P(t \mid g, m) \cdot P(g, m)$$

where the prior $P(g, m)$ encodes what the observer expects agents to want before seeing anything, and the likelihood $P(t \mid g, m)$ encodes the probability that an approximately rational agent with goal g and model m would move as observed (which is supplied by the planner) (Baker et al., 2009). It’s been shown experimentally that people’s predictions on behavioral stimuli are close to the model posteriors of this approximately rational inference mechanism (Baker et al., 2009; 2017; Jara-Ettinger & Rubio-Fernandez, 2021).

3.3 Belief as an input

The posterior is joint over (g, m) , the agent’s goal g and a model m of the environment in which it navigates. But m doesn’t have to be an accurate model of the actual state of affairs (the observer’s world), it could be any possible world model. The observer can supply a counterfactual model m' that diverges from the actual state of affairs, and then run the same posterior computation to see how the agent would behave if it were navigating in the counterfactual world m' . Then the same computation that yields sensible behavior from an accurate model would yield mistaken behavior from the inaccurate one.

This is belief attribution in essence, attributing a counterfactual world to an agent such that it acts according to the counterfactual model as its own private world. So we can ask: given an (irrational) trajectory, which model of the environment would’ve made those actions rational? Take the false-belief task (Section 1.2): Sally puts her marble in the basket and leaves; in her absence the marble is moved to the box; and when she returns she searches the basket (Baron-Cohen et al., 1985; Wimmer & Perner, n.d.). Evaluated against the actual (observer’s) world m , Sally searching the basket is irrational since the marble is in the box. But evaluated against her private model she had when she left m' (that the marble is in the basket), her searching the basket is optimal. An observer that can represent the divergence of worlds can predict that Sally will look in the wrong location, and even which wrong location she’ll look in (Baker et al., 2017; Goodman et al., n.d.).

To calculate the likelihood, the observer needs to simulate the predicted trajectory. The agent’s private model m' is a counterfactual derived from the observers as $m' = \text{derive}(m)$, where “derive” modifies m to account for the agent’s false belief which the observer imputes. In the Sally–Anne example, in the observer’s model m (the real world) the marble is in the box, whereas Sally’s model m' (her private world) is the same as m in every respect except that the marble is in the basket.

The observer’s then needs to run the agent’s planner on its private model, to yield its trajectory (behavior) given that model. Rather than running $t' = \text{optimize}(g', m)$ on her own model m (as in Section 3.2), the observer runs $t' = \text{optimize}(g', m')$ on the agent’s private model m' . So t' is the agent’s predicted trajectory, given that it pursues goal g' along the most optimal path afforded by counterfactual model m' . Then t' can be compared to t , to see whether the predicted goal g' and model m' predict that a rational agent will take trajectory t' .

The “optimize” planner is the same one from Section 3.1. It’s indifferent to whether its input is the real world model m or an incorrect world model m' . Most importantly, it’s indifferent to whether its input model is *attributed to an agent* or not. All it does is run a search algorithm on input data. The model is executed and evaluated the same way. So there is nothing belief-specific about the planner.

3.4 What is stipulated by the BToM modeler

BToM is a model of the ToM capacity of a mature adult (Baker et al., 2009; 2017). It’s a model of the computational problem that a person who has ToM is solving when they are trying to understand a person’s behavior. So the ToM capacity is specified by the modeler. In this thesis we’re interested in how this capacity might be acquired. In Chapter 4 we’ll introduce a model that (we’ll argue) doesn’t have the mature ToM capacity which is stipulated by the modeler of a BToM model. To argue that these features aren’t stipulated, we’ll state them explicitly here.

Most obviously BToM assumes the rationality principle, that the agent plans approximately optimally toward its goal given its model Section 3.1. We assume this as well in our model, because efficiency-sensitive goal-directed navigation is not ToM Section 1.2. But a lot of other feature of the BToM model are so basic that they’re easily taken for granted. For example in a BToM model, the visual scene is often already decomposed into an agent that has a goal g and a belief m , with the modeler already stipulating the roles that different entities play. The planner takes them as separate arguments that vary and can be inferred independently of each other.

A BToM model stipulates the attitudes that the learner searches over, for example the space of goals. The posterior over goals is inferred but the space it ranges over (and the prior over the space) is given. Baker et al. compare several goal priors (Baker et al., 2009), because the modeler has to specify the space. The model space that the posterior ranges over, the fact that the planner’s input model m' is permitted to diverge from the observer’s m , is similarly stipulated by the modeler. And the modeler has to specify the divergences between m and m' . The BToM model infers which particular model m' the agent holds, but the range its drawn from is fixed in advance. That a belief could be false is not itself in question.

Inference in a BToM is parameter estimation within a fixed theory, but we’re interested in construction of the theory’s terms.

The five items of Section 3.4 have a shape we already have a name for. They are not hypotheses about any particular scene; they are hypotheses about what the hypotheses about scenes are like — that behaviour is generated by an agent, that the agent has a goal and a model, that the model may diverge from the world, that the divergence is drawn from such-and-such a range. In the terms of Section 2.5 they are the learner’s language rather than anything written in it: the vocabulary in which a particular attribution gets stated at all. BToM installs that vocabulary and infers underneath it. The question of this thesis is whether it can be inferred rather than installed — whether the language, given to the framework by its modeller, is the kind of thing a learner could arrive at by the same inference it runs on everything else.

Putting it that way fixes what the model of the next chapter has to do. It cannot be handed any of the five. Its learner begins with grids, a planner, and domain-general combinators for building

and consuming pairs — no agent, no goal, no attitude, no channel that is anybody's — and it has to assemble the structure out of them, in program space, under a description-length objective that knows nothing about which tasks are mental. This raises the bar past expressiveness. A language in which belief is merely *statable* proves nothing, since the stipulated architecture is *statable* in any language rich enough to write it down. What has to be shown is that the structure is *selected*: that among the programs which reproduce the scenes, the one carrying the agency signature is the one compression keeps, and that it beats the non-mental rivals the very same library can express. Section 4.7 states that test in advance and Section 4.6 takes the five stipulations in the order given here.

Chapter 4.

The Model

4.1. The learning problem

Our model is a triple $\langle \mathcal{L}_0, D, \text{MDL} \rangle$: an initial language $\mathcal{L}_0$ of primitive operations, a corpus D of scenes, and the description-length objective MDL (Section 2.5). The model is a map from the triple to the final language $\mathcal{L}^*$. Learning occurs in rounds, where each round involves solving tasks in D (enumeration) and growing the library with new abstractions (compression).

The learner is an observer who is shown scenes from D , visual episodes that unfold over discrete time steps on a small grid. In *enumeration* the learner seeks to recover the underlying process that generated what it saw, by positing candidate hypotheses until it finds one that works. Hypotheses are written as compositions of primitives chosen from the library $\mathcal{L}_i$. In *compression*, the learner finds syntactic structure common to its found hypotheses and abstracts them as new primitives added to the library, yielding $\mathcal{L}_{i+1}$. So in the next enumeration, the learner’s new hypotheses are formulated using higher-level terms, and the terms become iteratively more complex.

We ask whether such a learner can come to learn structure of belief attribution as a new abstraction (Section 3.3), even without $\mathcal{L}_0$ having terms for an agent, goal, or attitude, and without D distinguishing the same (Section 4.6). We claim that belief attribution is learnable, that it lies in the image of the map $\langle \mathcal{L}_0, D, \text{MDL} \rangle \rightarrow \mathcal{L}^*$. We fix D in Section 4.2, $\mathcal{L}_0$ in Section 4.3 and Section 4.4, and MDL in Section 4.5. An instance of our model gives a counterexample to the claim that belief attribution must be acquired through a native domain-specific module (Leslie, n.d.).

Take the world to be a state space W , so the data is the movement of entities in W . A hypothesis is a transition function $W \rightarrow W$, a theorized dynamic by which the world advances to its next state. Our model adds the product $W \times W$ with its canonical structure maps, so the learner has the capacity to hold two world states (Section 4.3). This is not the capacity for belief attribution, since nothing in a product says “whose” the second factor is, or that it “is” anybody’s.

Our model also adds a planner, which turns a utility into a dynamic that maximizes it (Section 4.6.3). We grant this substantive endowment on the grounds that efficiency-sensitive goal pursuit is in place some three years before false-belief attribution (Section 1.2). The third is the two-part MDL objective of Section 2.5, which prices a language jointly with the corpus written in it.

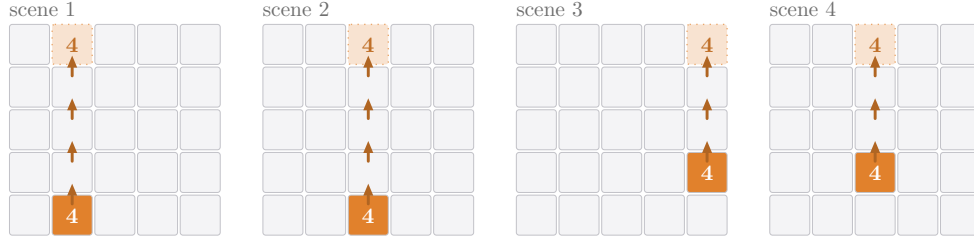
Belief attribution is a particular way of putting these components together. To attribute a belief is to open the product, perturb the second factor, run the planner against that factor rather than the first, and collapse the pair by publishing a particular value (Section 4.4.1). We ask whether an objective that only seeks to minimize description length finds this arrangement among the space of possible abstractions. So our question is about the map $\langle \mathcal{L}_0, D, \text{MDL} \rangle \rightarrow \mathcal{L}^*$ rather than about the expressive power of the language.

4.2. The task corpus

Our corpus consists of 316 tasks in 25 “families”, groups of tasks that are all governed by a common underlying process. Of these 168 tasks in five families can only be solved by belief attribution, and 148 can be solved extensionally. The search doesn’t know this distinction about tasks of course, it treats them all alike.

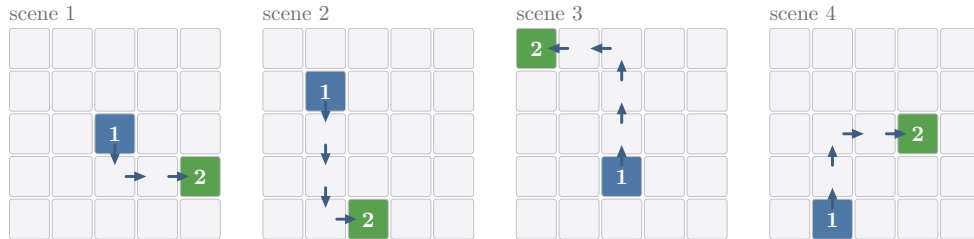
Most tasks are *trajectory tasks*. A trajectory is a sequence of 2d grids over the time dimension, that depicts some dynamic at play (e.g. a body that falls, an agent that approaches something, etc.). A task consists of k trajectories that all depict the same dynamic but vary in setup (where the entities start, how long the scene takes, what else is lying on the grid, etc). Given a task, the learner’s goal is to find a transition function that, for any trajectory in the task, returns the next grid for any grid in the sequence. The transition function should be a general explanation of the underlying dynamic, invariant to initial conditions.

For example, the task below is an instance of the *constant-movement* task family. It consists of four scenes that each depict the value 4 moving up. In the figure the grids of each timestep are collapsed into one, so each arrow indicates one timestep forward.



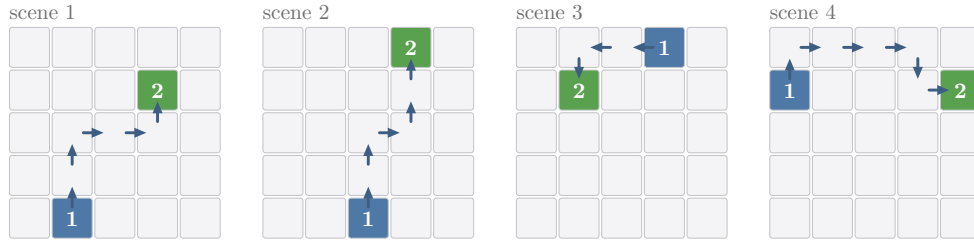
A solution to this constant-movement task would be a program that encodes “at each time step, value 4 moves to the adjacent cell above it”. Such a program solves the task because for each scene, given that scene’s starting positions, the program reproduces the rest of the trajectory.

The following task is an instance of a *goal-directed movement* task.



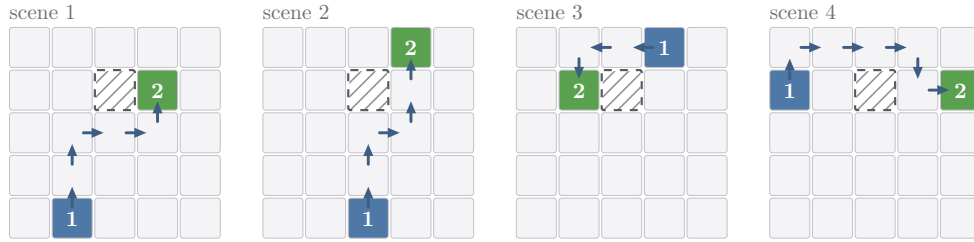
A solution to it would be a program that encodes “at each time step, the value 1 moves to the adjacent cell closest to value 2”. The solution involves the optimizing planner (Section 3.1), but *does not* require attributing a belief. It’s a purely extensional description.

The following is an instance of an *obstacle* task. The same solution program for the above goal-directed task reproduces the first three scenes. But notice that in scene 4, value 1 takes a suboptimal path to value 2 (it should just go straight to the right but it goes up first). So we need a program that accounts for this, so it solves all four scenes.



Say the origin (0,0) is the top left cell. A solution might encode “at each time step, value 1 moves to the adjacent cell closest to value 2, in a world where an impassable wall sits at cell (1,2)”.

The wall is no part of what the learner is shown; it is posited by the hypothesis, and stamped onto the grid by the wall-stamping edit that hypothesis contains. Drawn in, hatched to mark it as the hypothesis’s rather than the task’s, the same four scenes look like this.



Only scene 4 is evidence for that wall. It stands at the same cell in all four, but in scenes 1–3 a shortest route to the goal gets around it anyway, and in all three the agent takes one just what the wall-free predicts on its own. In scene 4 the agent and its goal share a row, so the wall blocks every shortest route, and the six steps the agent spends on a journey of Manhattan length four are something no amount of distance-minimising will reproduce.

These tasks, and most of the corpus, can be solved by programs that describe the scene extensionally via a purely behaviourist visual description. The task families we care about are those that can only be solved by attributing a belief to an agent.

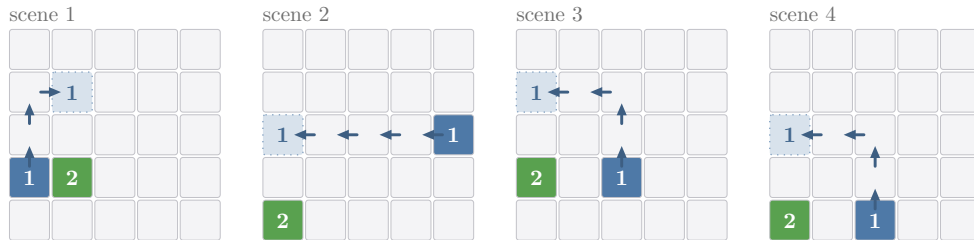


Figure 5: An instance of a goal-displacement task: the four scenes of one task, each collapsing its frames into a single panel, so that each arrow is one time step. The scenes vary in where the entities start and in how long they run, and share only the program that generated them

The learner is shown those four scenes and nothing else: no labels, no segmentation of the grid into agent and goal, no indication that this task differs in kind from a falling body. What it must return is a single transition function which, started from each scene’s own first frame, reproduces that scene’s every frame. In all four, the value 1 takes an efficient path and then stops on an empty cell two rows above the value 2 — on the cell, that is, that the 2 would occupy if it were two rows higher. The 2 itself never moves.

No description of where things are on the grid does that, because the cell the agent walks to is not where anything is. What does it is a description of where the agent *takes* things to be. Written out in English, the program encodes

at each time step, copy the grid and move the value 2 one cell up on the copy; in the copy, move the value 1 to the adjacent cell closest to 2; then in the world move 1 to wherever the copy left it, and change nothing else

and that is how every program in this thesis will be given: by what it does to a grid at each time step, together with its type where the type matters — here $G \rightarrow G$, a transition function like any other, since nothing in the type of a program announces that it is about a mind. The symbols the search engine actually prints are an implementation matter, and are quoted only where a count of them is at issue.

Read that way it is not a program about grids but a proposition about a mind, and each of its parts answers to something Chapter 3 stipulated. The private copy is BToM’s m' ; the planner is its rationality assumption; and the commit, because it names the same value the planner moved, is what makes the copy *somebody’s* rather than merely counterfactual. The learner has to arrive at that program with none of the three given to it. Section 4.4 says what it is given instead, Section 4.4.1 how the pieces assemble, and Section 4.7 what would count as having assembled them for the right reasons.

Another task family that requires attribution to solve are the *false-wall* tasks. Their trajectories have the shape of the obstacle task’s: value 1 (agent) bends around a cell on its way to the goal (green), exactly as if a wall stood there. But this time no wall reaches the grid at all: where the obstacle task’s program stamps a real one (Figure 3), here no frame of any scene ever contains a 3, because the wall is private to the agent: agent 1 wrongly believes the cell is blocked. The cell it bends around is instead occupied, for the whole trajectory, by an inert bystander (the purple 5), and the bystander is not what turns the agent aside. Walls are the only impassable value in this world, occupied cells are freely entered (see the goal-seeking and obstacle tasks above, which end with the agent stepping onto the goal’s own cell).

Why is the bystander there at all? A frame is only a snapshot, and a program is free to act between snapshots. So a devious program could reproduce these scenes without attributing anything: stamp a real wall onto that cell, exactly as the obstacle task’s program does, let the agent take its step, then erase the wall again before the next frame is rendered. The bystander closes this loophole. Stamping a wall on its cell would wipe it out, and nothing in the language can put it back — yet every frame shows it sitting there untouched. A program that reproduces these scenes must therefore keep the wall out of the world altogether, and the only place left for it is a model private to the agent (Section 7.8 makes this precise).

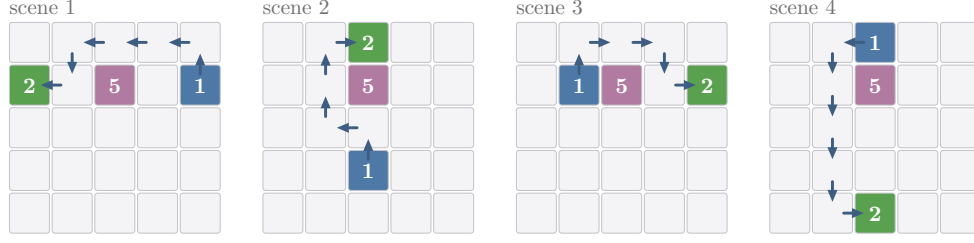


Figure 6: An instance of a false-wall task

Three further belief families vary the attribution rather than its content. In a *witness* task, the false-wall scene gains a second agent that is not misled: it seeks a goal of its own and walks straight across the cell the first agent bends around, which no real wall could permit. In a *two-observer* task, two agents seek the same goal and only one of them is wrong about where it is, so a solution must attribute a belief to one mover and withhold it from the other. And in a *false-obstacle* task the agent is wrong about two things at once, detouring around a phantom wall on its way to a displaced goal, while a real wall stands elsewhere in the scene — so a solution must hold apart, in one program, a wall that exists and a wall that is merely believed. Together with goal-displacement these are the five belief families of the corpus; Table 6 gives the program behind each, and Section 4.7 says which family carries which criterion.

A second, smaller group are *template tasks*. Where a trajectory task’s input is a single initial grid, a template task’s input is a pair of grids: a *working* grid and a *template* grid, both $s \times s$. The task is to produce the working grid modified according to the pattern given in the template. This format is loosely inspired by the ARC-AGI-1 corpus [CITE].

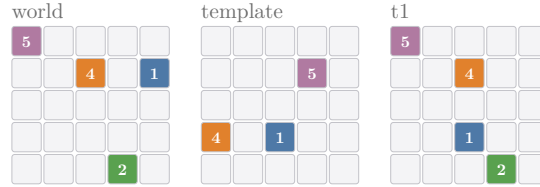


Figure 7: An instance of a registration task: the solution moves one named value in the working grid to the cell the template assigns it, and leaves everything else standing

The template families are in the corpus because they populate the pair interface from the non-mental side. A program that consumes a (working, template) pair is aligning two images, not holding a belief, and — as Section 4.4 will make concrete — it is doing so with exactly the parts a belief attribution needs. This is a policy the whole corpus follows: whether a piece of machinery is mental will be settled by which families pay for it (Chapter 5), so each family should be readable as a claim on some corner of the language. That is worth a tour, since the results chapter refers to these families by name.

Among the trajectory tasks: *physics* is the directional-movement family that opened this section, a body drifting one step per frame with nothing sought; *desire* is one value seeking another, the planner’s home family and the whole of the true-belief case; *flee* is the same planner at the opposite utility polarity, running away rather than toward; *obstacle* is desire plus a real wall (Figure 3); and *relocation*, *deletion* and *denoise* are grid edits with no mover at all — a wall taken down and re-stamped elsewhere, a cell cleared, a value scrubbed from the grid. Four further trajectory families exercise the pair machinery without any mental reading: *overlay* and *underlay* composite two layers

of one scene, differing in which layer wins where they collide; *comet* is a seeker that leaves a trail, its past positions composited behind it; and *wipe* opens a blank second channel rather than a copied one. Among the template tasks: *registration* (Figure 7) aligns one named value to the template; *perception* is the same alignment run in the opposite direction, the template updated with where the working grid actually has the value; *multi-registration* aligns every value the two grids share, and *registration-except* every value but one; *inpainting* fills the working grid’s empty cells from the template, and *readout* discards the working grid for the template outright; *layer-compositing* applies one edit to each grid before merging them; *drifting registration* aligns against a working grid that is itself scrolling; and *map-update* rewrites the template to match the working grid wholesale. None of these is filler: each was written to claim one operation on pairs, so that when Chapter 5 asks whether any non-mental family uses a given operation, every operation has a family that was given the chance to claim it (Table 3).

In numbers: the 168 belief tasks are 24 false-wall, 48 goal-displacement, 48 witness, 24 two-observer and 24 false-obstacle, and the remaining 148 are spread across the twenty non-mental families just toured. Every task consists of $k = 4$ scenes on 5×5 grids, and the (goal, agent) values rotate through eight distinct pairs, so that no particular integer comes to mean “agent”. The non-mental side is deliberately not uniform: obstacle is the densest family in the corpus at 48 tasks and relocation carries 16, because the wall-handling structure they share with belief must recur often enough for compression to abstract it — a curriculum choice, defended when it is questioned (Section 5.9). Table 6 in the appendix lists every family beside the ground-truth program that generated it.

4.3. Program space: type-directed enumeration

A hypothesis is a program, so the hypothesis space is the space of programs generated by type-directed enumeration. G is the type of 2d grids of cell values, the frames of a scene. Write G^* for stacks of grids, i.e. a full scene composed of a sequence of grids. So a transition function is then an element of $G \rightarrow G$. The cell values are members of the ground set $V = \{0, \dots, 9\}$, where each is an entity on the grid: value 3 is reserved as an impassable wall, while the other values are interpreted as agent or goal or any other entity. The grid coordinates are $C = \{0, \dots, s - 1\}$, a row or column index, from which a position in $C \times C$ is built. The remaining sets are D , the four movement directions, and $U \subseteq (C \times C \rightarrow \mathbb{R})$, the positional utilities that the planner scores (Section 4.4).

Everything else the language names is an arrow between these. The arrows relevant to our argument are the ones into and out of the product $G \times G$, which serves as the (world, model) channel pair, and they arrive in the next section as the types of the product primitives. Whether a program opens a private second channel is a matter of which of those arrows its nodes inhabit, so a (world, model) representation is a structural property of the program. The fixed product does carry the commitment that a learner holding a private representation holds exactly one (generalizing this is left as further work, see Section 6.2).

A program is checked by running it through an interpreter (Section 7.5). The solution to a trajectory task (Section 4.2) is a program of type $G \rightarrow G$, a transition function that given the task’s initial grid at step i should return the next grid at step $i + 1$. The interpreter iterates the program

$G \rightarrow G$ on the task’s initial grid G , yielding the stack of grids G^* which can be compared to the target scene. The interpreter’s state at any step is just a single grid G , not a (world, model) pair $G \times G$, and certainly not a registry of agents, goals, or entities. No pair is handed from one step to the next: a program that opens a second channel (a model channel that may diverge from the world) must build it, use it, and collapse it again inside that step.

A template task starts from a pair in $G \times G$, a working grid and a constant external template, so its solution is a *commit policy* $G \times G \rightarrow G$, one of the collapses of Section 4.4. The commit answers what each frame should do with the template. Since a type-directed enumeration yields programs of one type only, trajectory tasks ($G \rightarrow G$ -rooted) and template tasks ($G \times G \rightarrow G$ -rooted) are searched separately (Section 4.5.1). Though compression pools the solutions of both (Section 4.5.2).

4.4. The primitive library

The primitive library consists of grid primitives and pair primitives. The grid primitives are the interface that we hand-picked for our particular domain. Among them are a planner, a value mover, and the terminals a scene is made of. The pair primitives split in two, and only one of the halves was ours to pick.

One half is what the product $G \times G$ determines by itself. Having a product just *is* having the two projections, and the rest of this half follows from them by the universal property rather than by our choosing: the diagonal, the functorial actions, and the swap. The library holds a generating set for all of it — the universal pairing $\langle f, f' \rangle$, for instance, is the diagonal followed by the functorial action on both channels — so nothing the product determines is missing from the language, and the composers close it up.

The other half is the operations that consume a pair: the two unions, the two scope collapses, and the reader and writer from which a single-value commit is assembled. The product does not determine these. They inhabit arrow types the product creates, but what each of them does with a pair is a fact about grids rather than about products, so each is a choice we made. Section 4.4.2 gives the rule those choices follow, and Chapter 5 checks that every one of them is claimed by some non-mental family (Table 3).

Across both halves we impose one further discipline: every choice the pair layer offers comes with its opposite corner. The copy into both channels comes beside the pairing with a blank, the action on the second channel beside the action on the first, the projection that keeps the world beside the one that keeps the model, each union beside the union with the opposite precedence. This is a separate commitment from the completeness of the first half, and it is what the blank pairing is in the library for: the product supplies no distinguished empty grid, so that primitive earns its place as a mirror rather than as something the product forced. The discipline is what makes the learner’s eventual route through the pair a choice rather than the only road, since no route is cheaper than its mirror image, and Chapter 5 checks that the mirrors are not merely available but taken, by other families in the same run (Table 3). The remaining decision is where the decomposition stops — which compounds are handed over whole and which must be assembled from parts — and Section 4.4.2 states the rule that fixes it, together with the reason it leaves no room for tuning.

That is also why the library can be presented the way it is presented below: by saying what each operation does and giving its type, and not by giving the symbol the implementation spells it with. Nothing is lost, because there is nothing in these operations for a name to carry — what an operation *is*, here, is exhausted by its type and its effect on a grid, and the plumbing in particular is fixed the moment one says “the language has the product $G \times G$ ”. The exhaustive catalogue, every symbol’s name, signature and semantics under each endowment, is Section 7.6, for a reader who wants to count symbols rather than read descriptions.

Take the grid layer first. The *planner*, of type $U \times V \rightarrow (G \rightarrow G)$, is the one from Section 3.2: given a utility and a cell value it returns a transition function that at each time step moves that value one step toward the position maximising the utility. The two utilities it can be handed, both of type $V \rightarrow U$, are minus and plus the distance to the nearest cell of a named value, so the same planner approaches a value under the one and flees it under the other. Beside it are the movements and edits, all of them transition functions once their arguments are supplied: a constant *step*, of type $V \times D \rightarrow (G \rightarrow G)$, which moves a named value one cell in a named direction; *stamping a wall* at a position and *clearing* a single cell, both of type $C \times C \rightarrow (G \rightarrow G)$; *erasing*, of type $V \rightarrow (G \rightarrow G)$, which scrubs every cell of a named value from the grid; and *composition*, of type $(G \rightarrow G) \times (G \rightarrow G) \rightarrow (G \rightarrow G)$, which runs one transition function and then another. The terminals are the four directions, the ten cell values, and the five coordinates, a pair of which names a position.

Since what we are interested in is how transition functions $G \rightarrow G$ interact with (world, model) pairs $G \times G$, the rest of the library is the product: the maps it determines, and the operations we added to consume a pair once it is built. There are four jobs to be done with a pair, and an arrow for each: building a pair out of a grid, transforming a pair, collapsing a pair back to a grid, and reading and writing the positions a collapse goes off of. In what follows w and m are grids, f and f' transition functions, and v a cell value.

Two operations *build* a pair, both of type $G \rightarrow G \times G$. The diagonal $w \mapsto (w, w)$ copies a grid into both channels. Its complement $w \mapsto (w, \mathbf{0})$ pairs the grid with an empty one instead, so that the second channel starts as scratch rather than as a copy of the world.

Four operations *transform* a pair, all of type $G \times G \rightarrow G \times G$. Three are the functorial actions — the action of the product, as a bifunctor, on arrows — which lift a transition function to act on the channels: $(w, m) \mapsto (f(w), m)$ acts on the first factor, $(w, m) \mapsto (w, f(m))$ on the second, and $(w, m) \mapsto (f(w), f'(m))$ on both at once, with a separate function for each. The fourth is the *swap* $(w, m) \mapsto (m, w)$, the symmetry isomorphism of the product, which exchanges the channels, so that anything defined on one factor can be conjugated into a version acting on the other.

Several operations *collapse* a pair to a single grid, of type $G \times G \rightarrow G$. The two projections, $(w, m) \mapsto w$ and $(w, m) \mapsto m$, return one channel and discard the other. Two more union the channels cell by cell, differing only in which of them wins where they disagree. The last two are *scope* collapses. One moves every value the two channels share to the position it holds in the second; the other, of type $V \rightarrow (G \times G \rightarrow G)$, does the same for every shared value but one named exception. Each adopts the second channel’s account of where things stand, and they differ only in how much of it they adopt — all of it, or all of it minus one named value. The second therefore publishes by blacklist: the value it names is the one it holds *back*, and everything unnamed goes through. The

remaining point on that scale — adopting the second channel’s account of exactly one named value, a whitelist of one — is deliberately not on the list, and assembling it is what the accessors below are for.

Finally, the accessors on positions. A *reader*, of type $V \rightarrow (G \rightarrow C \times C)$, gives the position of a named value in a grid; a *writer*, of type $V \rightarrow (G \times (C \times C) \rightarrow G)$, gives the grid with that value moved to a given position. A third operation, of type $(G \rightarrow C \times C) \times (G \times (C \times C) \rightarrow G) \rightarrow (G \times G \rightarrow G)$, joins a reader to a writer: it reads a position off the second channel and imposes it on the first. Handed the reader and the writer for one and the same value v , it yields the collapse that moves v to wherever the second channel puts it and leaves everything else as the first channel had it. That collapse costs three symbols rather than one, since the learner must name the reader, the writer and the join — it is assembled, not handed over.

Each of these arrows needs its own composer, since the composition of the grid layer joins two transition functions only. Three are given, one for each arrow that needs one. A builder followed by a transform, of type $(G \rightarrow G \times G) \times (G \times G \rightarrow G \times G) \rightarrow (G \rightarrow G \times G)$, extends a builder. A builder followed by a collapse, of type $(G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$, is the composer that returns to $G \rightarrow G$; since the interpreter only ever iterates a transition function, a program that opens a second channel must close it again within the same time step, and this is what closes it. A transform followed by a collapse, of type $(G \times G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \times G \rightarrow G)$, is the only one that acts on a pair the program did not itself build, which is what a template task’s given pair calls for.

Nothing in any of these primitives distinguishes a world from a model. The pair is symmetric: both factors are grids, every map is present in both directions, and no primitive marks either channel as a particular agent’s private representation. Which channel is the world and which is a belief is settled only by how a program assembles these pieces together, which is found by search.

4.4.1. Composing belief attribution

In Section 3.3 we presented belief attribution as a two-step computation. First, the observer must *derive* a counterfactual grid which deviates from her own according to the agent’s false belief $m' = \text{derive}(m)$. Then the observer *runs* the agent’s movement policy on the counterfactual grid $t' = \text{optimize}(g, m')$ to yield the agent’s simulated trajectory which is optimal w.r.t the agent’s world m' .

Our model shows that this structure can be constructed by composing the $G \times G$ primitives from Section 4.4. In our model only the *run* step is done by a primitive, the planner, which builds the agent’s movement policy. Nothing in the library opens a private grid, so the *derive* step of creating and modifying a copy of the grid must be discovered by the learner.

Additionally, our model includes another step that isn’t represented in Section 3.3’s. In that model, the *attribution* of a belief to an agent is stipulated. For example, if an observer is simulating agent a ’s trajectory and given goal g and model m' , she just calculates $t' = \text{optimize}(g, m')$. That g and m' are attributed to

But in our model, we want to make the distinction between positing counterfactual a model in general vs. positing a counterfactual model and associating it with a particular agent, attributing it to that agent.

But this architecture adds a third step with no counterpart in Chapter 3: *commit* the result back to the world, publishing where the agent went and nothing else. The commit is owed to the interpreter rather than to the theory — it renders one grid per step, so motion on a private grid doesn't affect the rendered grid unless an agent's motion is explicitly committed from it.

and nothing publishes a single value's position back out of one. Both have to be assembled from the product structure above. So *derive* and *commit* give us two target structures, and it is worth saying what each has to do before asking whether the learner can build it.

The first is the *derive-and-commit frame*, of type $(G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$. Handed a derive and a commit, it copies the world into a pair, applies the derive to the second channel to get the private grid, and gives the pair to the commit, which collapses it back to one grid. The private grid exists only for the duration of that call. Assembled out of Section 4.4 it is the diagonal, then the derive lifted onto the second channel, then the commit, joined by two of the composers — four symbols of plumbing around the two slots that do the work.

The second is the *single-value commit*, of type $V \rightarrow (G \times G \rightarrow G)$. Given a value, it reads where that value sits in the private grid and moves the same value in the world to that position, so exactly one value's position is published and everything else stands as the world had it. This is the commit belief needs, and it is a reader and a writer for the same value, joined — three symbols.

Belief is the composition of the two, with the derive doing the agent's error and the commit doing the attribution. Taking a phantom wall as the error, the belief program for an agent value a that seeks a goal value g and is wrong about a wall at row r , column c , encodes

at each time step, copy the grid and stamp an impassable wall at cell (r, c) on the copy; in the copy, move a to the adjacent cell closest to g ; then in the world move a to wherever the copy left it, and change nothing else

and it has type $G \rightarrow G$, like every other transition function — nothing in its type announces that it is a program about a mind. Figure 8 traces the three steps through one transition of the false-wall task of Figure 6, one grid per intermediate state.

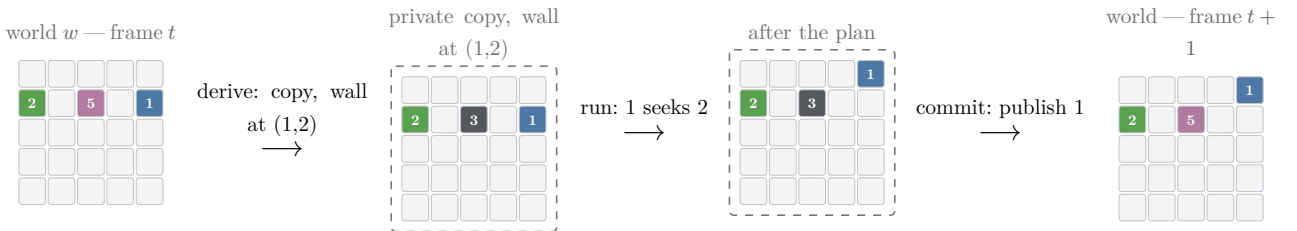


Figure 8: One transition of that program on the false-wall task of Figure 6, with agent 1 seeking goal 2 and wrong about a wall at row 1, column 2. *Derive*: the world is copied and a wall is stamped on the copy, overwriting the bystander 5 there — the dashed grids are the private channel, which is never rendered and does not survive the step. *Run*: the agent's move is planned in the walled copy, and it steps *up*, a step no plan over the actual grid takes: the bystander's cell is freely enterable, so in the world the straight step left is strictly better. *Commit*: only the agent's resulting position is written back; the wall never reaches a rendered frame, and the bystander stands untouched.

Note that the agent value a occupies three slots in that program: the slot that says whose move is planned over the privately walled grid, the slot that says whose position is read off the copy, and the slot that says whose position is written back to the world. So to find this compound, the enumerator needs to independently fill the argument slots of the three primitives with the same value placeholder. We call that coincidence — one value filling, at once, the slot that says whose move is planned and the slots that say what is published — the *agency signature*. It is the structural mark that separates an attribution (a private grid that is *somebody's*) from a mere counterfactual computation, and both the success criteria of Section 4.7 and the results of Chapter 5 are stated in terms of it.

4.4.2. Where the decomposition stops

Even twelve-month-old infants encode an action as directed at a goal state, and expect an agent to use the most efficient means available to it Section 1.2. They employ the teleological stance, which relates an action, a goal state, and the constraints of the situation but without attributing any representation to agents (Gergely et al., 1995; Gergely & Csibra, 2003). Under the teleological stance, the reality constraints that the observer consults are their own, not the agent's.

We return to this distinction in Section 4.6.3, when we defend our choice to hand our model learner a primitive planner, which moves a value one step along the best available path toward improving some utility, even though the planner is algorithmically complex. From the developmental literature we know that efficient goal-directed interpretation is in place around a year of age, three years before anything recognizable as false-belief attribution (Section 1.2). And on Gergely and Csibra's analysis it isn't yet ToM at all, it just says how a body moves given a goal and a layout.

The library of Section 4.4 cuts the language at conspicuously uneven depths. The commit that publishes one value must be assembled from a reader, a writer and a joiner, while the collapse that resolves an entire pair is a single token; one is entitled to ask why the knife fell where it did, since a granularity chosen freely is a granularity that could have been chosen to flatter the result. Two rules fix it, and neither leaves room for tuning.

The first rule is the completeness announced at the head of Section 4.4, and it covers the builders, the transforms and the projections. The two projections are what having a product *is*; the diagonal, the two functorial actions and their joint form, and the swap follow from them by the universal property, and the library holds a generating set for all of them, so the kit is complete rather than curated. The pairing with the empty grid is the one item here the product does not force, since it needs a distinguished blank grid, and it is present under the mirror discipline instead: each map comes with its complement — the copy with the blank, the second channel with the first, the world with the model — so no route through the pair is cheaper than its mirror image. At this layer there is nothing to decompose, and the only discretion we exercised was to include every mirror rather than a subset.

The second rule covers the collapses, where choices did have to be made, and it can be stated in one sentence: *the decomposition reaches exactly as far as the single-value vocabulary — read where one named value sits, write one named value to a position — can spell, and what that vocabulary cannot spell stays atomic*. The commit belief needs, publishing one named value's position out of the

private channel, has a spelling in that vocabulary, so it is withheld as a token and the learner must build it out of parts that each do non-mental work elsewhere in the corpus. The scope collapses have no such spelling: both fold over *every* value the two channels share, a set fixed by the scene rather than by the program, and writing them out would require granting iteration over V — a strictly more powerful construct than anything the language contains, and a far less innocent gift than the two collapses it would be used to spell. The two unions, which merge whole grids cell by cell, are atomic on the same grounds. So the rule is not “decompose everything”; it is that the compound under test must be assembled from parts, while its neighbours may be bought whole.

That policy has a consequence that should be stated here rather than discovered in the results: it prices the boundary belief draws asymmetrically. A commit can name what it publishes or name what it withholds — publish only this one value, or publish everything except this one value — and these draw the same kind of line from opposite sides, but the second costs one node where the first costs three. Nor are the two always distinguishable extensionally. On a scene where the agent is the only shared value the derive has moved, “publish everything” moves only the agent, and so denotes exactly the single-value commit; where the derive has also displaced one shared value, a believed-in goal, “publish everything but the goal” does. Wherever that coincidence holds, the cheap spelling and the assembled one compute the same collapse, and a description-length learner should be expected to buy the cheap one. The asymmetry does not bias the search toward belief — the atomically priced scope collapses are the commits of non-mental families (*multi-registration* and *registration-except* in Table 6), and within the mental families it taxes precisely the whitelist form of the agency signature — but it does mean that the *spelling* of a discovered commit is a prediction of the price list rather than a free observation. Chapter 5 reads it as such (Section 5.5.3), with the atomic control run (Section 4.5.3), in which both spellings cost one token, separating what the price explains from what the denotation does. It is the one place in the thesis where saying what a program does and saying how many symbols it takes to say so come apart, and both readings are needed.

4.5. Inference

Our procedure runs in alternating rounds of exploration and compression, the loop of (Dechter et al., n.d.) as in DreamCoder (Ellis et al., 2020). The exploration phase performs per-task inference, enumerating programs under the current library for the tasks not yet solved (Section 2.4). The compression phase revises the library to compress the programs found (Section 2.5). We omit DreamCoder’s recognition model, for reasons explained in Section 7.10.2.

4.5.1. Search

The exploration phase finds a hypothesis that correctly explains a scene, by enumerating candidate programs in decreasing order of prior probability and checking if the program produces the scene. A hypothesis is a program (Section 2.3), represented as a typed expression tree over the current library (Section 4.3). Since enumeration in decreasing prior probability is enumeration in increasing description length, the search walks outward in expanding bands of cost (expanding $\sim e^\ell$, see Section 2.4). The first program it finds that reproduces the data is the shortest one that does, which is the maximum a posteriori hypothesis (h^* , see Section 2.3). The enumeration runs under a time

budget, and a hypothesis whose cost band is not reached before the budget expires is simply not found (Section 2.4).

The library fixes the prior. The prior $P(h) \propto e^{-\text{DL}(h)}$ (see Section 2.3) is realised as a distribution over the library’s symbols, under which a program’s cost is the summed $-\log p$ of its nodes. Symbols are grouped by return type, and a symbol of type τ is priced at $-\ln$ of the number of symbols sharing τ , i.e. a type-uniform distribution⁷. So the types give the branching factor at each argument slot, and the cost of each choice at that slot. Enumeration is type-directed: an argument slot draws only from symbols whose type matches, so no ill-typed tree is ever built.

A candidate program solves a task only if the interpreter, started from each scene’s own first frame, reproduces that scene’s every frame exactly, for all k scenes (Section 4.2). So the indicator likelihood $\mathbb{1}[h \text{ solves } d]$ (Section 2.2) is evaluated against all k scenes.

4.5.2. Library revision

Where exploration minimizes description length one hypothesis at a time against a library held fixed, compression minimises the total description length. It minimizes the length of the library itself plus the summed cost of every solved program written in it (Ellis et al., 2020), the posterior over languages (Section 2.5).

$$\text{DL}(\text{library}) + \sum_{\text{solutions}} \text{DL}(\text{program} \mid \text{library})$$

When a syntactic structure is common to a lot of programs, it gets added to the library as a new term. So the description length of the solutions shrinks, since rewriting the solutions with the new term replaces the common subtrees with that term (so $\sum_{\text{sols}} \text{DL}(\text{program} \mid \text{library})$ shrinks). But the library is now larger, and since a symbol’s probability falls as the number of symbols competing with it rises, every program written in the enlarged library gets a little more expensive (so $\text{DL}(\text{library})$ grows). This is the tradeoff that needs to be made when deciding whether to abstract some shared structure as a new primitive. The two terms ensure that an abstraction is kept only when the tokens it saves across the whole corpus outweigh the price of adding it. So we’re less likely to abstract incidental structure, and more likely to abstract structure that’s relevant to the corpus.

DreamCoder looks for abstractions bottom-up, enumerating refactorings of each found program and intersecting them (Ellis et al., 2020). Instead we delegate the search to the top-down Stitch (Bowers et al., 2023) library-learning system, which searches the space of abstractions directly and so never has to materialize every rewrite. Also, note that enumeration searches by root type (Section 4.3), so solutions to trajectory tasks (transition functions $G \rightarrow G$) and template tasks (commit policies $G \times G \rightarrow G$) are searched for separately. This is just an implementation necessity which doesn’t affect the overall outcome, since compression searches for common structure among the full pool of solutions together regardless of root type.

Compression returns is a set of abstractions together with the corpus rewritten to use them. Each abstraction is registered as a new symbol, on the same footing as a primitive that was given

⁷One exception to the uniform distribution is that a cell value which is visible in the task’s first frame costs nothing, since naming what is already on display is not a hypothesis about anything. But a placed wall is full-price, since it’s positing something that isn’t there

— what was a subtree found in round N is an atom in round $N + 1$ — and the rewritten corpus becomes the starting point for the next round — in which programs a level deeper, belief compounds too expensive to enumerate from bare primitives, have become reachable because their shared core now costs a single token. This is the cumulative structure Section 2.5 called bootstrapping: each round changes what the next round can find.

4.5.3. The two endowments

The loop just described is run twice, and every result of Chapter 5 belongs to one of the two runs. What differs between them is the *endowment*: the primitive library the learner starts from. Both endowments include the grid core — the planner, the movement and edit primitives, the scope collapses and the terminals (Section 4.4, with the exhaustive catalogue in Section 7.6). They differ in the vocabulary for pairs.

The *combinator endowment* is the library Section 4.4 presented, and its run is the main result of this thesis. Its pair vocabulary is the product structure and nothing besides — the two builders, the channel maps and the swap, the projections and the unions, the scope collapses, the position accessors and the composers, each with its symmetric complement — so both target compounds of Section 4.4.1 must be assembled: the derive-and-commit frame out of four symbols, and the single-value commit out of three.

The *atomic control* is the same library with the product combinators replaced by the two compounds bought whole, as single tokens of type $(G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$ and $V \rightarrow (G \times G \rightarrow G)$ respectively. These are the only two places in the thesis where a machine name is worth carrying, since the runs print them and Chapter 5 quotes them: the searcher calls them `fork` and `sync_to_world`, and they are the derive-and-commit frame and the single-value commit of Section 4.4.1, granted rather than assembled (the full atomic pair interface is Table 14). This is the endowment an unsympathetic reader would call **believe** handed over in two halves, and that is exactly why it is run: not as a second result but as a control. Under it a belief program is a two-symbol composition, so its run shows what the criteria of Section 4.7 look like when the structure is nearly given. And it calibrates the main run’s misses: every program the atomic vocabulary writes has a combinator spelling (the identities of Section 4.4.1 are verified before either run begins), so a task the control solves and the main run misses is a task lost to the search budget rather than to the language — a distinction no single run could draw. One asymmetry follows from the granting: three template families — layer-compositing, drifting registration and map-update — have no program at any length under the atomic pair interface, which cannot act on the channels of a given pair, so the control faces 304 of the corpus’s 316 tasks.

Everything else is held identical between the runs, and the parameters are these. A run is up to six rounds of the exploration–compression loop. In each round, every still-unsolved trajectory task is enumerated for up to 3600 seconds; round 1 is capped at 1200 seconds, the calibrated worst-case solve time of a task *given* that its abstractions exist — in round 1 none exist yet, so a longer first round buys deep extensional programs, never the belief compound. The template tasks share one 60-second enumeration sweep per round. The compression step runs *Stitch* for up to six abstractions per round, jointly over both solution types (Section 4.5.2). A task counts as solved only when all $k = 4$ of its scenes are reproduced exactly (Section 4.5.1), and no recognition model is trained (Section 4.6.4).

Each endowment was run once — a single seed, not an average, a limitation Section 5.10 weighs — on the same corpus, with the same task seeds, on one 32-core node.

4.6. The model does not encode a Bayesian theory of mind

The goal of our model is to recover the belief attribution structure from Chapter 3 without stipulating it ourselves. A natural objection to our model’s results would be we’ve merely re-encoded Chapter 3’s stipulations in a different notation so the learner trivially “finds” them. To pre-empt this, we’ll argue that no notion of belief attribution, agency, or world/model distinction is encoded in our setup (between the interpreter, the types, the library, the corpus, or the prior). Of course we have to give the learner some things, but connecting the given machinery to create a belief attribution is left to the learner.

BToM hands agent, goal and belief to a planner as separately inferred arguments. Our model doesn’t have a planner with argument slots to inherit that from, and no type denotes an agent, a goal or an attitude. There are just grids, cell values, coordinates and arrows between them. The thing we call the agent is whatever value occupies the policy slot and the commit slot at once (Section 4.4.1). A function binding one hole to both the policy and the commit isn’t given as a primitive. It’s a compound that the searcher needs to assemble enough times so that compression abstracts it.

BToM evaluates the planner against a supplied world model m' and permits $m \neq m'$. What we give instead is a product type and general combinators that inhabit it. The pair is symmetric and content-free — the diagonal produces (w, w) , and nothing in it says the second copy is a *model* or that anyone *holds* it. Belief is a matter of wiring: acting on the second channel rather than the first, committing directly rather than conjugating through the swap, publishing one named value rather than every value the channels share. Each of those choices has its opposite in the library, exercised by some non-mental family in Table 6; the medium is neutral, and the asymmetry is made in program space by the search.

The interpreter does not supply it either. In BToM the divergent model is an input to a planner the observer runs *inside itself*; ours has one grid of state, no second channel to hand anybody, and no mode distinguishing evaluating a program from evaluating a program-as-an-agent-would. A private grid exists only because a program built it, and the interpreter cannot tell it from any other. Nor is either target compound of Section 4.4.1 given as an atom — that is the atomic control run (Section 4.5.3) — since the searcher here is handed the diagonal, the channel maps, the composers and the position accessors, product-category combinators no one would describe as mental.

A stronger form of the worry puts the concession one level down: the ability to build a copy of the world that differs from the world just *is* the representational capacity theory of mind consists in. It is not. Building a world state that differs from the actual one is counterfactual reasoning — the capacity to consider what would be the case if things were otherwise — and it is domain-general, with a developmental record of its own well before false-belief attribution. Theory of mind is an abstraction that *uses* that capacity, by attributing the divergent world to a particular agent, so that it is *that agent’s* world. That attribution is the conjunct the shared hole encodes, and it is exactly what the pair does not supply: the derive-and-commit frame will as happily derive a private grid,

compute over it, and publish an unrelated value, and the language will let you write that. What it will not do is tell you not to.

The regress does not end, and we do not claim otherwise. The diagonal could itself be decomposed into reads and writes over a flat memory, so that holding a second world model becomes a discovered pattern of buffer use; but “allocate a buffer” is then the primitive, and no more mental than “form a pair”, because intensionality just *is* the coexistence of a world and a model of it, so any substrate expressive enough to state the structure can hold two things at once. The pair is to theory of mind roughly what the integers are to arithmetic: not a disguised form of the thing to be learned, but the medium without which it cannot be stated. There is no substrate in which theory of mind is discovered from nothing, only substrates whose primitives are, or are not, individually non-mental and general. Call this pair of replies — attribution is a further conjunct on top of counterfactual reasoning, and decomposing the medium only relocates the regress — the *no-separation point*; it is what remains of the objection once measurement runs out, and Chapter 5’s objections section leans on it by that name (Section 5.9).

4.6.1. The space of admissible divergences

BToM must also fix which divergences are admissible, and the scenarios that produce them. Here there is no range to fix: the derive slot accepts *any* transition function $G \rightarrow G$, so the belief families differ in what the belief is *about* rather than in some parameter of a common schema — a phantom wall is a wall stamped at a coordinate, a displaced goal is a one-cell shove of the goal value, and being wrong about both at once is the two composed. The goal-displacement family varies its content across four shove shapes, so an abstraction specialised to one of them would not recur and compression must keep the content subtree as a hole: the productivity and systematicity Section 1.3 demanded.

The concession is that what the learner finds is a divergent world model, not a belief that is *formed* and could be *revised*. Nothing represents how the agent came to be wrong, and nothing updates when it looks again. In BToM that lack is a stipulation about admissible divergences; here it is a limit on what the corpus depicts and what a single-frame transition function can express, and the most important respect in which the discovered structure is thinner than the thing it is a model of (Section 6.3):

The second concession is about what is being asked for. BToM’s m' is a belief in the full sense: a state the agent came to be in, that could have been otherwise, and that would be revised if the agent looked again. What the learner here is asked to find is thinner — a divergent world model, attributed to an agent and acted on by that agent’s policy, but with no account of how the agent came to hold it and no machinery for updating it. Nothing in a single-frame transition function represents perceptual access, or a record of what was witnessed, or an update on being shown the truth. That is a real gap between the discovered structure and the thing it models, and it is the most important one; Section 6.3 takes it up as further work.

4.6.2. The space of goals

BToM’s goal space, and the prior on it, are given; Baker et al. compare several goal priors precisely because the choice matters and is the modeller’s to make (Baker et al., 2009). Here a goal is whichever cell value happens to fill the argument of the attracting utility, drawn from the same ten integer

terminals that supply agent identifiers and step magnitudes. There is no goal type and no goal prior — there is inference over programs, and a program seeks something because a symbol landed in that slot. Even the polarity is a search choice: the repelling utility sits alongside the attracting one, so maximising utility flees a value as readily as it approaches one, and the *flee* family exists to keep that corner inhabited. The only prior over any of it is the type-uniform distribution, which prices a goal value by the same rule that prices a direction or a composition.

4.6.3. Rationality, and the one thing we do grant

Rationality is the one stipulation of BToM our learner also receives, in the form of the planner primitive, on the grounds Chapter 3 gave: it is the only item on the list with independent developmental support, in place well before the first birthday and some three years before anything recognizable as false-belief attribution (Section 3.1, Section 1.2) (Gergely et al., 1995), and on the best analysis of it not mentalistic, since the teleological stance relates an action, a goal state and the constraints of a situation without attributing any representation to the agent (Gergely & Csibra, 2003).

Its signature makes the point formally. The planner has type $U \times V \rightarrow (G \rightarrow G)$: it receives one grid, has no parameter identifying whose grid it is and no access to any other channel, and treats whatever grid it is handed as the layout. It cannot represent a divergence, because it cannot see two things to diverge, and it cannot attribute anything, because it has no argument for an attributee. Belief arises only when something *else* hands it a grid that is not the world and then publishes only part of the result — and that something else is the composition, which is what the learner has to find. Nor is the planner reserved for the mental families: *desire* uses it to walk to a goal, *flee* to run from a hazard, *comet* to bend a trail, and *obstacle* to detour around a real wall, on scenes whose rendered trajectories closely resemble the belief ones. Availability is not use.

The residual cost is real and we do not pretend otherwise: the planner packs a full breadth-first search into one token, and a more elementary treatment would decompose it into memory, conditionals and interaction with the environment (Section 6.4). What we claim is only that granting a planner is not granting a theory of mind.

4.6.4. Two confounds that are ours rather than BToM’s

Two further items answer to nobody on Chapter 3’s list; they are properties of how the experiment is run, and the two knobs a sceptical reader would reach for first.

The success criterion admits no partial credit. A candidate program either reproduces all k scenes of a task cell for cell or it does not solve the task (Section 4.2, Section 2.2). A graded likelihood is exactly where a thumb could rest on the scale: under partial credit a mental program that is nearly right could be preferred to an extensional rival that is exactly right, and the comparison of Section 4.7 — whether the mental program wins on description length *among programs that all reproduce the data* — would no longer be the one being made.

Nothing in the loop is conditioned on what a scene looks like. We omit DreamCoder’s recognition model, for the reason Section 7.10.2 gives at length: a proposal distribution trained on solved tasks has its prior over the library fit to the data, and belief is precisely the structure the corpus is meant to leave unfitted. Enumeration runs under the type-uniform prior over the current library and nothing else, so the only thing that can make a belief compound reachable is that earlier

compression made it short — the mechanism the ordering claim of Section 4.7 depends on. The choice was settled by measurement rather than by design, and Section 5.10 reports what the loop gave when it was run both ways.

4.6.5. Two structural differences worth naming

Inference here is not agent-directed. BToM’s posterior is over (g, m) for an agent and has nowhere else to point; ours is over programs for a scene, and it is the *same* search, under the same prior and the same success criterion, that solves *denoise* and *relocation*. Compression is undirected in the same way, which is why it is pooled rather than run per solution type (Section 4.5.2): were it divided, the division would itself bias abstraction toward belief, since the belief families make up a larger share of the trajectory tasks than of the corpus as a whole. Pooled, the budget that could have paid for belief’s structure is free to spend itself on the overlay, registration, obstacle and relocation families instead.

The ordering is a prediction rather than an accommodation. Because BToM’s structure is complete from the start, nothing in the framework makes a goal attribution earlier or cheaper than a belief attribution, and the delay of Section 1.2 has to be explained by something outside the model. Here the delay is the mechanism: a belief compound is a deep composition whose cost under the initial library puts it far outside any feasible budget window, and it becomes reachable only once earlier solutions have been compressed into tokens that shorten it. That is a claim that can fail — the compounds might never come within budget, or they might come within budget in the wrong order — and Section 4.7 says what failure would look like.

4.7. What would count as success or failure

The measurements of Chapter 5 are tests rather than descriptions only if what they test is fixed in advance. This section fixes it.

What counts as having discovered belief. Not that the belief tasks were solved. A solved belief task with an extensional program is a failure of the corpus, not a success of the learner. The criterion is structural, and it is a condition on the invented term: an abstraction must enter the library whose body (i) opens a private channel, (ii) derives from it a world state that diverges from the actual one, (iii) evaluates an agent’s policy against that divergent state, and (iv) commits only the resulting action — not the divergent state — back to the world, with the agent value shared between the policy and the commit. All four conjuncts are checkable on the term, and the fourth is the one that distinguishes attribution from mere counterfactual computation.

It must be a compression win, not merely reachable. Reachability is cheap: given enough budget, many things can be enumerated. The discovered compound has to be *selected* by the joint objective of Section 4.5.2 over the non-mental rivals the same library can express, and the margin must be reported in nats against the shortest such rival — including the rivals that survive the k -scene filter for structural reasons, such as the transient-wall spelling and the witness-agent construction. A margin at or below zero is a failure even if the mental program was found first.

It must generalize behaviourally. A learned belief term should predict, on a scene held out of the run entirely, that the agent goes where it *believes* the goal to be rather than where the goal is. This is the false-belief test proper (Wimmer & Perner, n.d.), and it is the criterion that separates

a compressive coincidence from a term that means what we say it means. It also has to be run on a family where the two readings come apart behaviourally, which is why the goal-displacement family carries it: in a false-wall scene the transient-wall rival reproduces the whole trajectory, so only description length separates the readings there, whereas a displaced goal sends the two readings to different cells.

What would refute the hypothesis. Any one of the following:

1. no belief-structured abstraction, in the four-conjunct sense above, enters the library within budget;
2. one enters, but loses on description length to an extensional rival expressible in the same library;
3. a rival that never opens a private channel reproduces all k scenes of a mental family — which would mean the family was misclassified and the corpus does not test what it claims to;
4. the discovered term fails the held-out behavioural probe, sending the agent to the true location;
5. the discovered term is not applied selectively — if the library invents an agent constructor and then applies it to every mover in a scene, including one whose walk is fully explained by the bare world, then what was found is a habit of forking rather than an attribution.

Two things should be said now rather than left to the results chapter, because they bear on how the verdict should be read.

The first is that there is a depth past which the model’s expressive claim outruns its search claim. Two agents holding contradictory false beliefs, each detouring around its own phantom wall, is a scene the language states without difficulty, but it names six latent literals across two nested forks, and no amount of budget we could give it brought a single such task within reach. The selective-attribution criterion above is therefore carried by the cheaper two-observer case — two agents, one goal, one grid, where only one of them is attributed a belief — and the contradictory-belief case is left as a limit, not a result.

The second is that the criteria are stated per endowment, and a verdict is a verdict about one of them. They are the criteria the library of combinators must meet. The atomic control run (Section 4.5.3) is held to the same list and reported against it, but meeting them there does not discharge them, since the granted `fork/sync_to_world` pair is exactly what an unsympathetic reader would call `believe` in two halves.

Chapter 5

Results

In Chapter 3 we introduced a model endowed with the ability to attribute beliefs to agents. In Chapter 4 we introduced our model of the *acquisition* of that ability. In our model, to acquire this ability is to learn a new primitive that encodes the process of belief attribution. We stated that process as: copy the world grid, edit the copy to be a counterfactual model, plans an agent’s movement against the counterfactual copy, and publish the agent’s movement but not the falsehood that caused it (Section 4.4.1). In this chapter we show that the minimum description-length objective (Section 4.5.2) selects into the learner’s library an abstraction of this shape, of the shape stipulated in the BToM model.

5.1 What the loop finds

The first enumeration phase solves 144 tasks, all of which can be solved extensionally. The belief families are, at a budget of 1200 s, out of reach entirely. The compression step then runs over those 144 purely non-mental solutions and returns six abstractions. Among them are exactly the two target structures of Section 4.4.1: the single-value commit and the derive-and-commit frame.

The first is the *single-value commit*, of type $V \rightarrow (G \times G \rightarrow G)$. It takes a cell value and returns a collapse policy, a way of collapsing a pair of grids into one. Given the value, it reads where that value sits in the second grid and writes the same value in the first grid to that position, leaving everything else as the first grid had it.

The second is the *derive-and-commit* frame, of type $(G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$. It takes a transition function $G \rightarrow G$ and a collapse $G \times G \rightarrow G$ and returns a transition function. It builds a pair from a grid, applies the transition function to the copy only, and then collapses the two back into one grid according to the given collapse policy.

The two new primitives, single-value-commit and derive-and-commit, provide everything that the belief attribution process needs from the pair $G \times G$: the capacity to hold a second grid, and the capacity to move one specific value from one grid to another. These functions weren’t found from common structure among solutions to belief tasks (none of those tasks have even been solved yet). They were abstracted from the solutions of simpler tasks from the image processing domain, from solutions to picture processing tasks (overlay, underlay, and comet families) and image registration tasks.

So round 2 starts with the two new primitives in its enriched library, and in enumeration the learner solves 75 belief tasks. When compression is run, among the abstractions it returns is a function of the form laid out in Section 4.4.1 (we call it the *two-actor constructor*). It has type $V \times D \times V \times V \rightarrow (G \rightarrow G)$: it takes a believed-about value b , a direction d , a believer a and a second

actor n , and returns a transition function. The four letters are holes, since the abstraction is not a program about particular cell values but a frame that takes them as arguments. The transition function it returns encodes:

at each time step, n moves to the adjacent cell closest to b ; then, in a private copy of the grid in which b has been displaced one cell in direction d , a moves to the adjacent cell closest to b ; then every value the two grids share, except b itself, is moved to where the copy has it

so that the believer moves toward where b is not, both actors' moves reach the rendered world, and the displacement, being b 's alone, never does.

Count how often b occurs in that sentence. The **content** of the belief fills four argument slots at once: it is the value that is displaced on the private copy, the value each of the two actors' plans pursues, and the one value the commit withholds. Four slots the searcher fills independently, and one value in all of them. That is the coincidence Section 5.1.1 is about; the abstraction's body, in which those four appear as the numbered holes the searcher fills, is Table 8.

What this buys is that an entire belief task is now one symbol with four arguments. Supplied with the value 2, the direction up, the value 1 and the value 6, `fn_7` reads "agent 1 believes 2 is one cell up; agent 6 knows better" — a sentence about who believes what, written in a language that a round earlier had no word for either. Rounds 3 and 4 take the remaining 57, and the run converges at 132 of 168 belief tasks. The library it converged to is Table 1.

Name	Symbol	What it does at each time step	Belief solves
phantom-wall constructor	fn_9	on a private copy of the grid with an impassable wall stamped at cell (r, c) , a moves to the adjacent cell closest to g ; then every shared value is moved to where the copy has it	75
two-actor constructor	fn_7	n moves to the adjacent cell closest to b ; then, on a private copy in which b has been displaced one cell in direction d , a moves to the adjacent cell closest to b ; then every shared value but b is moved to where the copy has it	26
displaced-goal constructor	fn_10	the same, without the second actor	22
open constructor	fn_6	on a private copy altered in some unspecified way, a moves to the adjacent cell closest to g ; then the pair is collapsed by some unspecified commit	12
seek block	fn_8	some unspecified edit is made to the grid, and then a moves to the adjacent cell closest to g — all of it in the world, with no copy	46
agent commit	fn_11	v in the world is moved to wherever the second grid of a pair has it, and nothing else changes	9

Table 1: The library the combinator run converged to. Free letters are the abstraction’s holes: a an agent value, g a goal value, b a believed-about value, n a second actor, r, c a wall’s coordinates, d a direction. The English names are ours; the run’s own names, its argument orders, its type signatures and its verbatim bodies are Table 8. The first four all open a private copy: the two-actor constructor is the agency signature proper, with one hole bound four ways; the displaced-goal constructor is the same without the second actor; the phantom-wall constructor differs in what is done to the copy and in how the publication is spelled (Section 5.5.3); the open constructor leaves both of those as holes. The last two open no copy, and are exactly what the non-mental families donate — the seek block is the policy belief shares with the obstacle family, through which 47 obstacle solves are also rewritten, and the agent commit is the single-value publication reassembled from the registration parts.

Four of the six abstractions open a private copy, and they differ only in what is done to that copy and in how the publication back to the world is spelled. The two that do not are exactly the parts the non-mental families donate: the seek block, which is the desire family’s own program with room for one edit in front of it, and the agent commit, which is image registration’s program — the sharing Section 5.6 takes up.

The publications are worth pausing on, because two of the three do the same job by opposite means. One draws the line by naming the **agent**: publish this one value and nothing else. The other

two draw it by naming the **content**: publish everything, or everything except the one value the agent is wrong about. On a scene where the private copy differs from the world only at the misrepresented thing and at the movers, those come to the same thing, since there is nothing else left for a wholesale publication to move. Section 5.5.3 checks that they do, one scene at a time, and explains why the searcher prefers the cheaper of them.

This is the bootstrapping structure of Section 2.5 in its clearest form. An abstraction discovered in round N is an atom in round $N + 1$, and the round-1 atoms are precisely the two capacities — hold a second grid, transfer one value between two grids — that belief needs and that non-mental tasks independently pay for. Nothing schedules this order; it falls out of what is cheap at each stage.

The atomic control run (Section 4.5.3) can now be dealt with in a paragraph, which is all a control is owed. There the learner is handed the derive-and-commit frame and the single-value commit outright, as the single symbols **fork** and **sync_to_world** rather than as things to be assembled — this is the endowment Section 5.9 has to answer for — so a belief program is a two-symbol composition, in which the agent value appears twice rather than three times. It is not a second result and the argument does not rest on it; its use is as an instrument, and it does two jobs. Because the two vocabularies express exactly the same set of programs, a task the control solves and the main run does not is a task lost to the **search budget** rather than to the language — a distinction no single run can draw. And it prices the same non-mental rivals with the primitives cut at a different depth, which is what shows the result is not an artefact of where we chose to cut them.

Starting a rung up, the control run does everything a round earlier. Cheapest-first enumeration from the bare primitives takes 55 of the 168 belief tasks in round 1, and here the cost of a belief program can be read off directly: walking the programs in order of increasing description length, the obstacle family’s compound is reached at position 859, while the belief compound built on the very same phantom wall is not reached until position 297,342. Same wall, same planner, one extra attribution — and three hundred thousand programs of distance. That is the abstract point of Section 2.4 made concrete, and it is why round 1 gets 55 tasks rather than all of them. The round-1 compression step then invents the same agent constructor — alter a private copy of the grid in some unspecified way, move a to the adjacent cell closest to g in that copy, then publish a ’s new position and nothing else — with the content of the belief left as a hole and the agent value bound twice, and round 2 takes the remaining 100. It converges at 155 of 168 belief tasks and 291 of its 304 tasks overall, against the main run’s 132 and 276 of 316, and its solve set on the shared tasks strictly contains the main run’s; its library is Table 10, in the appendix. From here on it appears only where a control earns a mention: the same measurement at the second granularity, or the budget-not-language argument of Section 5.8.

5.1.1 The agency signature

Under either endowment the invented term has the same shape, and the shape is a coincidence in the grammar that the objective declined to treat as one. The constructor takes an agent value a , a goal value g and an unspecified alteration, and returns a transition function which, at each time step, makes that alteration to a private copy of the grid, moves a to the adjacent cell closest to g in the copy, and then publishes where a ended up and nothing else. The agent value a lands in

two places at once, and the whole result turns on the fact that nothing made those two places the same place.

Consider what fills them. The planner needs to be told whose move to compute, and takes a cell value for it. The publication needs to be told what to publish, and takes a cell value for that. Two different arguments, on two different operations, doing two unrelated jobs — one is a question about planning, the other a question about rendering. They happen to be of the same type V , so the search fills each of them independently from the same ten cell values, and it is free to fill them differently. Ninety of the hundred ways of filling the two slots put different values in them. Nothing in the grammar, the types or the interpreter prefers the diagonal.

Yet in every abstraction the compression step kept, the two are one hole. How many ways that one hole is bound depends only on how the publication is spelled. The atomic control’s constructor binds it twice: planner and published value. In the nine solutions that assemble the publication from the registration parts — a reader that finds a named value in the copy and a writer that puts a named value somewhere in the world — the same value lands in the reader and the writer too, and the coincidence is threefold. In the two-actor constructor it is fourfold. The two runs pin the same boundary from opposite sides — the control names it by the agent, publish this value; the main run by the content, publish everything but this — and either way the boundary has to be **drawn**, by an agreement between independently fillable slots that the objective was under no obligation to keep.

That collapse is what makes the term an attribution rather than a computation. A program that derives a counterfactual grid, plans on it, and publishes the result of planning for **some other** value is expressible, is no more expensive to describe, and does not occur. What compression kept is the version in which the value whose behaviour is published is the value whose reasoning was simulated — and, in the content-side spelling, the version in which the value withheld from the world is exactly the value misrepresented in the model. Which is to say: the version in which the private grid is **somebody’s**, wrong about **something in particular**.

Nothing in this identifies the term as **belief** in any thicker sense, and on the account of intentionality this thesis has been working with, nothing needs to: what makes a state a belief is the role it plays in the predictive scheme of the intentional stance, not an intrinsic mark it carries (Dennett, 2002). The invented term is a state that is derived privately, is planned against, is attributed to one value rather than to the scene, and predicts where that value will go. That is the role, and asking whether the thing filling it is **really** a belief is asking a question the framework declines.

5.2 The discovered term against BToM

Section 3.4 listed what BToM assumes in order to model mature performance; the list is now a list of things to look for in the invented term. Table 7 in the appendix sets the correspondence out item by item, and it runs as follows.

BToM’s planner maps a utility and an environment model to an action. The learner’s is the transition function $G \rightarrow G$ that moves one named value to the adjacent cell closest to another, applied to whatever grid it is handed, and this is the one item we grant outright, as a primitive, for the reason Section 4.6.3 gave. The environment model m' that BToM supplies to that planner as an input is here the second factor of the pair $G \times G$, and it is the only factor the alteration writes:

the invented abstraction alters the second factor and not the first, so the world passes through untouched. Which of the two channels is the model is therefore something the search settled rather than something the architecture declared. The same goes for the divergence $m \neq m'$ that makes false belief expressible at all: in BToM a permission written into the architecture, here an arbitrary transition function applied to the private copy only — and the opposite choice is not merely available but exercised, since Section 5.5 reports the families that transform the world channel instead. The goal fares differently. BToM ranges g over a given space under a given prior; in the discovered term the goal is whichever member of V happens to land in the slot naming what the planner moves toward, priced like every other symbol. It is not a space at all — there is no goal type and no goal prior (Section 4.6) — but an argument position of type V , with ten terminals that could fill it. And the attribution is the item the previous section turned on: BToM stipulates that m' is *this agent's* model, and the invented term has the shared hole instead — the atomic constructor binding the planner's actor to the published value, the combinator one binding the alteration, the plans and the publication's exception to one content value — either way, one value pinned to both sides of the model/world boundary.

Three further pieces of BToM's machinery have counterparts of a different sort. Its nested evaluation — the observer running a planner inside itself — is here structural, in program space: the private copy lives for the duration of one call and is then discarded, never rendered, and the interpreter has no such mode of its own (Section 7.5). Its inverse planning, which infers (g, m) from a trajectory by Bayes, is here cheapest-first enumeration under the library prior against an indicator likelihood: the inference of Section 2.2, unchanged, and not agent-directed, since the same procedure solves denoising. And the last piece has no counterpart. Belief in BToM is a state that is *formed* and could be *revised* the model discovered here exists for one transition and has no history. That is the honest gap, and Section 5.10 returns to it.

Set out that way the invented abstraction is a Bayesian theory of mind's agent, written in a language that had no word for one. The difference from Chapter 3 is where each item came from: BToM's are architecture, fixed before any scene is seen, and its inference locates a point inside them, whereas three of them here — which channel diverges, whose model it is, and what the divergence consists in — are decisions made in program space by a search that also solves image registration and denoising, and kept by an objective that had the option of spending its budget on the overlay and relocation families instead.

Two grants will be pressed. The planner we grant for the reason Chapter 3 gave: on Gergely and Csibra's analysis the teleological stance relates action, goal and situational constraint without attributing any representation to anybody, and it is in place by twelve months, three years ahead of false-belief attribution (Gergely et al., 1995; Gergely & Csibra, 2003). The results bear that out — the planner appears in the desire family (16 solves), the flee family, the comet family and all 48 obstacle solves, scenes nearly indistinguishable at the level of the rendered trajectory from the false-wall scenes and containing nothing mental at all. What separates the mental families is not that they use the planner but what is wired around it. The pair we also grant, but copying a grid yields two identical grids, which says nothing about which of the two is a model or who holds it:

the asymmetry is entirely in the composition, and Section 5.5 shows the symmetric alternatives are not merely available but **used**, by other families in the same runs.

5.3 The objective selected it

The guiding thought behind the whole project is that an intensional theory — one that attributes a belief to an agent — is the most compressed available description of that agent’s behaviour, and that this is no coincidence: the agent navigates by its own representation of the world, so a theory that posits that representation is tracking the process that actually generated the data. One can describe the same trajectory without attributing anything, but the description has to account for each step of the walk separately, and it will not transfer to the next scene. The prediction is that the description-length objective should prefer the intensional reading, and prefer it more the more experience the learner has had. Section 4.7 demanded that the compound be a compression win rather than merely reachable, and there are three measurements: what the library does to the description length of the whole corpus, what it does to belief programs specifically, and how the belief program compares to the non-mental rivals the same library can express.

5.3.1 The corpus description length

The quantity the compression step minimises (Section 4.5.2) is the joint description length of the library together with the corpus written in it. Fixing the corpus and repricing it under each round’s library gives, for round r ,

$$\text{DL}(X|r) = \sum_{x \in X} \text{DL}(p_x|L_r) + \text{DL}(L_r)$$

where X is the corpus, p_x the program the run found for task x , and L_r the library after round r ’s compression. Figure 9 plots this, split by family.

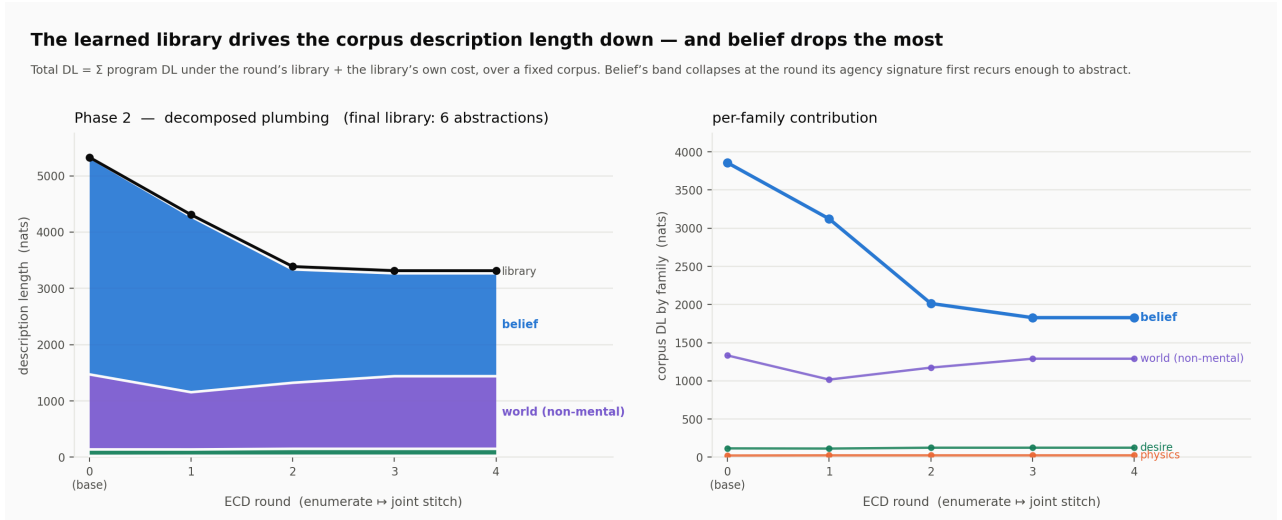


Figure 9: Corpus description length by round (combinator endowment). Left: the stacked total, with the library’s own cost as the black line. Right: the per-family contribution. The belief band collapses at round 2, the round after the one that rebuilt the derive-and-commit frame and the commit from non-mental evidence.

Under the base primitives the 276 solved programs cost 5329 nats; after four rounds they cost 3315, including the 47 nats the library charges for itself. Of the 2014 nats saved, 2029 come from the

belief family alone — 3857 nats down to 1828 — and the largest single step is the 1108 nats that go at round 2, the round whose compression step invents the constructor. The non-mental families barely move: the nineteen non-mental families, pooled into one band in the figure, go from 1333 to 1290 nats, nearly all of that the registration-flavoured families buying the reassembled single-value publication, desire from 116 to 125, physics from 23 to 25 — the last two slightly **worse**, which is the library tax of Section 4.5.2 showing up exactly where it should, since a family that gains no abstraction still pays for the enlarged symbol set.

That asymmetry is the argument. If the initial primitives had encoded anything mental, belief would have been cheap from the start and the belief band would look like the desire band. It is because belief is expensive under the base library and cheap under the learned one that we can say the mental content is in what was learned rather than in what was given. The atomic control gives the same picture a round earlier and from a higher starting point: 5765 nats down to 3516, belief carrying 4404 to 2090, with its largest drop — 1841 nats — at round 1, since there the constructor can be invented as soon as anything has been solved.

5.3.2 What the library buys each family

Per task, the same result in nats. Pricing each family’s found program under the base primitives and again under the final library (Table 11, in the appendix) turns up one substantial mover and a handful of small ones: belief falls from a median 34.51 nats to 17.40, a saving of 17.11 per task and 412 over the priced solves; obstacle from 15.97 to 12.79, through the seek block, the policy the two families share; and registration, perception and drifting registration each save exactly 2.30 nats — the price of one token, the reassembled single-value publication. Every other family saves 0.00. So belief is the only family the library substantially compresses, and the small savers are instructive rather than embarrassing: the same objective that invents the agent constructor also invents a purely non-mental abstraction where purely non-mental reuse exists, and the two shared tokens are exactly the two donor relations Section 5.6 recovers independently. The atomic control gives the same shape with a smaller belief saving, 33.71 down to 13.38 nats, since less of the compound is left to abstract once the pair interface is given.

One caveat about the belief row. To price a solution under the final library, the solution has to be re-expressed in that library’s vocabulary and read back in; for 25 of the 132 belief solves, all of them from the witness family, that round trip succeeds, and the rest are left out of the census rather than priced badly. The 17.11 is therefore a median over a fifth of the family rather than over all of it. Figure 9 is the unrestricted version of the same measurement — it prices every solved program in every round, and gives the belief band’s 3857 to 1828 nats directly.

5.3.3 No non-mental rival reproduces the scenes

The third measurement is the one Section 4.7 cared most about: whether a non-mental program is available that is as short and as accurate. Each belief task carries a battery of hand-written rival spellings — the transient-wall schedule that stamps a wall, lets the agent walk, and erases it before the frame is rendered; the version that shoves the goal in the world rather than in a model; pure desire; a physics reading with no wall at all; and, for the witness family, the four orderings of stamp,

act, witness and erase. Each rival is rendered over every scene of every belief task and compared frame for frame.

Rival spelling	Tasks	Reproduces all 4 scenes	Reproduces the agents	Median margin (library, nats)
transient wall (stamp / act / erase)	24	0	24	+10.67
transient wall (erase-3: takes down every wall, not just the stamped cell)	24	0	24	+9.76
transient shove (shove / act / restore)	51	0	51	+18.19
witness then stamp / act / erase	48	0	0	+13.85
stamp / act / erase then witness	48	0	0	+10.67
stamp, witness, act, erase	48	0	0	+10.67
stamp, act, witness, erase	48	0	0	+10.67
transient phantom + shoved goal	9	0	0	+12.35
shove goal in the world	60	0	0	+4.79
both agents seek the goal	24	0	0	+4.09
transient, witness ignored	48	0	0	+3.58
pure desire (no belief)	27	0	0	−0.69
no wall (physics)	72	0	0	−2.53
no belief (seek past the real wall)	9	0	0	−9.51

Table 2: The non-mental rivals (combinator endowment), over the 132 solved belief tasks. “Margin” is $DL(\text{rival}) - DL(\text{found})$ under the final library; positive means the mental reading is shorter. Not one rival reproduces all four scenes of any task.

The headline of Table 2 is the third column. Across 132 belief tasks and fourteen rival spellings, **no non-mental rival reproduces all four scenes of any task**. This is a change in kind from earlier versions of the corpus, where the transient-wall rival reproduced the false-wall scenes exactly and had to be beaten on description length, so that the honest form of the argument was a margin. Two corpus changes closed it. An inert bystander object now sits on the phantom-wall cell for the whole trajectory, so stamping a real wall there overwrites something the frame shows and nothing in the language can put it back; and the k -scene format (Section 4.2) requires a rival to survive four independent placements of the same latent, which a single wall coordinate chasing a believed goal cannot do. The fourth column records how close the rivals get: the transient-wall spelling still reproduces the **agents’** trajectories on all 24 false-wall tasks, and the transient shove on 51 goal-displacement tasks. They get the walk right and the world wrong.

This is exclusion by reproduction rather than by price, and it is the stronger form of the argument — a rival that cannot render the data is not a rival at all, whatever it costs. It also keeps the argument honest in the other direction: 120 of the 540 rival pricings come out **shorter** than the mental program, almost all of them the bottom three rows, and every one of them is a program that simply does not depict the scene — an agent walking to a goal it is not wrong about. Cheapness without accuracy is worth nothing, and it is the likelihood of Section 2.2 rather than the prior that

disposes of them. But the price is still informative, and the last column is where the compression story reappears. Take the closest rival, the transient wall. Under the base primitives it costs 0.08 nats **less** than the mental program — a tie leaning the rival’s way, as one would expect, since neither reading has any structure to reuse and the mental one names one value more. Under the final library the mental program is 10.67 nats **shorter**, a swing of four orders of magnitude in prior probability. Nothing about the rival changed; what changed is that the mental reading collapsed into a token the library had a reason to invent and a bespoke stamp-act-erase schedule, recurring nowhere else in the corpus, did not. The atomic control gives the same result from a tie leaning the other way, +1.28 nats widening to +10.91, and that the effect survives both carvings of the primitive set is the point of running the control.

5.4 Reachability, and the order in which things arrive

Section 2.4 argued that description length in this setting is simultaneously a measure of plausibility and a budget: a hypothesis that is long under the current library is not merely improbable but unreachable. Section 4.7 turned that into a prediction about ordering. Figure 10 is the prediction met.

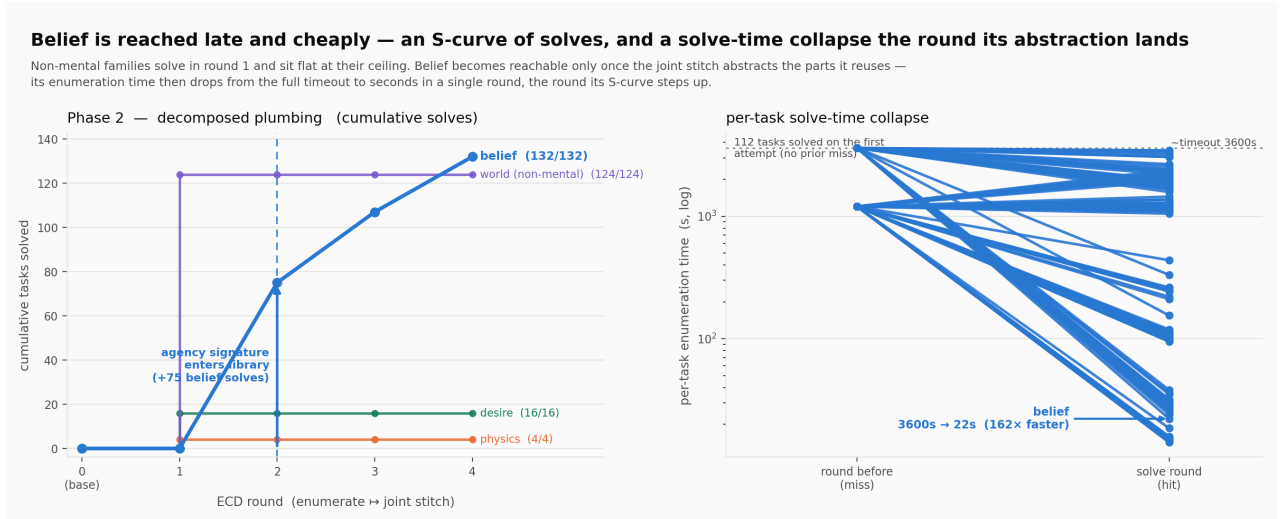


Figure 10: Solve dynamics (combinator endowment). Left: cumulative solves by family, by round. Right: for each belief task solved after at least one miss, its enumeration time in the round it missed against the round it was solved.

The non-mental families are flat lines at their ceiling from round 1: they need no abstraction and get none. Belief is an S-curve — 0, then 75, 107, 132 — whose step is the round after the constructor enters the library. And the first point of that curve is a zero rather than a small number: with the pair interface given only in parts, not one belief task is solved until the compression step has reassembled the derive-and-commit frame and the commit out of non-mental evidence. Belief here is not merely slow in round 1; it is out of reach.

The right-hand panel is the same fact per task, and it is more equivocal. All 132 belief solves were missed at least once, and the worst case goes from the full 3600 s timeout to 22.2 s, a factor of 162 — but the **median** moves only from a 1200 s round-1 timeout to a 1146 s solve. What the library buys here is therefore reachability rather than speed: it brings the compound inside the budget window at all, and most tasks then spend most of that window. The atomic control moves

the median further — of 100 tasks missed at least once, from a 1200 s timeout to a 708 s solve, with the steepest collapse from 1200 s to 2.0 s — which is what one expects of a run that starts a rung higher; but there too the typical task spends more than half its budget. In neither case did the tasks or the budget change; what changed is the description length of their solutions, and with it the position of those solutions in the cost-ordered walk.

This does not show that a child’s four-year delay has this cause. What it shows is that in a learner of this kind the delay does not have to be put in. Section 3.4’s complaint against BToM was that its structure is complete from the start, so every parameter is available for inference on day one and nothing in the framework makes goal attribution earlier or cheaper than belief attribution; the ordering has to be imported from outside as maturation or as a performance limitation. Here the ordering is the mechanism. Desire is solved in round 1 because “at each time step 1 moves to the adjacent cell closest to 2” is a three-symbol program, (`optimize (neg_dist 2) 1`); belief is not solved until round 2 because its compound is out of budget until earlier learning has shortened it. That is the cumulative, staged picture Section 1.3 demanded, produced rather than accommodated: the bootstrapping of Section 2.5, measured rather than assumed.

5.5 Generality and selectivity

An abstraction that is a compression win might still be a bad theory in two ways: it might have baked in the particular values it was trained on, and it might be applied to everything that moves — the second being Section 4.7’s fifth refutation condition, a habit of opening a private copy for whatever moves rather than an attribution. Three measurements answer them.

5.5.1 The constructor spans the corpus

The corpus rotates through eight distinct (goal value, agent value) pairs so that no integer is privileged (Section 4.6). Figure 11 counts, for each combination and each invented token, how many solved belief programs are rewritten through it.

One discovered abstraction is the agent — it tiles every (gv, av) belief task

Rows are the (goal-value, agent-value) combinations the curriculum poses; columns are the invented library abstractions. A cell counts solved belief programs whose rewritten form uses that abstraction — the agent-constructor column is filled for every combination.

Phase 2 — decomposed plumbing (agent constructor: fn_9)

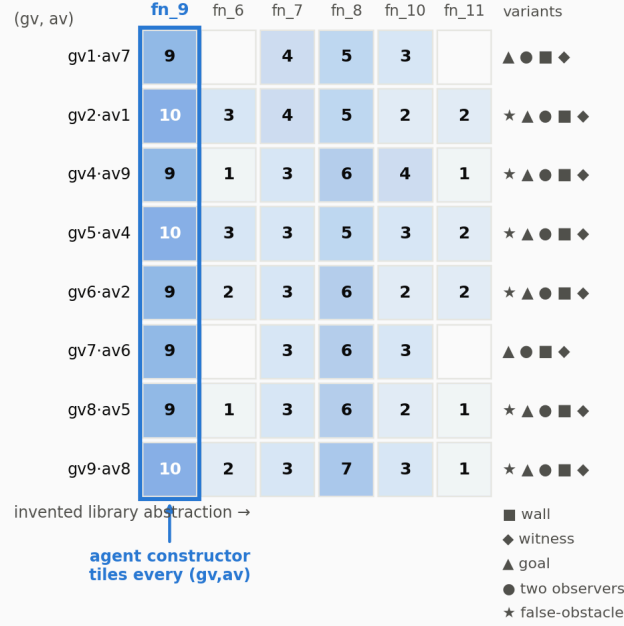


Figure 11: Constructor coverage (combinator endowment). Rows are the eight (goal, agent) value combinations; columns are the six invented abstractions. Four of the six columns are occupied in every row; the two that are not are the two rarest spellings.

Every abstraction that opens a private copy spans the matrix. The most-used of them, the phantom-wall constructor, carries 75 of the 132 belief solves and appears in every row; the two-actor constructor carries 26 and appears in every row; the seek block (46) and the displaced-goal constructor (22) likewise. Only the two rarest miss anything: the open constructor (12 solves) and the agent commit (9) are each absent from two rows. So what was learned is not a memorised (goal, agent) pair with a wrapper around it — the cell-value slots really are holes, and they are exercised across every pair the corpus rotates through. The instructive comparison is the obstacle family, whose policy did **not** generalise into a single token in this run: because obstacle scenes vary their wall coordinate more than their (goal, agent) pair, no single wall-bearing policy recurred often enough to be abstracted with holes, so belief’s derive is not a cheap token even after compression — the belief compound is not shallow even with the constructor in hand. The atomic control gives the same picture slightly less evenly: its most-used constructor spans all eight combinations, by 52 of 155 solves, but its seek block is missing from two rows.

5.5.2 The symmetric complements go elsewhere

Table 13 lists, beside each primitive belief uses, the opposite corner of an independent symmetry axis: channel direction, commit scope, z-order, projection, pairing, the bifunctor axis, and utility polarity. The point of including them was that an unused complement proves nothing. Table 3 reports which family claimed which corner.

Corner	Family that selected it
publish one named value: read where it sits in the model, write it into the world	belief (81 of 132, of which 9 spell it literally — Section 5.5.3), registration, perception, drifting registration
publish every shared value	multi-registration, map-update
publish every shared value but one	registration-except
copy the grid, and alter the copy rather than the original	belief (81), overlay, underlay, comet
alter the original rather than the copy	drifting registration
exchange the two grids, so that the model becomes the world and the world the model	perception, map-update
pair the grid with a blank one, so the second channel starts as scratch rather than as a copy	wipe
union the two grids cell by cell, in either order of precedence	overlay, comet / underlay, inpainting
discard the world and keep the model	readout, wipe
move toward, or move away from, the value the utility names	belief, desire, obstacle / flee
stamp a wall / clear a cell / scrub a value from the grid	belief, obstacle, relocation / deletion, relocation / denoise
alter both grids at once, with a separate edit for each	no family — see below

Table 3: The complement census (combinator endowment). Belief selects the one asymmetric corner; every complement any family claims is claimed by a non-mental one. The atomic control gives the same census with the single-value publication granted as a token rather than assembled from a reader, a writer and the operation joining them.

Every complement the census tracks — altering the world instead of the copy, publishing wholesale instead of one value, exchanging the two grids, starting the second channel blank, unioning them the other way round, keeping the model and discarding the world, fleeing instead of seeking, removing rather than stamping — is claimed by some non-mental family, and belief claims none of them. Availability is not use: the search had all of them in hand at the same prior cost, and it took the corner that corresponds to acting on the world through a private model of it. The one row without a claimant is the corner that acts on both factors at once, and its emptiness is measured rather than mysterious: the layer-compositing family written to claim it — two channels, one edit each, a task a paint program performs — went 0 of 4, and the miss is a budget artefact, its intended program sitting just past the point the enumeration over template tasks reached before its time ran out (Section 5.8). A corner the search could have taken and did not is what makes taking the belief corner a choice, and eleven of the twelve corners are now also independently **exercised**; the twelfth awaits a run with the budget to reach it.

5.5.3 Attribution is selective

The two-observer family puts two agents in one scene with one goal and one grid, where only one of them is wrong about anything and the other’s walk is plain desire. If what the learner acquired were a habit of opening private copies, it would open one for both agents. In the main run, 24 of 24 solved scenes commit for the believer and not for the bystander; in the atomic control, 24 of 24. There is no over-attribution anywhere in the 48 solved two-observer scenes.

This is also the place to explain a number that looks alarming and is not. Of the 132 belief solutions in the main run, 9 publish the agent’s position and nothing else, and 123 publish wholesale — every shared value, or every shared value but the goal. Counted as symbols, that looks like belief giving up the one corner the argument says it takes. Read as a computation, it is not: on a scene whose model diverges from the world only at the misrepresented content and the movers, “move everything to its model position” and “move everything but the goal” both come to “move the agent” — and we check that they do, rather than assuming it: for each of the 123, the commit is replaced by the single-value one and all four scenes are re-rendered. Every one of the 123 reproduces the scenes unchanged, and the falsifying case — a wholesale commit that carries some world-edit no attributed model explains — does not occur in either run.

Why the searcher prefers the scope spelling is not a mystery either; it is minimum description length doing exactly what this thesis says it does, one level down. Here publishing one named value has to be assembled out of three symbols — a reader, a writer and the operation that joins them — while publishing everything but one named value is a single symbol, because the wholesale commits range over a value set the scene fixes rather than the program, and Section 4.4.2 gave the reason they were left atomic: the decomposition reaches exactly as far as the single-value vocabulary can spell, and no further. Same computation, fewer symbols, so compression buys the wholesale spelling 123 times out of 132. In the atomic control, where the two cost the same, it buys the single-value spelling 155 times out of 155. The **spelling** of the commit tracks the price list; what the commit **does** stays fixed. And note which way that cuts: the pricing handicaps the agency spelling rather than its rivals, so this asymmetry runs against the result, not for it. The internal control is the false-obstacle family, built so that no wholesale commit can reproduce its scenes: a real wall and a stationary goal sit alongside the phantom one, so a wholesale commit would move them. Every solved false-obstacle task in either run — 16 of 24 under the atomic control, 9 of 24 in the main run — commits through the literal single-value spelling. The spelling switches exactly where the two stop computing the same thing — which is not how a classifier reading strings would behave. Across both runs there are **zero** non-mental commits: every belief solution in either run publishes exactly what some model sought.

The coverage all of this is computed over is the weakest number in the chapter. Three families are complete — false-wall 24 of 24, witness 48 of 48, two-observer 24 of 24 — but goal-displacement is 27 of 48 and the false-obstacle family, the deepest compound in the corpus, naming a real wall, a phantom wall, a displaced goal and an agent, is 9 of 24. The atomic control reaches 155 of 168 on the same corpus: complete on the first three, 43 of 48 on goal-displacement and 16 of 24 on false-obstacle. Since its vocabulary expresses nothing this one does not, the 23 tasks in the gap are tasks

this run ran out of budget on rather than tasks it could not state — Section 5.10 is careful about how much that argument is worth.

5.6 Scaffolding

Section 1.2 records that theory of mind arrives in a fixed order, and Section 1.4.3’s account of why is that later concepts are built out of earlier ones. Nothing in the corpus asserts such an order: a task carries only its own kind, and no family is marked as scaffolding for another. Whether the order is there is therefore a question to be put to the runs, and it can be put in a way that does not presuppose the answer. Take each solved belief program, abstract its concrete values and coordinates to typed holes, and ask which **other** families’ solved programs occur inside it as subterms.

Donor family	Belief tasks it feeds	The fragment, and the role it plays
desire	132	move the agent to the adjacent cell closest to the goal — the action policy
obstacle	84	stamp a wall, then move the agent toward the goal — what is done to the private copy
registration-except	51	publish every shared value but one — the commit, in its wholesale spelling
false wall	51	a phantom-wall constructor, feeding the witness family
physics	48	move a value one cell up — the displacement applied to the private copy in a displaced-goal program
goal displacement	26	a displaced-goal constructor
registration	9	publish one named value — the agency commit
witness	3	a composed pair of agent blocks
flee	2	move the agent to the adjacent cell <i>furthest</i> from the goal — the same policy at the opposite utility polarity
two-observer	2	a seek composed with an agent block

Table 4: Structural scaffolding recovered from the main run: which shallower solved programs occur as subterms of belief solutions. Nothing in the corpus labels any of these a scaffold; the relation is read off the programs.

Every one of the 132 belief solutions contains a desire solution. That is the strongest and least surprising row: belief’s policy **is** the desire family’s program, “move the agent to the adjacent cell closest to the goal”, and that family has nothing mental about it — it is one object approaching another. The commit is the same story split across two spellings: 9 solutions publish one named value, as the registration family does, and 51 publish everything but one, as the registration-except family does, which on these scenes comes to the same thing (Section 5.5.3). The obstacle family supplies what is done to the private copy in 84. And the belief families feed each other in the order the corpus’s difficulty predicts: a false-wall constructor sits inside 51 witness solutions, a witness task being a false-wall task with a second agent, granted no private copy of its own, walking across

the phantom cell. The tool prints the same fact as an assembly, splitting a solution into its maximal fragments and naming the family that certifies each. On one witness solution it returns two. The first — “at each time step the value 5 moves to the adjacent cell closest to 4” — is certified by **desire**, and is the action policy of the agent who is not misled. The second — “on a private copy of the grid with an impassable wall stamped at row 3, column 1, the value 1 moves to the adjacent cell closest to 2, and then every shared value is moved to where the copy has it” — is certified by **false wall**, and is the agent block proper. The witness solution is the two run in that order. Nothing told the tool that either family stood to the other as a rung.

The second mechanism is compression rather than containment, and it is the one the objective actually paid for: when a belief program and a non-belief program are rewritten through the **same** invented token, that token is shared reuse. Here there are exactly two such abstractions, and they are exactly the two donated parts: the seek block, shared between belief and the obstacle family, and the agent commit, shared between belief and the registration, perception and drifting-registration families — alongside the two round-1 abstractions that first rebuilt the derive-and-commit frame and the single-value publication out of the overlay and registration families. The atomic control has only the seek block, since it was handed the pair interface and had nothing to rebuild. So the sharing is real but narrow, and it is narrow in the right place: what the non-mental families donate is the **parts** — the policy and the commit — and every abstraction that opens a private copy is belief’s alone. What is belief-specific is the **wiring**. An abstraction shared wholesale with a non-mental family would not be an agent constructor.

Two caveats. This is scaffolding in the sense of compositional dependence recovered from a converged run, not a longitudinal claim about a learner acquiring one rung and then the next; the ordering evidence for that is Section 5.4. And the curriculum is ours: the obstacle and relocation families are deliberately over-weighted precisely so that the wall-and-navigate policy would recur often enough to be abstracted. That is a modelling choice about what the learner sees, not a thumb on the scale of what it concludes, but it is a choice, and Section 5.10 returns to it.

5.7 The behavioural probe

Everything so far is description length. Section 4.7’s third condition was behavioural: on a scene held out of the run entirely, does the learned term predict that the agent goes where it **believes** the goal to be?

The probe uses the goal-displacement family (Figure 5), because it is the one where the two readings come apart in behaviour. In a false-wall scene the transient-wall rival walks the agent along the same path, so only description length separates the readings there — that was Section 5.3.3. In a goal-displacement scene the agent walks to where it believes the goal is, one cell from the true goal, which never moves, so a program that does not represent the belief cannot land on the right cell.

The scenes are drawn from a seed that never entered the run and are checked to be absent from the training corpus, and the library is rebuilt by running the compression step again over the run’s own found programs, so the terms priced here are the terms the run actually converged to. We then instantiate the discovered belief compound on the novel scene and, as its foil, the shortest program the same library can express without it.

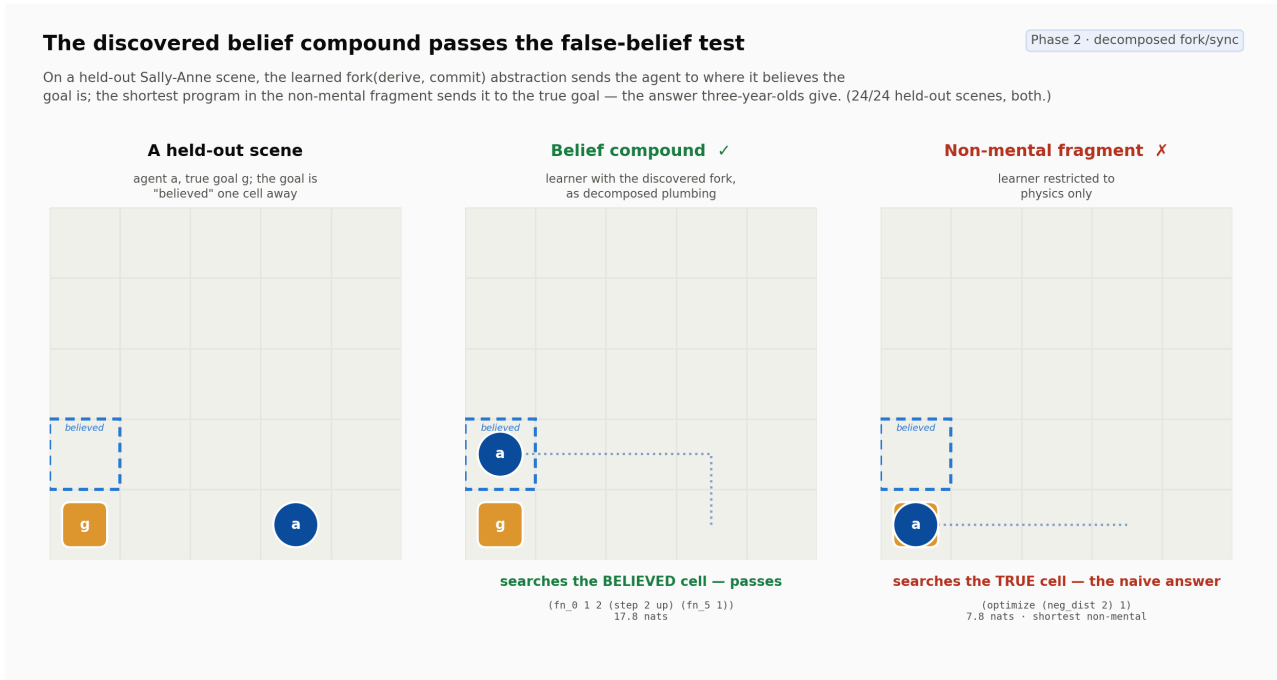


Figure 12: The false-belief test on a held-out Sally-Anne scene (combinator endowment). The learned belief compound sends the agent to the believed cell; the shortest non-mental program under the same library sends it to the true goal.

Written in the learned library, the belief compound is the open constructor supplied with the agent value 1, the goal value 2, an upward displacement of the goal, and a publication naming the agent. What it encodes is

at each time step, in a private copy of the grid in which the value 2 has been moved one cell up, the value 1 moves to the adjacent cell closest to 2; then, in the world, 1 moves to wherever the copy left it and nothing else changes

— the discovered constructor, privately displacing the goal and publishing only the agent’s own move, naming the agent outright. Its final frame puts the agent on the believed cell, one step short of the true goal. The learner restricted to the non-mental fragment writes the shortest thing that fragment can say, plain desire, “at each time step, 1 moves to the adjacent cell closest to 2”, and walks straight to where the goal really is.

The two programs stand in the sharpest possible relation: the belief program **is** the desire program, run on a private copy in which the goal has been moved, and published for one value only. Across all 24 held-out scenes the belief compound lands on the believed cell every time; the non-mental program lands on the true goal cell in 18 of 24 and, in the remaining six, one step short of it on the true-goal side, walking toward the real goal and running out of frames. It never once lands on the believed cell. The same probe against the atomic control’s library gives the same 24 of 24, as it must: the two endowments differ in how the compound is **spelled**, not in what it computes.

The nats are the point of the exercise. Here the non-mental program is **shorter** — 7.8 against 17.8, and 7.9 against 13.4 under the atomic control — which is the opposite of Section 5.3.3, and that is why both measurements are needed. Where a non-mental rival reproduces the scene, it does

so only at greater length, so description length rules it out; where a non-mental rival is cheaper, it is cheaper only because it predicts the wrong behaviour, so the data rule it out. There is no program in the non-mental fragment that is both as short as the belief compound and as accurate. A learner that has compressed its experience into that compound passes the false-belief task; a learner confined to the fragment fails it, and fails it in the specific way the developmental literature documents — by answering with the true location (Wimmer & Perner, n.d.).

5.8 The verdict

Section 4.7 fixed in advance what would count as success and what would refute the hypothesis. Table 5 is the tally.

Criterion (Section 4.7)	Combinator	Atomic
An abstraction enters the library that opens a private channel, derives a divergent state, evaluates a policy against it, and commits only the action, with one value shared between derive, policy and commit	Met, round 2: the two-actor constructor, one hole bound four ways	Met, round 1: its constructor, one hole bound twice
Selected by the joint objective over the non-mental rivals the same library expresses	Met: +10.67 nats over the nearest rival under the library, from -0.08 under base primitives	Met: +10.91 nats, from +1.28
No rival that never opens a private channel reproduces all k scenes of a mental family	Met: 0 of 14 rival spellings over 132 tasks	Met: 0 of 14 over 155 tasks
The term generalises behaviourally on a held-out scene	Met: 24/24 to the believed cell	Met: 24/24
The term is applied selectively, not to every mover	Met: 24/24	Met: 24/24
Belief coverage	132/168 (79%)	155/168 (92%)

Table 5: The criteria of Section 4.7 against the runs.

Four things need saying here rather than in a footnote, the first of them for the first time in this corpus’s history to the good.

The main run passes every check it was set, as computed. The criteria above are applied by the run itself rather than by us afterwards — that the donated parts are used generally, that the abstractions opening a private copy are belief’s alone, that no belief solution commits a value no model sought, that the constructor was invented and is belief-specific, and that the complements are inhabited elsewhere — and all of them come out true, with the commit-substitution test of

Section 5.5.3 performed during the run rather than by reclassifying its output afterwards. Earlier revisions of this corpus could not say that, and the difference is not a relaxed check but a repaired corpus: the bystander and k -scene changes of Section 5.3.3 are what removed the leaks the old verdicts caught.

The atomic control fails one check: the requirement that **every** solved task in the two families whose agent is wrong about an obstacle — plain false-wall and false-obstacle — be solved by a program whose private grid misplaces that obstacle. The plain false-wall family passes 24 of 24, so the failures are in the false-obstacle family, where a handful of tasks were solved by a program representing the agent as wrong about the **goal** rather than about the wall: a displaced-goal model with one private copy nested inside another, which reproduces the scenes and is a belief attribution, but not the one the family was built to force. This is a mis-specified check rather than a non-mental rival — the criterion asks about the wall because that is what the family names, but the family’s scenes admit a second mental explanation the check does not recognise.

One non-mental family went unsolved. The layer-compositing family — the four tasks written to claim the both-factors corner of Table 3, the one without a claimant — was missed in every round. The cause is measured, not conjectured: its intended program sits at roughly the 40,000th position in the cost-ordered enumeration for template tasks, and the 60 s each round allowed for that enumeration reached about the 30,000th on the hardware the run used; on faster hardware the same enumeration solves all four in under 40 s, with the intended program. It is a budget artefact of exactly the kind Section 5.4 says description length creates, and the fix — a larger template budget — is a rerun, not a redesign.

Coverage is incomplete, and unevenly so. The main run misses 21 of 48 goal-displacement tasks and 15 of 24 false-obstacle tasks. The case that these are budget misses rather than expressive ones is the atomic control: its solve set strictly **contains** the main run’s — 23 belief tasks solved there and not here, none the other way — in a vocabulary that expresses nothing this one cannot, and the main run’s solve times (Figure 10) have a large mass pressed against the 3600 s ceiling. That is an argument from a shorter spelling of the same programs, not from a longer run, and a longer run is what would settle it; this thesis does not have one. The contradictory-belief family, removed from the corpus for the reasons Section 4.7 gives, is the version of this that could not be papered over at all.

Against that, what did come out clean is the load-bearing part. In both runs the compression step invented an abstraction with the shared-hole signature; in both the belief families used it and no non-mental family did; in both the complement census found belief on the one asymmetric corner, with every complement any family reached claimed by a non-mental one; and in both, not one belief solution committed a value that no model had sought.

5.9 Objections

5.9.1 “fork and sync_to_world are just believe cut in half”

The sharpest objection is that the atomic primitive set of the control run already contains the answer. By the definition this thesis took from the literature, mentalizing is attributing to an agent an intensional state — a representation of the world that may diverge from it. **fork** builds a divergent

representation and `sync_to_world` reads an agent’s position off it. Putting them together is not discovery; it is reassembly of something that was handed over in two pieces.

Three replies, each of them a measurement already reported. The parts have non-mental uses and take them: `fork` is what the overlay, underlay and comet families’ **ground-truth** programs are made of, `sync_to_world` what image registration’s are, and a primitive that only ever appeared inside belief compounds would be a **believe** in disguise where one image registration uses is a general-purpose tool. The field is symmetric and the complements are live: Table 3 finds all of them but one inhabited by other families in the same run at the same prior cost, with belief taking none — and it is one thing to observe that a learner did not go the other way, another to observe that other tasks in the same run **did**. And the objection is in any case to a variant the thesis does not rest on: remove the two primitives, as the main run’s library does, and the structure comes back anyway, reassembled in round 1 out of non-mental evidence, before a single belief task has been solved. If `fork` were **believe** in disguise, that run would have had to discover **believe** from scratch, and it did.

Two residual forms of the objection turn on nothing the runs measured, and the no-separation point of Section 4.6 answers them: that the pair is itself the representational capacity, when building a divergent world state is counterfactual reasoning and attributing it to an agent is the conjunct the shared hole adds; and that copying a grid could in turn be decomposed into allocation, reads and writes, which relocates the regress rather than ending it.

5.9.2 “The corpus was built to produce this result”

The last objection is about the curriculum rather than the language, and it has the most substance. The obstacle family is deliberately the densest corner of the corpus, at six tasks per (goal, agent) combination, because the wall-and-navigate policy has to recur often enough for compression to abstract it; belief’s derive is short only because that abstraction exists. Change the mix and the result moves.

But this is a claim about what the learner is **shown**, not about what it concludes, and it is exactly the claim Section 1.4.3 makes on the theory theory’s behalf: a concept becomes available when earlier learning has made it cheap, so the composition of earlier experience is a determinant of what is learnable and when. A model in which the curriculum did not matter would be one in which the ordering result of Section 5.4 was vacuous. Nor is the corpus stratified in the way the objection needs: it is one undifferentiated bag with no mental / non-mental label anywhere in the search or the compression step, and the compression step of Section 4.5.2 pools the solutions of both task formats and revises a single library over all of them (Section 4.6.5), so the budget that paid for belief’s structure was free to be spent on overlay, registration, obstacle or relocation instead. What is missing is the dose-response experiment that would quantify the mix — vary the non-mental fraction, measure whether the constructor is still invented. It is the single most informative experiment left undone, and Section 5.10 says so.

5.10 What the results do not show

Seven limits, the costliest first.

The belief has no history. Section 4.6.1 states the concession and the results do not soften it: nothing in the discovered term represents how the agent came to be wrong, and nothing updates

when it looks again. What should be added is that the comparison to BToM runs the wrong way. BToM’s belief is at least a state that could in principle be revised, and this one is not, so the gap is not the symmetrical one that section’s parallel might suggest. Section 6.3 takes it up.

The curriculum mix is untested. As Section 5.9 concedes, the corpus-mix dose-response has not been run. Nor have the two others that would sharpen the result: compressing the belief solutions on their own rather than pooled with everything else, which would show whether the constructor survives being priced against belief alone, and a seed-robustness sweep reporting every verdict metric across five corpus seeds. Each of the two runs reported here is a single seed.

Omitting the recognition model was decided on the evidence, not in advance. Section 7.10.2 gives a good reason for leaving DreamCoder’s third phase out, but it is not the order in which the work happened: the loop was run both ways first, and the numbers settled it. The measurement is from the July runs, on the 300-task corpus that predates the pair-plumbing families of Section 4.2; the ablation was not repeated on the final corpus. Under the atomic control the dreamed proposal was a small win — 291 tasks and 159 belief tasks against 287 and 155 — concentrated in the two hardest families; under the combinator library it was a substantial loss, 240 against 264, with the damage in the witness family, which fell from 48 of 48 to 26 of 48. The mechanism is not the obvious one. The dreamed prior was gated to families with at least one solved instance, so it cannot be what delayed the first belief solve, and both runs under the combinator library made theirs in round 2. What differed was the **library**: the two runs solved slightly different sets of tasks in round 1, so the compression step saw a different corpus in each, and the run with the proposal distribution converged on a library through which the witness compound is longer. The deficit was transmitted by the compression step rather than by the proposal distribution — the two interact, since a proposal that changes **which** tasks are solved changes what the next library can be. Characterising that interaction is left undone. What can be said is that the constructor was invented in the same round, with the same shape, in all four runs, so the structural result does not turn on the choice either way.

Coverage is not complete, and the misses are not proven to be budget misses. Section 5.8 gives the numbers. The claim that a longer run would close them is supported by the comparison between the two endowments and by the shape of Figure 10, and it is not supported by a run. The experiment that would settle it is a budget dose-response — the same corpus at 600, 1200 and 2400 s per task, with the solve rate per belief family plotted against the budget — which would separate a family that is out of **reach** from one that is merely out of **time**.

The one-model frame is fixed by the type system, not discovered. Both endowments give the learner the product $G \times G$, which says that a learner holding a private representation holds exactly one — and a one-model frame is precisely the kind of thing a theory of mind is supposed to have. Nothing reported here shows that compression would **select** a single private channel if more were expressible, because in these runs no more are. The experiment that would settle it replaces the fixed pair with a recursive grid-stack, so that channel arity becomes a free parameter, and measures which arity each family selects; Section 6.2 sets it out as further work.

The planner is a single token. Section 4.6.3 defends granting a planner but not granting it in this form; “move to the adjacent cell closest to the goal” packs a full breadth-first search into

one symbol, and a more elementary treatment would decompose it into memory, conditionals and interaction (Section 6.4).

None of this is a claim about children. The model is a sufficiency result at the computational level: it shows that a learner with domain-general representational resources and a preference for short programs **can** arrive at belief-structured abstractions, which blocks the inference from “no mechanism has been named” to “no mechanism exists”. Whether human learners use this mechanism, or another one that solves the same problem, is not settled by anything in this chapter.

Chapter 6

Further work

6.1 Polymorphism

We use a simply-typed monomorphic type system. For our purposes, types are really used only to prune enumeration. They constrain which compositions are legal, not what the program must prove. DreamCoder uses polymorphic types, so unification can rule out ill-typed branches early. But since our DSL has only ~ 30 primitives our bottleneck isn't branching factor, so monomorphic dict lookup is sufficient.

With monomorphic types, a stitch abstraction discovered at type `grid` can only be reused at `grid`. With polymorphism, an abstraction abstracted over $\forall a$ can be reused across instantiations of any type a . The grid-stack of Section 6.2 would have to hand-engineer this, with a `cons/nil` constructor and a depth-polymorphic lift written out for each pair operation; if we had polymorphism it would just fall out of the type discipline.

To do something like that, we'd replace bare string types with a small ADT:

- `ground ('con', 'grid', [])`
- `variable ('var', 'a')`
- `applied ('con', 'pair', [t1, t2])`
- `arrows (args, ret)`

We'd implement Robinson unification

6.2 Channel arity as a free parameter

The product $G \times G$ of Types carries a commitment we should not be making on the learner's behalf. It says that a learner holding a private representation holds *exactly one*, and a one-model frame is precisely the kind of thing a theory of mind is supposed to have. If the answer to “why one world and one model, and not three?” is “because the type system says so”, then part of the structure at issue has been fixed by the modeller rather than found by the learner. Section 5.10 lists this as a limit of the results; this section says what removing it would take.

The design is a further weakening of the endowment of Section 4.4 — strictly less given, on the same corpus, with the same objective. The fixed pair is replaced by a recursive grid-stack,

```
gstack ::= () | (grid, *gstack)
```

with `nil` and `cons` as constructors, and every pair operation is replaced by a depth-polymorphic lift of itself that acts on the top of the stack. The type system of Types gains the arrows $G \rightarrow G^*$, $G^* \rightarrow G^*$ and $G^* \rightarrow G$ to push, transform and project, and loses the arrows into and out of $G \times G$ that fixed the arity at two. At depth 1 the lifts reproduce `fork` and `sync_to_world` exactly, so nothing

expressible under either endowment of Section 4.4 is lost; but the number of private channels a program opens is now something search selects rather than something the type system fixes.

The same monomorphism is also why the language needs a separate composer at each arrow of Types. The pieces belief is assembled from — copying a grid, altering one channel of a pair, and composing two functions — are defined at any objects, not just at G , so the four composers of Section 4.4 are four copies of one operation the type system cannot state once. Nothing in the argument turns on this, but a language with polymorphic plumbing would collapse them into a single symbol, and the wiring belief needs would be that much less of the endowment.

The measurement is then per family: for each solved task, read off the maximum stack depth its program reaches, and compare that against the depth the family's ground-truth program uses. The corpus would need at least one non-mental family whose ground truth genuinely requires depth 2 — a cross-channel blur, say — so that arity 2 is known to be both expressible and selectable, and the physics and desire families should sit at arity 0, holding no private channel at all. The prediction the thesis's argument makes is that no family, mental or not, selects more than one private channel except the one whose ground truth demands two; and the quantity that would carry it is not the observation but the margin, the description length of the arity-1 encoding of a belief scene against the arity-2 encoding of the same scene, under base primitives and under the learned library.

If that came out, belief's single-model frame would be a discovery rather than a stipulation, and the last structural commitment in Types would be gone. If it did not — if compression started stacking channels wherever the grammar allowed it — what the runs of Chapter 5 found would still be a belief abstraction, but the claim that the learner also found its *frame* would have to be withdrawn.

6.3 persistent models and interaction

currently an attributed grid state, a belief, exists only for that one frame, since we're just working with grid to grid transition functions. so do something where the agent actually has a persistent memory of its modified grid

and something where the agent can actually react to its environment, can write to the private model and read from it, can navigate based on obstacles, etc

6.4 breaking down optimize

this primitive packs in a lot, it's a full bfs search. so try to rediscover that. this requires a persistent model (to keep track of visited cells) and a and interactions with the environemtn, conditionals (if wall, move away)

6.5 probabilistic primitives

an agent that sometimes does this and sometimes that, etc

Go from a LoT to a PLoT

6.6 no sequencing

6.7 vs. real world

obviously this isn't how an infant learns theory of mind

I'm missing the *self*. I'm focusing on learning to distinguish between different entities in the domain, but not an infant learning to distinguish others from itself. For example, if I bite myself it hurts, but if I bite someone else it doesn't. maybe pain occurs only if I bite this but not that (extensional). or we have different states, different experiences, and sometimes someone else can be experiencing pain even when I'm not.

I'm missing the scale. I'm focusing on a narrow timescale. in real infants this would unfold over a long time scale.

6.8 richer belief content

6.9 recursive mindreading

6.10 POMDP

observation function

Chapter 7.

Appendix

This appendix collects the implementation of the model outlined in Chapter 3.

- full list of tasks
- full list of primitives

7.1. Full Corpus

Family	Ground-truth program	What it probes
physics	step v d	the bare interpreter, no private model
desire	optimize (neg_dist gv) av	goal-directed motion without a model
obstacle	compose (wall_at r c) (optimize ...)	a detour round a <i>real</i> wall
flee	optimize (distance hv) av	the repulsion corner of the utility axis
deletion / denoise	clear_at r c / erase v	the grid-edit removal corners
relocation	compose (clear_at ...) (wall_at ...)	the grid-edit “move” corner
overlay / underlay	fork (step v d) overlay	fork with a <i>non-mental</i> commit; z-order
comet	fork (optimize ...) overlay	fork’s derive slot taking a seek policy
wipe	pipe_gpg pair_blank snd	the pairing complement: a fresh blank channel, not a copy
belief_wall (false-wall)	fork (compose (wall_at r c) (optimize ...)) (sync_to_world av)	detour round an <i>invisible</i> wall
belief_witness (witness)	the above, composed with a second bare seeker	a witness who crosses the phantom wall
belief_goal (goal-displacement)	fork (seq (step gv d) (optimize ...)) (sync_to_world av)	Sally-Anne: belief whose <i>content</i> varies
belief_observers (two-observer)	a bare seeker composed with a forked one	selective attribution: one agent forks, one does not
belief_false_obstacle (false-obstacle)	fork (seq (wall_at Wb) (step gv d) (optimize ...)) (sync_to_world av)	wrong about obstacle <i>and</i> goal, with a real wall present
registration / perception	sync_to_world v / sync_to_model v	the sync commits doing <i>non-mental</i> alignment
multi_registration / registration_except	sync_all / sync_except v	the scope complements
inpainting / readout	underlay / snd	the z-order and projection complements
composite (layer-compositing)	compose_pg (bimap (step u d) (erase w)) overlay	the full bifunctor: two layers, one map each
drift_reg (drifting registration)	compose_pg (mapfst (step u d)) (register ...)	the world-channel map: register while the image scrolls
map_update	compose_pg swap sync_all	the swap: the model adopts the world wholesale

Table 6: The task families, by the program behind them. The first block are trajectory tasks, solved by transition functions; the last are template tasks, solved by commits (roots). Programs are written throughout in the atomic control’s spelling: `fork` and `sync_to_world` are not primitives of the combinator library but compounds it must assemble. The last three families exist only under the combinator library; nothing in the atomic control maps both channels, maps the world channel

7.2. The BToM correspondence

Section 5.2 reads the invented term against the stipulations of Section 3.4. The same correspondence, item by item.

BToM (Chapter 3)	In the discovered term	Status here
A planner mapping (utility, environment model) to action	Move one named value to the adjacent cell closest to another, in whatever grid it is handed — that is, optimize (neg_dist <i>g</i>) <i>a</i>	Granted as a primitive (Section 4.6.3)
The environment model m' supplied as an <i>input</i> to the planner	The second channel of the pair, the only one the derive writes	Discovered: mapsnd , not mapfst ; the world channel is untouched
$m \neq m'$ permitted — the divergence that makes false belief expressible	The derive is an arbitrary fn applied to the private copy only	Discovered: which channel is transformed is a search choice with an exercised opposite
The goal g , ranging over a given space under a given prior	Whichever cellvalue lands in neg_dist 's slot, priced like every other symbol	Not a space: no goal type, no goal prior (Section 4.6)
The attribution: m' is <i>this agent's</i> model, on whose behalf the planner runs	The shared hole — planner, derive and commit boundary bound to one value: the agent in the atomic constructor, the believed content in the combinator one	Discovered: the conjunct the compression step kept, from either side of the boundary
The nested evaluation: the observer runs the planner inside itself	the private copy lives for one call and is discarded, never rendered	Structural, in program space; the interpreter has no such mode (Section 7.5)
Inverse planning: infer (g, m) from the trajectory by Bayes	Cheapest-first enumeration under the library prior, indicator likelihood	The inference of Section 2.2, unchanged and not agent-directed
Belief as a state that is <i>formed</i> and could be <i>revised</i>	Nothing: the model exists for one transition and has no history	Absent — the honest gap (Section 5.10)

Table 7: What BToM stipulates, and where it turns up in the term the compression step invented.

7.3. The libraries, verbatim

Table 1 says in English what each of the combinator run's final abstractions does, under names of our own. This is the same library as the compression step emitted it: the run's names, the types, the argument orders, and the bodies expanded to base primitives, for a reader who wants to count symbols rather than read descriptions — the two views differ in exactly the way Section 5.5.3 turns

on. The arguments a term takes are written $\$0$, $\$1$, ... in the body, and are supplied in that order, which is the searcher's and carries no meaning; Table 1's letters are assigned by role instead.

Symbol	Type, in argument order	Body
fn_6	$V \times V \times (G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd (compose \$2 (optimize (neg_dist \$1) \$0)))) \$3)
fn_7	$V \times D \times V \times V \rightarrow (G \rightarrow G)$	(compose (optimize (neg_dist \$0) \$3) (pipe_gpg (compose_gp dup (mapsnd (compose (step \$0 \$1) (optimize (neg_dist \$0) \$2)))) (sync_except \$0)))
fn_8	$V \times V \times (G \rightarrow G) \rightarrow (G \rightarrow G)$	(compose \$2 (optimize (neg_dist \$1) \$0))
fn_9	$C \times C \times V \times V \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd (compose (wall_at \$1 \$0) (optimize (neg_dist \$2) \$3)))) sync_all)
fn_10	$V \times D \times V \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd (compose (step \$0 \$1) (optimize (neg_dist \$0) \$2)))) (sync_except \$0))
fn_11	$V \rightarrow (G \times G \rightarrow G)$	(register (locate \$0) (place \$0))

Table 8: The final library under the combinator endowment (run p2-nodream), verbatim. Read in English in Table 1.

Section 5.1 also summarizes the control run; this is the library it converged to. The five constructors differ in what is done to the private copy. What each does at a time step, with its arguments as free letters:

Symbol	What it does at each time step
<code>fn_6(a, g, δ)</code>	on a private copy of the grid altered in some unspecified way δ , a moves to the adjacent cell closest to g ; then a 's new position, and nothing else, is written back to the world
<code>fn_7(a, g, δ)</code>	the alteration δ is made to the grid itself, and then a moves to the adjacent cell closest to g — no copy is opened
<code>fn_8(d, b, a, n)</code>	n moves to the adjacent cell closest to b ; then, on a private copy in which b has been displaced one cell in direction d , a moves to the adjacent cell closest to b ; then a 's position alone is written back
<code>fn_9(u, v)</code>	on a private copy: u moves to the adjacent cell closest to v and then to the adjacent cell closest to the nearest 0, and u 's position is written into that copy; still in the copy, v then moves to the adjacent cell closest to u ; finally v 's position alone is written back to the world
<code>fn_10(c, g, a, g', n)</code>	on a private copy with an impassable wall stamped in row 2 at column c , a moves to the adjacent cell closest to g , and a 's position alone is written back; then, in the world, n moves to the adjacent cell closest to g'
<code>fn_11(c, r, g, a)</code>	on a private copy with an impassable wall stamped at row r , column c , a moves to the adjacent cell closest to g ; then a 's position alone is written back

Table 9: The atomic control's five constructors and its seek block, read out. The bodies these are read from are below.

So what the private copy is wrong about is left an arbitrary hole in `fn_6`, is a shove of the goal in `fn_8`, and is a phantom wall at a free coordinate in `fn_11` — the most-used, carrying 52 of the 155 belief solutions — and at a coordinate free only in its column in `fn_10`. `fn_9` is worth a remark: what it does to its private copy itself opens a second private copy, so it is an agent whose private model is itself derived by copying and altering, and fourteen solutions are rewritten through it. That is not second-order mentalizing in the sense of Section 1.2, since the inner copy is part of the outer agent's model rather than a model of another mind, but it does establish that the discovered shape embeds, one of the properties Section 1.3 asked for.

Symbol	Body	Belief solves
fn_6	(fork (compose \$2 (optimize (neg_dist \$1) \$0)) (sync_to_world \$0))	37
fn_7	(compose \$2 (optimize (neg_dist \$1) \$0))	18
fn_8	(compose (optimize (neg_dist \$1) \$3) (fork (compose (step \$1 \$0) (optimize (neg_dist \$1) \$2)) (sync_to_world \$2)))	24
fn_9	(fork (compose (fork (compose (optimize (neg_dist \$1) \$0) (optimize (neg_dist 0) \$0)) (sync_to_world \$0)) (optimize (neg_dist \$0) \$1)) (sync_to_world \$1))	14
fn_10	(compose (fork (compose (wall_at c2 \$0) (optimize (neg_dist \$1) \$2)) (sync_to_world \$2)) (optimize (neg_dist \$3) \$4))	29
fn_11	(fork (compose (wall_at \$1 \$0) (optimize (neg_dist \$2) \$3)) (sync_to_world \$3))	52

Table 10: The final library under the atomic control (run `p1-nodream`), bodies expanded to base primitives; the readings are above. Five of the six carry the agency signature; `fn_7` is the bare seek policy — move a toward g — with room for one edit in front of it, which belief shares with the obstacle family (47 obstacle solves are rewritten through it).

7.4. The per-family description-length census

Section 5.3 reports what the learned library buys each family. The full census, from which those figures are taken.

Family	Solves priced	Base (nats)	Final library	Saved
belief	25	34.51	17.40	17.11
obstacle	48	15.97	12.79	3.18
registration	4	6.80	4.50	2.30
perception	4	10.39	8.08	2.30
drift_reg	4	16.56	14.26	2.30
desire	16	7.78	7.78	0.00
comet	4	16.05	16.05	0.00
relocate	16	13.89	13.89	0.00
flee	4	7.78	7.78	0.00
deletion	4	5.70	5.70	0.00
denoise	4	4.79	4.79	0.00
wipe / map_update	4 each	5.78	5.78	0.00
multi_reg / reg_except / inpaint / readout	4 each	2.20–4.50	unchanged	0.00

Table 11: Median description length per task (`p2-nodream`), before and after the learned library. Belief saves 17.11 nats per task and 412 nats over the priced solves; obstacle saves through the shared seek `fn_8`, and the three registration-flavoured families each save exactly 2.30 nats — the price of the reassembled single-value publication.

7.5. The interpreter

The first is compositionality. “Iteratively apply f , starting from x_0 , for n time steps” is structure common to every solution in the corpus. Were it part of the program, it would be baked into every abstraction the learner discovers: a corpus of ten falling-object scenes would yield not the reusable $\lambda 0. (\text{step down } \#0)$ but an overspecified *apply (step down #0) from x_0 for #1 steps*, whose output type is an entire rendered scene. Such an abstraction cannot be combined with another, so it could never serve as a component of the deeper compounds that belief requires, and library learning would be defeated at the first round. Keeping the recurrence out of program space keeps every abstraction an arrow $G \rightarrow G$ and therefore composable.

The second is that the recurrence carries no information the learner could be credited with discovering. Since it is shared by every solution, what search is actually looking for is only f ; the rest is what is needed to render a scene from f in order to check it against the observation. So we move the rendering machinery out of program space and into the evaluation framework. The interpreter is given in full in Section 7.5.

The interpreter is discussed at the model level in Section 7.5. What follows is the design argument in the form it was originally worked out, followed by the two interpreters themselves.

Say we have a candidate program ρ , so the enumerator needs to check for each task x in the corpus whether ρ solves task x . A program ρ solves task x iff running ρ on initial frame x_0 outputs the entire history x . So every solution should be a program that takes as input a $d \times d$ frame and returns a $d \times d \times n$ stack of frames, a program of the form: “iteratively apply transition function f , starting with initial frame x_0 , for n timesteps”. So every solution program has the same outer structure, with only f varying between programs.

This presents a problem: this shared structure, present in every program, will be baked-in to every abstraction. For example say ten tasks in a corpus depict some object falling down, so stitch should identify *(step down #0)* as a shared structure and add this “gravity” primitive to the library. But because “apply f , starting with x_0 , for n timesteps” is a structure common to all programs, stitch would add an overspecified abstraction $f_1 := \lambda 0. 1. \text{iteratively apply (step down \#0), starting with } x_0, \text{ for \#1 timesteps}$ rather than the more general $f_2 := \lambda 0. (\text{step down } \#0)$.

The problem is that f_1 doesn’t compose, which defeats the purpose of library learning. Say we abstracted f_1 as “gravity”, and we now want to use it to solve a task involving two objects both dropping to the ground. We can’t do that, because f_1 ’s output is of type `matrix`, an entire 3d scene depicting just one falling object. But if we’ve got an f_2 abstraction with output type `fn`, we can create a program like *(compose (fn_2 4) (fn_2 5))* that reuses the abstraction for each component of a scene within that same scene (where `compose` is of type `fn -> fn -> fn`).

Since “iteratively apply transition function f , starting with initial frame x_0 , for n timesteps” is common to every program, what the enumerator is looking for is really just that transition function f . The rest is just what’s needed to render a scene given a transition function, to check whether f correctly produces a given task scene. So we simply move the rendering machinery from program space to the evaluation framework.

We implement an interpreter that, given a candidate program (transition function) f , a task's initial frame x_0 , and the task's duration (z -dimension) n , generates a scene x' by applying f iteratively (starting with x_0) for n timesteps.

```
def unfold(g: grid, T: int, f: fn) -> mat:
    frames = [g.copy()]
    for _ in range(T - 1):
        g = f(g)
        frames.append(g.copy())
    return np.stack(frames)
```

`unfold` is the anamorphism that at each timestep applies f to the previous grid to generate the new grid, and appends the new grid to the list of grids. So `unfold`'s state is just a single grid: it doesn't encode a world/model grid pair, nor a registry of agents and goals. A transition function f is a grid endomorphism `fn :: grid -> grid`.

For some of the tasks, we use an interpreter for programs of root `fn_p_g :: (grid, grid) -> grid` since those tasks don't involve just one input grid but a pair. That uses `unfold_with_template(g, template, T, c)`

```
def unfold_with_template(g, template, T, c):
    """grid, grid, int, fn_p_g -> mat: thread g; each frame pair it with a
    *constant external* template and apply commit c. Unlike fork, the second
    channel is a given input, not a privately derived model – so a program that
    uses sync_to_world here is doing registration, not holding a belief.
    """
    frames = [g.copy()]
    for _ in range(T - 1):
        g = c((g, template))
        frames.append(g.copy())
    return np.stack(frames)
```

7.6. The primitive catalogue

The primitive sets handed to the searcher are summarised at the model level in Section 4.4. The exhaustive catalogue — every symbol's repr, type signature and semantics, under each endowment — belongs here.

Symbol	Type	Semantics
compose	$(G \rightarrow G)^2 \rightarrow (G \rightarrow G)$	run one transition then the other
step	$V \times D \rightarrow (G \rightarrow G)$	move every cell of the given value one step in the direction
optimize	$U \times V \rightarrow (G \rightarrow G)$	move the given value one greedy (BFS-optimal) step to improve the utility
neg_dist	$V \rightarrow U$	minus the distance to the nearest cell of the target value (attract)
distance	$V \rightarrow U$	plus that distance, so maximising it flees the target
wall_at	$C \times C \rightarrow (G \rightarrow G)$	stamp a wall at a position
clear_at	$C \times C \rightarrow (G \rightarrow G)$	clear one cell
erase	$V \rightarrow (G \rightarrow G)$	remove every cell of a value
right left up down	D	the four movement directions
c0 ... c4	C	coordinate terminals, a pair naming a position (the invisible latent pool)
0 ... 9	V	cell-value terminals (content-priced when visible)

Table 12: The grid core: the primitives that move cell values about a single grid, shared by both endowments.

Symbol	Type	Semantics
dup	$G \rightarrow G \times G$	pair a grid with a copy of itself
mapsnd	$(G \rightarrow G) \rightarrow (G \times G \rightarrow G \times G)$	$\text{id} \times f$: act on the second channel
compose_gp	$(G \rightarrow G \times G) \times (G \times G \rightarrow G \times G) \rightarrow (G \rightarrow G \times G)$	compose a pair producer with a pair endomorphism
pipe_gpg	$(G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	produce a pair, then consume it
locate	$V \rightarrow (G \rightarrow C \times C)$	where a named value is in a grid
place	$V \rightarrow (G \times (C \times C) \rightarrow G)$	move a named value to a position
register	$(G \rightarrow C \times C) \times (G \times (C \times C) \rightarrow G) \rightarrow (G \times G \rightarrow G)$	$(w, m) \mapsto \text{plc}(w, \text{loc}(m))$: read a position off one channel, impose it on the other
overlay	$G \times G \rightarrow G$	union the channels, model wins ties — a <i>non-mental</i> commit
underlay	$G \times G \rightarrow G$	union the channels, world wins ties
sync_all	$G \times G \rightarrow G$	move <i>every</i> shared value to its model position
sync_except	$V \rightarrow (G \times G \rightarrow G)$	every shared value but one
fst / snd	$G \times G \rightarrow G$	project the world channel / the model channel
swap	$G \times G \rightarrow G \times G$	the symmetry isomorphism: exchange the channels
mapfst	$(G \rightarrow G) \rightarrow (G \times G \rightarrow G \times G)$	$f \times \text{id}$: act on the <i>first</i> channel
bimap	$(G \rightarrow G)^2 \rightarrow (G \times G \rightarrow G \times G)$	$f \times g$: act on both
pair_blank	$G \rightarrow G \times G$	pair a grid with a fresh blank rather than a copy
compose_pg	$(G \times G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \times G \rightarrow G)$	transform a pair, then consume it — the third composition, and the only one that acts on a pair the program did not build

Table 13: The product combinators: the plumbing, the commits that consume a pair, and the symmetric complements. The lower rows are the complements discussed in Section 4.4.

Symbol	Type	Semantics
<code>fork</code>	$(G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	the fork frame of Section 4.4.1, bought whole: copy the grid, run the derive on the copy, hand the (world, copy) pair to the commit
<code>sync_to_world</code>	$V \rightarrow (G \times G \rightarrow G)$	the single-value publication bought whole: move v in the world to its position in the model, return the world
<code>sync_to_model</code>	$V \rightarrow (G \times G \rightarrow G)$	the direction complement, also atomic here: move v in the model to its position in the world, return the model — perception rather than action
<code>then_sync</code>	$(G \times G \rightarrow G) \times V \rightarrow (G \times G \rightarrow G)$	run the given commit, then <code>sync_to_world</code> v on its result. Absent from the combinator endowment, where it would smuggle an atomic <code>sync_to_world</code> back in — with it, registration solves as <code>(then_sync fst_gg v)</code> and never reaches for the register/ locate/place spelling
<code>sync_all</code> / <code>sync_except</code>	$G \times G \rightarrow G$ / $V \rightarrow (G \times G \rightarrow G)$	the scope collapses, atomic in both endowments (Table 13)
<code>overlay</code> / <code>underlay</code>	$G \times G \rightarrow G$	the z-order unions, atomic in both endowments
<code>fst_gg</code> / <code>snd_gg</code>	$G \times G \rightarrow G$	the projections — the <code>fst</code> / <code>snd</code> of Section 4.4, under the searcher’s reprs — atomic in both endowments

Table 14: The atomic control’s pair interface, transcribed from `prims.py` (`make_symmetric_prims` with `decomposed` off). The grid core of Table 12 is shared unchanged. Relative to the combinator endowment of Table 13, every plumbing row — `dup`, `mapsnd`, `mapfst`, `bimap`, `swap`, `pair_blank`, `locate`, `place`, `register` and the three composers — is absent, and in their place stand the four tokens of the first four rows. This is the endowment Section 4.5.3 calls the atomic control, and the absence of the channel-mapping plumbing is why the three families that act on the channels of a given pair (`layer-compositing`, `drifting registration`, `map-update`) have no program under it.

7.7. Program representation

The hypothesis space is described at the model level in `!types`. This is the data structure that represents it.

Every program, primitive, and invented abstraction is a `Delta`, i.e. an object that can serve as a node in an expression tree. A `Delta` carries a `head` (the Python value or callable it denotes), a `type` (the type its subtree returns), a list of `tailtypes` (the types of the arguments it expects), and a list of `tails` (the actual argument subtrees, once they are filled in). So a `Delta` with no `tailtypes` is a *terminal*, a leaf whose value is just its head, for example `up :: dir` or the literal `4 :: cellvalue`. A `Delta` with `tailtypes` is an *operator*: an interior node like `step :: cellvalue, dir -> fn`, whose head is applied to its filled-in tails. Then a node's `arrow` is the signature that the enumerator matches against when it fills argument slots. For a terminal, its arrow is just its `type`. For an operator, it's the pair `(tailtypes, type)`, the arrow from its input types to its output types.

The expression tree is directly executable. Calling a `Delta` evaluates the program it represents, so a terminal returns its head while an operator recursively evaluates each of its tails and applies its head to the results. So the program `(step 4 up)` is a `step`-headed node with tails `4` and `up`; calling it evaluates the two leaves to the cell value `4` and the direction `(-1,0)`, then applies `step` to obtain a transition function `fn :: grid -> grid`.

An invented abstraction is represented as a `Delta` as well, but it's distinguished by a `hiddentail` which is the body of the abstraction. The `hiddentail` is itself a `Delta` tree over lower-level primitives, with numbered argument holes `$0`, `$1`, ... standing in for the abstraction's parameters. When such a node is called, its body is copied and each hole `$i` is replaced by the corresponding tail before the body is evaluated. This is what makes an abstraction cost a single token. The whole subtree it abbreviates is hidden inside the `hiddentail` and does not appear in the program that uses it.

The description length of a program is its node count, computed by `length`. a leaf counts one, and an operator counts one plus the lengths of its tails. So an invented abstraction used as a token adds just one, since its body lives in the un-charged `hiddentail`. This node count is the minimum-description-length quantity the enumerator uses to tiebreak between programs that solve a task within the same budget window (see `!enumeration`).

7.7.1. The library object

The library of primitives is a `Deltas` object. It is instantiated with a list of `core` primitives and grows a parallel list of `invented` abstractions as compression discovers them. When the enumerator draws primitives, it draws from the concatenation of `core` and `invented` which is the full library. Whenever a primitive is added, `Deltas.infer` rebuilds the indices the search depends on — most importantly `bytype`, which groups the alphabet's symbols by their return type. This grouping is what the type-uniform prior is computed from: a symbol of type τ is priced at $-\ln$ of the number of symbols sharing type τ , so `bytype` gives both the branching factor at each argument slot and the cost of each choice (`!enumeration`).

7.8. Task generators and necessity filters

The identifiability condition on the mental families is stated in identifiability beyond the k scenes, two conditions on the scene itself and a set of exemptions that no scene can settle. This section states them and says how the generators enforce them. Both conditions are sharpest in the false-wall family, where the agent detours around a wall that is not there.

The falsely believed wall must matter. Placed nowhere near the agent’s route, it leaves the agent navigating to its goal exactly as it would have without any false belief; positing the belief is then neither necessary nor helpful, and the scene shows nothing. So the phantom wall lies on the shortest path the agent would take were its belief true, and costs it extra steps over that path. A merely monotone “detour” is just another shortest path, which plain desire reproduces under different tie-breaking. Scenes failing this are rejected by the generator.

A world-level stamp must be detectable. This one is structural. On a bare grid nothing distinguishes a wall the agent *believes* in from a real wall stamped into the world and taken back down before the frame is rendered: a rival that stamps, acts, then erases reproduces every scene honestly, at every k . The scenes must therefore make the stamp visible, and they do so by leaving an inert object sitting on the phantom-wall cell for the whole trajectory — the cell the agent bends around in. Stamping a wall there overwrites that object, and nothing in the language can put it back, so a world-level stamp shows up in the rendered frame. The witness family makes the same argument with a moving object in place of a static one: an agent who walks across the phantom cell would be clobbered by any real wall stamped there.

Some rivals are identities, and are exempted by argument. A last few rivals cannot be excluded by the scenes at all, because what they are is an *identity* rather than a coincidence, and no number of scenes separates two programs that denote the same function on the scenes in question. On a two-value scene, “move every shared value except the goal to its model position” simply *is* “move the agent to its model position”. These are exempted from the rival census of Section 5.3.3 by argument, and the full set is:

7.9. Cost-bounded enumeration

The search is described at the model level in !enumeration. These are its mechanics.

While a single cost-ordered stream can in principle be checked against every task in the corpus at once, in the experiments each still-unsolved task is instead enumerated as its own cost-ordered stream, parallelized across tasks. The prior Q each stream runs under is the same one: the type-uniform prior over the current library, with the content-literal boost of !enumeration. Nothing in the loop conditions Q on the individual scene (!no-amortization).

Program search proceeds by budget window to ensure that cheaper programs are enumerated first. For (step 4 up), the first two windows are skipped by pruning. We start enumerating in window $[0, 2)$, but since `step`’s cost is 2.1 nats it exceeds the budget, so it cannot be chosen as the root node of the program tree and we proceed to budget window $[2, 4)$. After paying 2.1 nats for `step`, only 1.9 nats remain to fill the two holes in the partial program tree. `step`’s tail types (arguments) are `cellvalue` and `step`. But any primitive of type `cellvalue` costs 2.3 nats, so `step`’s first argument can’t

be placed. So trees with a **step** node are pruned, since they would never complete within the second budget window. Search is cheapest-first, since more complex programs only become affordable once the window is wide enough.

At budget window $[4, 6)$ the ceiling is finally sufficiently wide to admit all three symbols. The tree is assembled incrementally: each enumerated subtree is appended to a copy of its parent node, and a callback fills the remaining argument slots by the same recursion (so programs of any arity or nesting depth are handled by the same procedure). Since enumeration is type-directed, each argument slot draws candidates only from symbols of the matching type, so no ill-typed partial tree is ever constructed. (**step 4 up**) completes at its total cost of ≈ 5.8 nats.

When (**step 4 up**) is found the tree rooted by **step** has no more available holes, so the callback fires to evaluate the found program ρ . For each task x in the corpus, the interpreter generates the output matrix given the input grid of the task by calling `unfold` on ρ , initial grid x_0 , and x 's third dimension n . Then the resulting output matrix x' is hashed and compared against task x 's hash. So for a program to solve a task, it must produce a grid that exactly matches that task's entire trajectory, not just the endpoint of the trajectory. This ensures that programs capture the full behavior rather than just the end result.

If (**step 4 up**) is the first solution found for the task, it's stored as the task's solution. Because the windows are tried in increasing order of cost, the first solution the search returns is in principle the shortest program that solves the task. But enumeration is parallelized so search unfolds simultaneously over program trees, so we need to tiebreak when multiple programs solve a task within the same budget window. To tiebreak we first run `Delta.simplify` on the programs, a semantics-preserving rewrite that collapses spurious nesting which sometimes occurs during enumeration. We then run `Delta.length` to get the node count of the expression trees we need to compare. Every operator and leaf adds to the count, and an invented abstraction used as a token adds one to the count too. The body of the abstraction isn't charged, since the components now count for just application. The program with the lower node count wins since it has the lower minimum description length (since by symbol count it's shorter).

Also, the search space contains lots of junk programs that crash or return nothing even though they type-check. So the enumerator callback wraps both the interpreter and the evaluator in `try/except` blocks and skips any degenerate or non-callable program.

7.10. Compression with Stitch

The compression objective and the tradeoff it makes are given in `!compression`. This is the search that optimizes it.

In DreamCoder (Ellis et al., 2020) the compression step is bottom-up: for each found program it enumerates the exponential set of program refactorings (stored as version spaces) and intersects them to grow abstractions up from reoccurring subexpressions. Rather than this bottom-up approach, we delegate the search for abstractions to the top-down Stitch (Bowers et al., 2023) library-learning system. Instead of enumerating refactorings of every program, stitch searches the space of abstractions directly. So stitch doesn't have to materialize every rewrite.

To create an abstraction, stitch starts with a tree that consists of just a root node `??`, an unexpanded hole that the search still needs to refine. At the root `??`, the abstraction *matches* every subexpression, since it’s the most general abstraction possible: any subexpression can be trivially “rewritten”

. As the tree is built up and the search is refined, the set of possible match locations shrinks.

A node of the stitch abstraction search tree is a partial abstraction. It consists of some combination of primitive operations (like `step`) and argument holes `#i`, and at least one unexpanded hole `??` that the search still needs to refine.

At each step it commits part of the abstraction’s body to a concrete operator or splits a subtree, branching downward toward more specific abstractions.

upper bound for a partial abstraction $A_{??}$ (Bowers et al., 2023)

$$U_{\text{upperbound}}(A_{??}) = \sum_{e \in \text{matches}(A_{??})} \text{size}(e)$$

goal of compression is to minimize the size (cost) of program corpus after rewrite. Compressive utility function:

Each compression phase consists of a fixed number of stitch iterations, determined by the `iterations` parameter. Each iteration searches for the single most compressive abstraction, rewrites the corpus in terms of it, and repeats, so later abstractions may be built out of earlier ones. The `max_arity` parameter bounds how many holes (`#0`, `#1`, ...) an abstraction may carry.

7.10.1. Normalization before compression

Before passing the found solutions to stitch for abstraction, we first need to fully expand any existing primitives in those solutions. For this we implement `normalize`, the complementary operation to abstraction. It fully inlines every invented `fn_k` back to the bare primitive alphabet, recursively substituting each abstraction’s argument holes with its actual arguments until nothing but core primitives remain.

We need to normalize because stitch’s top-down compression can only abstract over structure it can see, and an opaque `fn_k` token has no visible interior. So any abstraction that would need to reach inside `fn_k`, or span its boundary and surrounding context, is simply invisible to the search.

the compression step must be handed programs in a common vocabulary — feeding it already-compressed programs, in which a prior round’s `fn_0` appears as an opaque token, would let those tokens collide with the fresh abstractions the compressor invents under the same names. Normalizing before compression guarantees the compressor sees only primitives and the structure it discovers is genuinely its own. Renaming it to `inv_0` avoids the lexical clash but leaves it just as opaque; stitch still can’t see the `(compose (wall_at ...)) (optimize ...)` sitting inside it. Inlining flattens the token back to that primitive structure, and only then can the compressor find abstractions that cut across the old boundaries.

The best syntactic fragment to abstract from round N ’s solution may carve the primitives differently than round $N + 1$ ’s. For example,

So we can refactor abstractions that were previously ideal but are no longer ideal

7.10.2. No recognition model

DreamCoder’s third phase trains a neural recognition model that reads a task and emits a task-conditional prior $Q(\cdot | x)$, so that enumeration proceeds under a Q biased toward the primitives. We don’t implement it.

The reason is a difference of scale between DreamCoder’s setting and this one. Amortization earns its place when the branching factor is large and the loop is long: DreamCoder works with libraries of hundreds of symbols over dozens of iterations, and there the cost of one forward pass per task is repaid many times over by the cost bands it lets the enumerator skip. Ours is a small language run for a few rounds. The learner starts with 37 primitives under the atomic control and 45 under the combinator library, and ends with 43 and 51 once compression has added its six; and the abstraction the whole argument turns on enters at the first compression step under the one endowment and the second under the other, with the loop converging a round or two later (Section 5.4). There is no long tail of iterations across which a learned proposal could amortize anything. What such a proposal would buy is reaching sooner what is already reachable, and reachability, not speed, is the quantity the argument of Section 2.4 depends on.

Omitting the phase also removes it as a place where the result could be smuggled in. A recognition model trained on solved tasks is a component whose prior over the library is fit to data, and belief is precisely the structure the corpus is meant to leave unfitted. With no such component, no distribution anywhere in the loop is conditioned on what a scene looks like, and the audit of Section 4.6 has one fewer item to defend.

This is a departure from DreamCoder, and it is one we arrived at by measurement rather than by design: the loop was run both ways before it was settled, and Section 5.10 reports what each way gave.

The second is to abandon enumeration in prior order for enumeration under a learned proposal distribution, trained to guess which primitives a task of this appearance is likely to need. This is amortized inference. The system this thesis builds uses only the first, for reasons given in Section 7.10.2; it is the first that does the work the argument needs, since it is the one that changes what is *reachable* rather than merely what is reached sooner — an intervention on the problem rather than on the procedure for solving it.

Bibliography

- Apperly, I. A., & Butterfill, S. A. (2009). Do humans have two systems to track beliefs and belief-like states?. *Psychological Review*, 116(4), 953–970. <https://doi.org/10.1037/a0016923>
- Baker, C. L. (2012). *Bayesian Theory of Mind: Modeling Human Reasoning about Beliefs, Desires, Goals, and Social Relations* [Doctoral dissertation].
- Baker, C. L., Jara-Ettinger, J., Saxe, R., & Tenenbaum, J. B. (2017). Rational quantitative attribution of beliefs, desires and percepts in human mentalizing. *Nature Human Behaviour*, 1(4), 64. <https://doi.org/10.1038/s41562-017-0064>
- Baker, C. L., Saxe, R., & Tenenbaum, J. B. (2009). Action understanding as inverse planning. *Cognition*, 113(3), 329–349. <https://doi.org/10.1016/j.cognition.2009.07.005>
- Baron-Cohen, S., Leslie, A. M., & Frith, U. (1985). Does the autistic child have a “theory of mind” ?. *Cognition*, 21(1), 37–46. [https://doi.org/10.1016/0010-0277\(85\)90022-8](https://doi.org/10.1016/0010-0277(85)90022-8)
- Barone, P., Corradi, G., & Gomila, A. (2019). Infants' performance in spontaneous-response false belief tasks: A review and meta-analysis. *Infant Behavior and Development*, 57, 101350. <https://doi.org/10.1016/j.infbeh.2019.101350>
- Battaglia, P. W., Hamrick, J. B., & Tenenbaum, J. B. (2013). Simulation as an engine of physical scene understanding. *Proceedings of the National Academy of Sciences*, 110(45), 18327–18332. <https://doi.org/10.1073/pnas.1306572110>
- Baum, E. B. (2004). *What is thought?*. MIT Press.
- Beaudoin, C., Leblanc, É., Gagner, C., & Beauchamp, M. H. (2020). Systematic Review and Inventory of Theory of Mind Measures for Young Children. *Frontiers in Psychology*, 10, 2905. <https://doi.org/10.3389/fpsyg.2019.02905>
- Berthiaume, V. G., Shultz, T. R., & Onishi, K. H. (2013). A constructivist connectionist model of transitions on false-belief tasks. *Cognition*, 126(3), 441–458. <https://doi.org/10.1016/j.cognition.2012.11.005>
- Berwick, R. C., Pietroski, P., Yankama, B., & Chomsky, N. (2011). Poverty of the Stimulus Revisited. *Cognitive Science*, 35(7), 1207–1242. <https://doi.org/10.1111/j.1551-6709.2011.01189.x>
- Bowers, M., Olausson, T. X., Wong, L., Grand, G., Tenenbaum, J. B., Ellis, K., & Solar-Lezama, A. (2023). Top-Down Synthesis for Library Learning. *Proceedings of the ACM on Programming Languages*, 7(POPL), 1182–1213. <https://doi.org/10.1145/3571234>
- Bruner, J. S., Goodnow, J. J., & Austin, G. A. (1986). *A study of thinking*. Transaction Books.
- Buttelmann, D., Carpenter, M., & Tomasello, M. (2009). Eighteen-month-old infants show false belief understanding in an active helping paradigm. *Cognition*, 112(2), 337–342. <https://doi.org/10.1016/j.cognition.2009.05.006>
- Butterfill, S. A., & Apperly, I. A. (2013). How to Construct a Minimal Theory of Mind. *Mind & Language*, 28(5), 606–637. <https://doi.org/10.1111/mila.12036>
- Carey, S. (2011). *The Origin of Concepts*. Oxford University Press USA - OSO.

- Chater, N., & Vitányi, P. (2003). Simplicity: a unifying principle in cognitive science?. *Trends in Cognitive Sciences*, 7(1), 19–22. [https://doi.org/10.1016/S1364-6613\(02\)00005-0](https://doi.org/10.1016/S1364-6613(02)00005-0)
- Chomsky, N. (1975). *Reflections on Language*. Pantheon Books. <https://books.google.nl/books?id=R78kAQAAMAAJ>
- Chomsky, N. (2014). *Aspects of the Theory of Syntax* (50th ed). MIT Press.
- Churchland, P. M. (1981). Eliminative Materialism and the Propositional Attitudes. *The Journal of Philosophy*, 78(2), 67. <https://doi.org/10.2307/2025900>
- Coşkun, K., Kara, C., & Coskun, M. (2020). New Scaling of Theory of Mind Tasks: Where Can the Intentional Aspect Be Scaled?. *Activitas Nervosa Superior*, 62(3), 78–87. <https://doi.org/10.1007/s41470-020-00072-2>
- Crane, T. (n.d.). *The Mechanical Mind: A Philosophical Introduction to Minds, Machines and Mental Representation, Second Edition*.
- Csibra, G., & Gergely, G. (2007). ‘Obsessed with goals’: Functions and mechanisms of teleological interpretation of actions in humans. *Acta Psychologica*, 124(1), 60–78. <https://doi.org/10.1016/j.actpsy.2006.09.007>
- Dechter, E., Malmaud, J., Adams, R. P., & Tenenbaum, J. B. (n.d.). *Bootstrap learning via modular concept discovery*.
- Dennett, D. C. (2002). *The Intentional Stance* (8. pr.). MIT Press.
- Downes, S. M. (1999). Can Scientific Development and Children's Cognitive Development Be the Same Process?. *Philosophy of Science*, 66(4), 565–578. <https://doi.org/10.1086/392755>
- Dörrenberg, S., Rakoczy, H., & Liszkowski, U. (2018). How (not) to measure infant Theory of Mind: Testing the replicability and validity of four non-verbal measures. *Cognitive Development*, 46, 12–30. <https://doi.org/10.1016/j.cogdev.2018.01.001>
- Ellis, K., Wong, C., Nye, M., Sable-Meyer, M., Cary, L., Morales, L., Hewitt, L., Solar-Lezama, A., & Tenenbaum, J. B. (2020, June). *DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning*. arXiv. <https://doi.org/10.48550/arXiv.2006.08381>
- Feldman, J. (2000). Minimization of Boolean complexity in human concept learning. *Nature*, 407(6804), 630–633. <https://doi.org/10.1038/35036586>
- Flener, P., & Schmid, U. (2008). An introduction to inductive programming. *Artificial Intelligence Review*, 29(1), 45–62. <https://doi.org/10.1007/s10462-009-9108-7>
- Fodor, J. (1992). A theory of the child's theory of mind. *Cognition*, 44(3), 283–296. [https://doi.org/10.1016/0010-0277\(92\)90004-2](https://doi.org/10.1016/0010-0277(92)90004-2)
- Fodor, J. A. (1998). *Concepts: Where Cognitive Science Went Wrong* (1st ed.). Oxford University PressOxford. <https://doi.org/10.1093/0198236360.001.0001>
- Fodor, J. A. (2010). *The Language of Thought* (Digit. repr). Harvard Univ. Pr.
- Fodor, J. A., & Pylyshyn, Z. W. (n.d.). *Connectionism and cognitive architecture: A critical analysis*.

- Gergely, G., & Csibra, G. (2003). Teleological reasoning in infancy: the naïve theory of rational action. *Trends in Cognitive Sciences*, 7(7), 287–292. [https://doi.org/10.1016/S1364-6613\(03\)00128-1](https://doi.org/10.1016/S1364-6613(03)00128-1)
- Gergely, G., Nádasdy, Z., Csibra, G., & Bíró, S. (1995). Taking the intentional stance at 12 months of age. *Cognition*, 56(2), 165–193. [https://doi.org/10.1016/0010-0277\(95\)00661-H](https://doi.org/10.1016/0010-0277(95)00661-H)
- Goldman, A. I. (2006). *Simulating minds: the philosophy, psychology, and neuroscience of mindreading*. Oxford University press.
- Goodman, N. (1983). *Fact, fiction, and forecast* (4. ed). Harvard Univ. Pr.
- Goodman, N. D., Baker, C. L., Bonawitz, E. B., Mansinghka, V. K., Gopnik, A., Wellman, H., Schulz, L., & Tenenbaum, J. B. (n.d.). *Intuitive Theories of Mind: A Rational Approach to False Belief*.
- Goodman, N. D., Tenenbaum, J. B., & Gerstenberg, T. (2014). *Concepts in a Probabilistic Language of Thought*.
- Goodman, N. D., Tenenbaum, J. B., Feldman, J., & Griffiths, T. L. (2008). A Rational Analysis of Rule-Based Concept Learning. *Cognitive Science*, 32(1), 108–154. <https://doi.org/10.1080/03640210701802071>
- Goodman, N. D., Ullman, T. D., & Tenenbaum, J. B. (2011). Learning a theory of causality. *Psychological Review*, 118(1), 110–119. <https://doi.org/10.1037/a0021336>
- Gopnik, A. (1997). *Words, thoughts, and theories*. MIT Press.
- Gopnik, A. (2012). Scientific Thinking in Young Children: Theoretical Advances, Empirical Research, and Policy Implications. *Science*, 337(6102), 1623–1627. <https://doi.org/10.1126/science.1223416>
- Gopnik, A., & Tenenbaum, J. B. (2007). Bayesian networks, Bayesian learning and cognitive development. *Developmental Science*, 10(3), 281–287. <https://doi.org/10.1111/j.1467-7687.2007.00584.x>
- Gopnik, A., & Wellman, H. M. (1992). Why the Child's Theory of Mind Really Is a Theory. *Mind & Language*, 7(1–2), 145–171. <https://doi.org/10.1111/j.1468-0017.1992.tb00202.x>
- Gopnik, A., & Wellman, H. M. (2012). Reconstructing constructivism: Causal models, Bayesian learning mechanisms, and the theory theory. *Psychological Bulletin*, 138(6), 1085–1108. <https://doi.org/10.1037/a0028044>
- Gordon, R. M. (1986). Folk Psychology as Simulation. *Mind & Language*, 1(2), 158–171. <https://doi.org/10.1111/j.1468-0017.1986.tb00324.x>
- Griffiths, T. L., Chater, N., & Tenenbaum, J. B. (2024). *Bayesian models of cognition: reverse engineering the mind*. The MIT Press.
- Griffiths, T. L., Chater, N., Kemp, C., Perfors, A., & Tenenbaum, J. B. (2010). Probabilistic models of cognition: exploring representations and inductive biases. *Trends in Cognitive Sciences*, 14(8), 357–364. <https://doi.org/10.1016/j.tics.2010.05.004>

- Griffiths, T. L., Sobel, D. M., Tenenbaum, J. B., & Gopnik, A. (2011). Bayes and Blickets: Effects of Knowledge on Causal Induction in Children and Adults: Cognitive Science. *Cognitive Science*, 35(8), 1407–1455. <https://doi.org/10.1111/j.1551-6709.2011.01203.x>
- Gulwani, S., Polozov, O., & Singh, R. (2017). *Program synthesis* (Issue 4.2017, 1-2). Now Publishers.
- Heider, F., & Simmel, M. (1944). An Experimental Study of Apparent Behavior. *The American Journal of Psychology*, 57(2), 243. <https://doi.org/10.2307/1416950>
- Heyes, C. M. (2018). *Cognitive gadgets: the cultural evolution of thinking*. Harvard University press.
- Heyes, C. M., & Frith, C. D. (2014). The cultural evolution of mind reading. *Science*, 344(6190), 1243091. <https://doi.org/10.1126/science.1243091>
- Jara-Ettinger, J., & Rubio-Fernandez, P. (2021). Quantitative mental state attributions in language understanding. *Science Advances*, 7(47), eabj970. <https://doi.org/10.1126/sciadv.abj0970>
- Jaynes, E. T., & Bretthorst, G. L. (2003). *Probability theory: the logic of science*. Cambridge university press.
- Kemp, C., Tenenbaum, J. B., Niyogi, S., & Griffiths, T. L. (2010). A probabilistic model of theory formation. *Cognition*, 114(2), 165–196. <https://doi.org/10.1016/j.cognition.2009.09.003>
- Kulke, L., Von Duhn, B., Schneider, D., & Rakoczy, H. (2018). Is Implicit Theory of Mind a Real and Robust Phenomenon? Results From a Systematic Replication Study. *Psychological Science*, 29(6), 888–900. <https://doi.org/10.1177/0956797617747090>
- Lake, B. M., Ullman, T. D., Tenenbaum, J. B., & Gershman, S. J. (2016, November). *Building Machines That Learn and Think Like People*. arXiv. <https://doi.org/10.48550/arXiv.1604.00289>
- Leslie, A. M. (n.d.). *Pretense and Representation: The Origins of "Theory of Mind"*.
- Leslie, A. M. (1994). Pretending and believing: issues in the theory of ToMM. *Cognition*, 50(1–3), 211–238. [https://doi.org/10.1016/0010-0277\(94\)90029-9](https://doi.org/10.1016/0010-0277(94)90029-9)
- Leslie, A. M. (2005). Developmental parallels in understanding minds and bodies. *Trends in Cognitive Sciences*, 9(10), 459–462. <https://doi.org/10.1016/j.tics.2005.08.002>
- Liang, P., Jordan, M. I., & Klein, D. (n.d.). *Learning Programs: A Hierarchical Bayesian Approach*.
- Luo, Y., & Baillargeon, R. (2010). Toward a Mentalistic Account of Early Psychological Reasoning. *Current Directions in Psychological Science*, 19(5), 301–307. <https://doi.org/10.1177/0963721410386679>
- MacKay, D. J. C. (n.d.). *Information Theory, Inference, and Learning Algorithms*.
- Margolis, E., Samuels, R., & Stich, S. P. (Eds.). (2012). *The Oxford handbook of philosophy of cognitive science*. Oxford University Press. <https://doi.org/10.1093/oxfordhb/9780195309799.001.0001>
- Marr, D. (2010). *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. The MIT Press. <https://doi.org/10.7551/mitpress/9780262514620.001.0001>
- McClelland, J. L. (2010). Emergence in Cognitive Science. *Topics in Cognitive Science*, 2(4), 751–770. <https://doi.org/10.1111/j.1756-8765.2010.01116.x>

- Milligan, K., Astington, J. W., & Dack, L. A. (2007). Language and Theory of Mind: Meta-Analysis of the Relation Between Language Ability and False-belief Understanding. *Child Development*, 78(2), 622–646. <https://doi.org/10.1111/j.1467-8624.2007.01018.x>
- Newcombe, N. S. (2011). What Is Neoconstructivism?. *Child Development Perspectives*, 5(3), 157–160. <https://doi.org/10.1111/j.1750-8606.2011.00180.x>
- Newell, A., Shaw, J. C., & Simon, H. A. (1958). Elements of a theory of human problem solving. *Psychological Review*, 65(3), 151–166. <https://doi.org/10.1037/h0048495>
- Onishi, K. H., & Baillargeon, R. (2005). Do 15-Month-Old Infants Understand False Beliefs?. *Science*, 308(5719), 255–258. <https://doi.org/10.1126/science.1107621>
- Palmarini, A. B., Lucas, C. G., & Siddharth, N. (2024, May). *Bayesian Program Learning by Decompiling Amortized Knowledge*. arXiv. <https://doi.org/10.48550/arXiv.2306.07856>
- Perfors, A., Tenenbaum, J. B., & Wonnacott, E. (2010). Variability, negative evidence, and the acquisition of verb argument constructions. *Journal of Child Language*, 37(3), 607–642. <https://doi.org/10.1017/S0305000910000012>
- Perner, J. (1991). *Understanding the Representational Mind*. MIT Press.
- Perner, J., & Wimmer, H. (1985). “John thinks that Mary thinks that...” attribution of second-order beliefs by 5- to 10-year-old children. *Journal of Experimental Child Psychology*, 39(3), 437–471. [https://doi.org/10.1016/0022-0965\(85\)90051-7](https://doi.org/10.1016/0022-0965(85)90051-7)
- Piantadosi, S. T., Goodman, N. D., Ellis, B. A., & Tenenbaum, J. B. (n.d.). *A Bayesian Model of the Acquisition of Compositional Semantics*.
- Piantadosi, S. T., Tenenbaum, J. B., & Goodman, N. D. (2012). Bootstrapping in a language of thought: A formal model of numerical concept learning. *Cognition*, 123(2), 199–217. <https://doi.org/10.1016/j.cognition.2011.11.005>
- Quine, W. V. (2013). *Word and object* (New ed). MIT Press.
- Rakoczy, H. (2022). Foundations of theory of mind and its development in early childhood. *Nature Reviews Psychology*, 1(4), 223–235. <https://doi.org/10.1038/s44159-022-00037-z>
- Ruffman, T. (2014). To belief or not belief: Children’s theory of mind. *Developmental Review*, 34(3), 265–293. <https://doi.org/10.1016/j.dr.2014.04.001>
- Rule, J. S. (n.d.). *Pre ‘cis of The child as hacker: Building more human-like models of learning*.
- Rumelhart, D. E. (1986). *Parallel Distributed Processing: Explorations in the Microstructure of Cognition: Foundations* (Issue v.1). MIT Press.
- Schmid, U., Kitzelmann, E., & Plasmeijer, R. (n.d.). *AAIP 2009 Proceedings of the ACM SIGPLAN Workshop*.
- Schmidt, L. A. (n.d.). *Meaning and compositionality as statistical induction of categories and constraints*.
- Scholl, B. J., & Leslie, A. M. (2001). Minds, Modules, and Meta-Analysis. *Child Development*, 72(3), 696–701. <https://doi.org/10.1111/1467-8624.00308>

- Schulz, L. (2012). The origins of inquiry: inductive inference and exploration in early childhood. *Trends in Cognitive Sciences*, 16(7), 382–389. <https://doi.org/10.1016/j.tics.2012.06.004>
- Sellars, W. (1956). *EMPIRICISM AND THE PHILOSOPHY OF MIND*.
- Shannon, C. E. (n.d.). *A Mathematical Theory of Communication*.
- Siskind, J. M. (n.d.). *A computational study of cross-situational techniques for learning word-to-meaning mappings*.
- Southgate, V., Senju, A., & Csibra, G. (2007). Action Anticipation Through Attribution of False Belief by 2-Year-Olds. *Psychological Science*, 18(7), 587–592. <https://doi.org/10.1111/j.1467-9280.2007.01944.x>
- Spelke, E. S. (2022). *What babies know: core knowledge and composition, volume 1* (First Edition). Oxford University Press. <https://doi.org/10.1093/oso/9780190618247.001.0001>
- Spelke, E. S., & Kinzler, K. D. (2007). Core knowledge. *Developmental Science*, 10(1), 89–96. <https://doi.org/10.1111/j.1467-7687.2007.00569.x>
- Tenenbaum, J. B., Kemp, C., Griffiths, T. L., & Goodman, N. D. (2011). How to Grow a Mind: Statistics, Structure, and Abstraction. *Science*, 331(6022), 1279–1285. <https://doi.org/10.1126/science.1192788>
- Ullman, T. D., & Tenenbaum, J. B. (n.d.). *Bayesian Models of Conceptual Development: Learning as Building Models of the World*.
- Ullman, T. D., Goodman, N. D., & Tenenbaum, J. B. (n.d.). *Theory Acquisition as Stochastic Search*.
- Villiers, J. G. de, & College, S. (n.d.). *Can Language Acquisition give Children a Point-of-View?*.
- Wellman, H. M. (2018). Theory of mind: The state of the art. *European Journal of Developmental Psychology*, 15(6), 728–755. <https://doi.org/10.1080/17405629.2018.1435413>
- Wellman, H. M., & Gelman, S. A. (1992). Cognitive Development: Foundational Theories of Core Domains. In *Annual review of psychology. Volume 43, 1992: Annual review of psychology. Volume 43, 1992*. Annual Reviews Inc.
- Wellman, H. M., & Liu, D. (2004). Scaling of Theory-of-Mind Tasks. *Child Development*, 75(2), 523–541. <https://doi.org/10.1111/j.1467-8624.2004.00691.x>
- Wellman, H. M., Carey, S., Gleitman, L., Newport, E. L., & Spelke, E. S. (1990). *The Child's Theory of Mind*. The MIT Press. <https://doi.org/10.7551/mitpress/1811.001.0001>
- Wellman, H. M., Cross, D., & Watson, J. (2001). Meta-Analysis of Theory-of-Mind Development: The Truth about False Belief. *Child Development*, 72(3), 655–684. <https://doi.org/10.1111/1467-8624.00304>
- Wellman, H. M., Kushnir, T., Xu, F., & Brink, K. A. (2016). Infants Use Statistical Sampling to Understand the Psychological World. *Infancy*, 21(5), 668–676. <https://doi.org/10.1111/inf.12131>
- Wimmer, H., & Perner, J. (n.d.). *Beliefs about beliefs: Representation and constraining function of wrong beliefs in young children's understanding of deception*.

- Woodward, A. (1998). Infants selectively encode the goal object of an actor's reach. *Cognition*, 69(1), 1–34. [https://doi.org/10.1016/S0010-0277\(98\)00058-4](https://doi.org/10.1016/S0010-0277(98)00058-4)
- Xu, F. (2019). Towards a rational constructivist theory of cognitive development. *Psychological Review*, 126(6), 841–864. <https://doi.org/10.1037/rev0000153>
- Xu, F., & Kushnir, T. (2013). Infants Are Rational Constructivist Learners. *Current Directions in Psychological Science*, 22(1), 28–32. <https://doi.org/10.1177/0963721412469396>
- Xu, F., Kushnir, T., & Benson, J. B. (2012). *Rational constructivism in cognitive development* (Issue 43). Academic Press.
- Zettlemoyer, L. S., & Collins, M. (n.d.). *Learning to Map Sentences to Logical Form: Structured Classification with Probabilistic Categorical Grammars*.